



Brain-Computer Interface (BCI) – A tutorial

Jaime A. Riascos-Salas

Potsdam Embodied Cognition Group (PECoG)

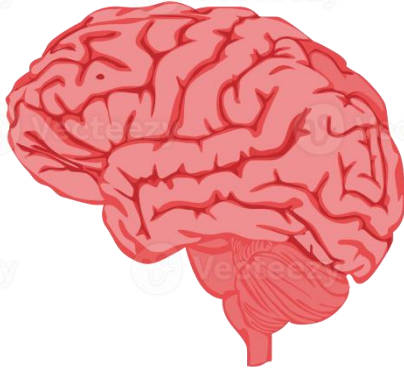
Department of Psychology

Today's plan

- Brain-Computer Interface (BCI)
 - Overview
 - Stages - Pipeline
 - Applications
 - Neurophysiological patterns, Categorization, Paradigms
- Create the experiment
 - Psychopy
 - Labstreaminglayer
- Pipeline
 - Signal processing
 - Feature extraction
 - Classification
 - Evaluation
- Hands on

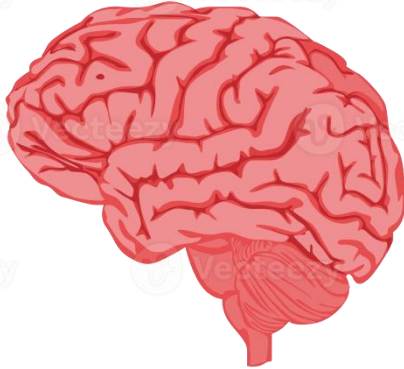
Brain-Computer Interface (BCI)...

What is our first thought when we heard this ?



Brain-Computer Interface (BCI)...

What is our first thought when we heard this ?





Formal (working) Brain-Computer Interface (BCI) definition



About ▾

Membership ▾

BCI Meeting ▾

BCI Thursdays: Online Events ▾

Resources ▾

Connect with us

BCI Working Definition:

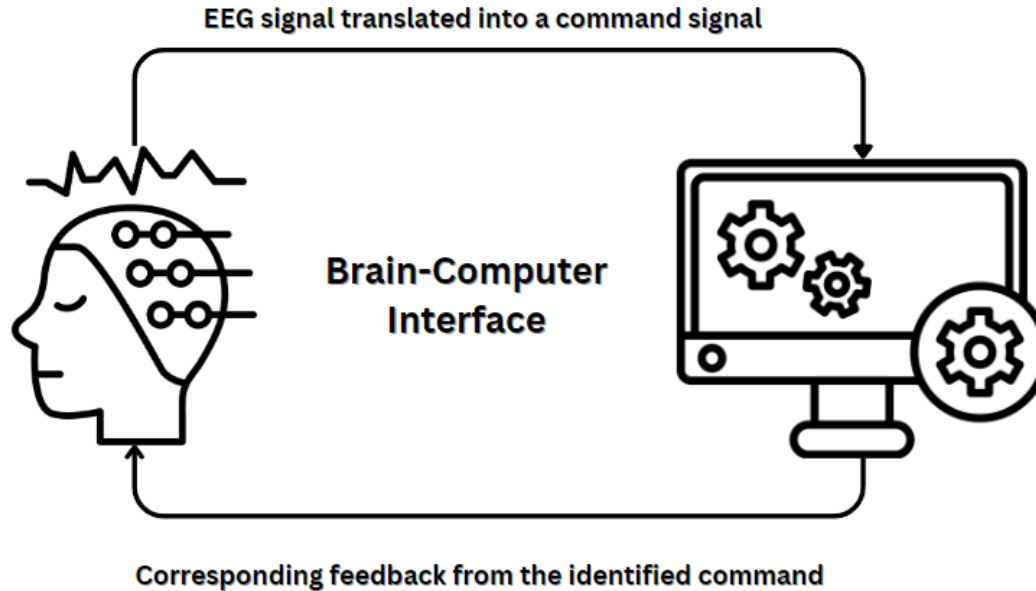
A brain-computer interface is a system that measures brain activity and converts it in (nearly) real-time into functionally useful outputs to replace, restore, enhance, supplement, and/or improve the natural outputs of the brain, thereby changing the ongoing interactions between the brain and its external or internal environments. It may additionally modify brain activity using targeted delivery of stimuli to create functionally useful inputs to the brain.

Key terms:

- “Brain” includes the cerebrum, the diencephalon, the brainstem, and the cerebellum. It does not include the cranial nerves. Furthermore, it does not include the measurement of intention or information (that could have originated in the brain) obtained from downstream areas of the nervous system (e.g. peripheral nerves or muscles).
- “Computer” may refer to a device typically referred to as a computer, but also to other computerized devices, such as robot arms, electrical stimulation devices, powered wheelchairs etc.
- The terms “internal”, “external”, “input”, and “output” should be read in relation to the brain as defined above.

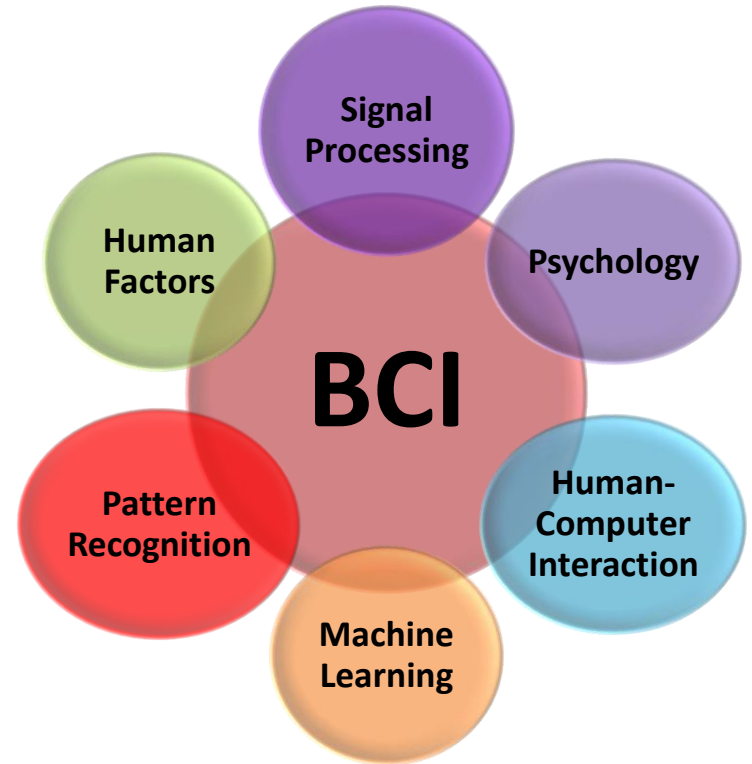
Recent definition: [BCI Society, <https://bcisociety.org/bci-definition/>, 2024]

Formal (working) Brain-Computer Interface (BCI) definition



BCI overview

BCI is the technology that seeks to enable alternative communication with machines or devices; this is done using the translation of brain signals elicited during a specific task into command outputs (Neuper and Pfurtscheller, 2010).

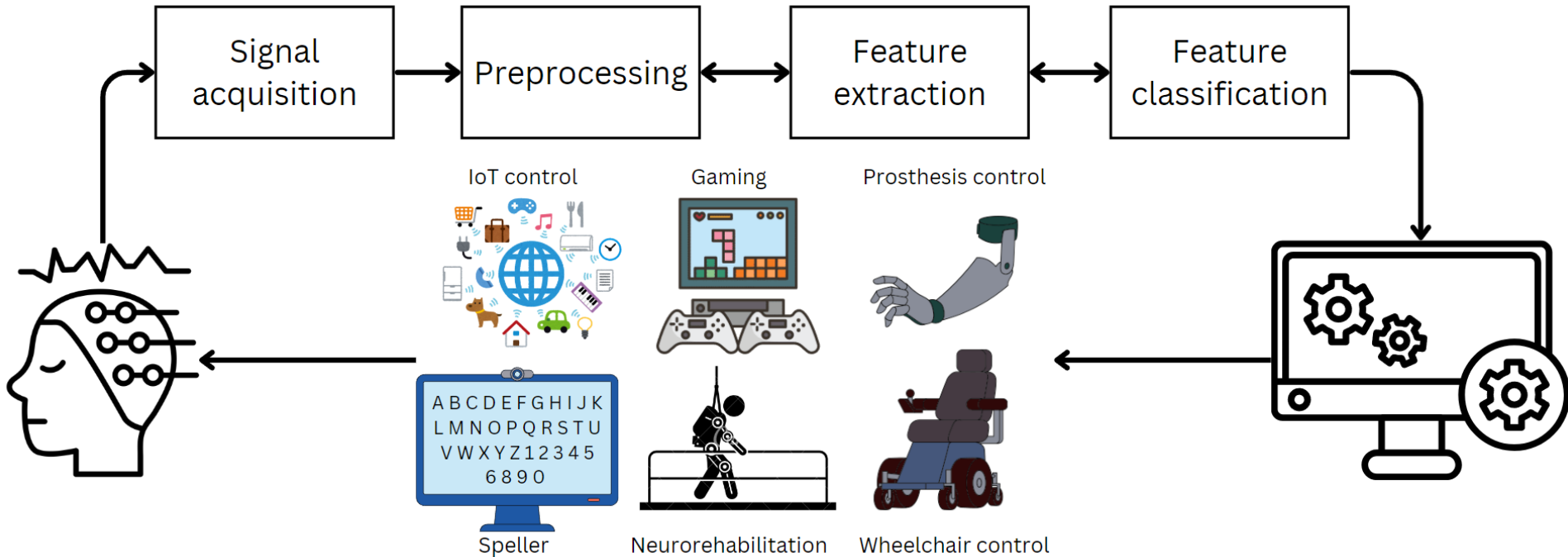


MYSTERIES OF THE BRAIN

 NBC
LEARN



Traditional BCI Pipeline



Signal acquisition

BCI relies on the brain signals recorded from the user to decode the mental states into output commands. The most used techniques are: Electroencephalography (EEG), Electrocorticography (ECoG), Magnetoencephalography (MEG), Functional Magnetic Resonance Imaging (fMRI).

Recording method	Activity measured	Temporal resolution	Spatial resolution	Risk	Portability
EEG	Electrical	0.05s	10mm	Non-invasive	Portable
ECoG	Electrical	0.003s	1mm	Invasive	Non-portable
MEG	Magnetic	0.05s	5mm	Non-invasive	Non-portable
fMRI	Metabolic	1s	1mm	Non-invasive	Non-portable

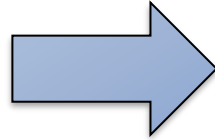
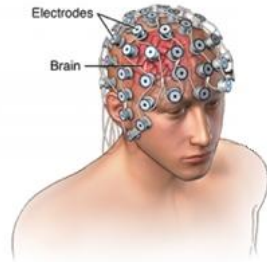
Signal acquisition

BCI relies on the brain signals recorded from the user to decode the mental states into output commands. The most used techniques are: Electroencephalography (EEG), Electrocorticography (ECoG), Magnetoencephalography (MEG), Functional Magnetic Resonance Imaging (fMRI).

Recording method	Activity measured	Temporal resolution	Spatial resolution	Risk	Portability
EEG	Electrical	0.05s	10mm	Non-invasive	Portable
ECoG	Electrical	0.003s	1mm	Invasive	Non-portable
MEG	Magnetic	0.05s	5mm	Non-invasive	Non-portable
fMRI	Metabolic	1s	1mm	Non-invasive	Non-portable

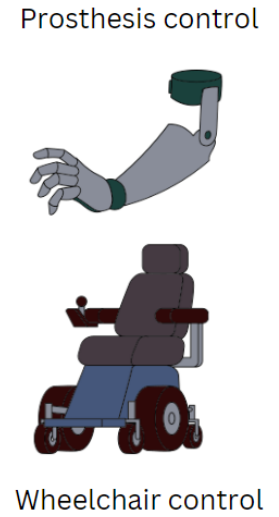
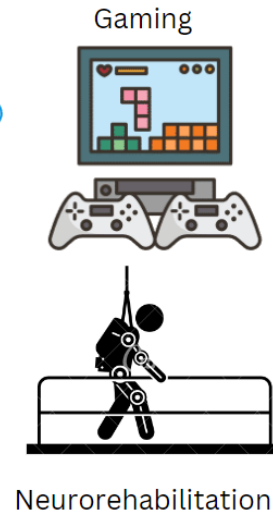
Signal acquisition

Electroencephalography (EEG) is a medical imaging technique used for recording and monitoring the electrical brain activity produced by the transmission of neuronal information (both chemical and electrical).



Applications

The feedback application is responsible for giving the user a notion of how well he/she is performing the mental task. Such applications could be a speller program, a robotic prosthesis, a game or merely a gauge showing the user's performance. The application uses the outcome from the classifier to perform the specific task or event associated with the user's mental state.



BCI – Neurophysiological patterns

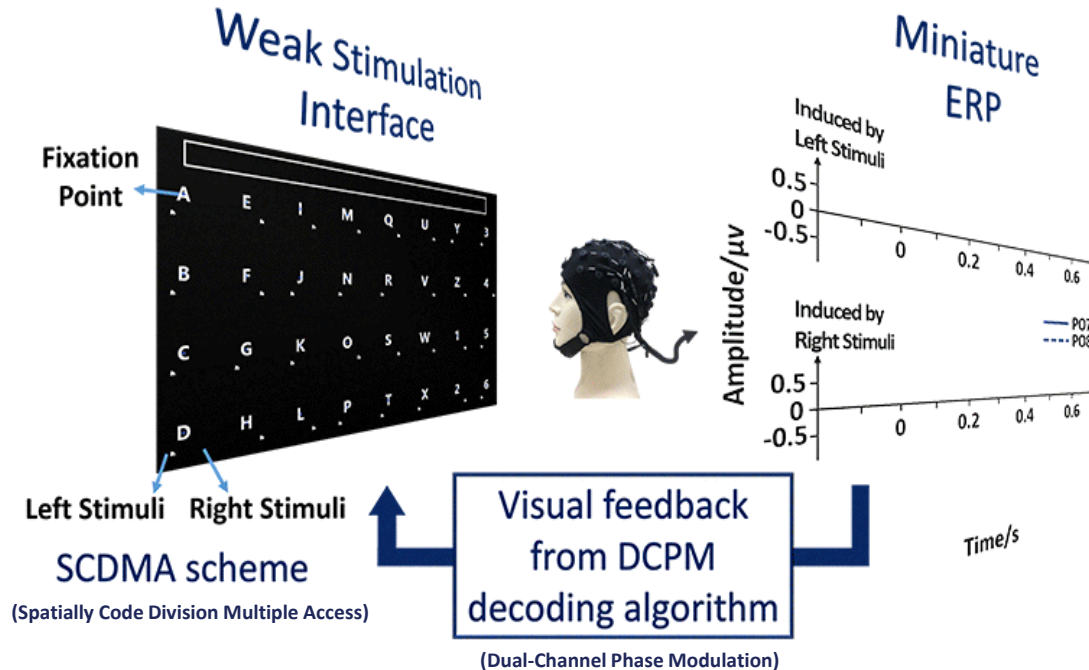
The brain produces different neurophysiological patterns resulting of cognitive responses. These patterns are characterized by their frequency (brainwaves oscillation), latency (time-locked), amplitude (phase-locked), and spatiotemporal distribution to internal or external stimuli and cognitive responses of the brain. By how the brain signals are generated, Brain-Computer Interface (BCI) can be classified as Exogenous or Endogenous.

BCI - Neurophysiological patterns - Exogenous BCI

Key features and characteristics:

- **External stimuli:** visual patterns, auditory tones, or tactile sensations, to evoke brain responses.
- **Evoked potentials:** The brain responses elicited by the external stimuli are known as evoked potentials, which are specific patterns of electrical activity in the brain.
- **Minimal user training:** generally, require less user training as they depend on automatic brain responses rather than learned control of brain activity.

Applications: communication aids for individuals with severe disabilities, control of external devices, and neurogaming.

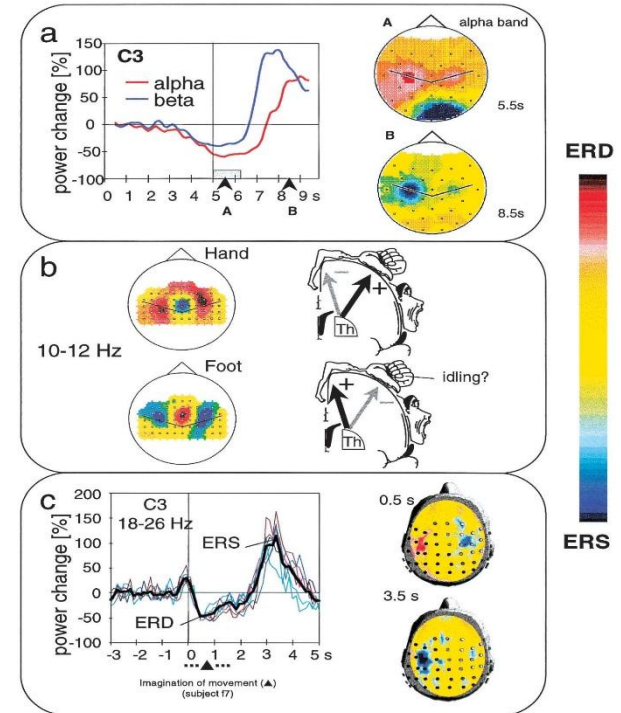


BCI - Neurophysiological patterns - Endogenous BCI

Key features and characteristics:

- **Self-generated brain activity:** Endogenous BCIs utilize brain signals that are produced naturally by the user's thoughts or intentions.
- **User training:** Endogenous BCIs typically require extensive user training to learn how to control their brain activity and produce distinct patterns for different commands.
- **Flexibility and control:** Once trained, users can have greater flexibility and control over the BCI, allowing for more complex interactions.

Applications: Endogenous BCIs are used in various applications, including control of prosthetic limbs, wheelchairs, and computer cursors, as well as in neurorehabilitation and gaming.



BCI – Neurophysiological patterns

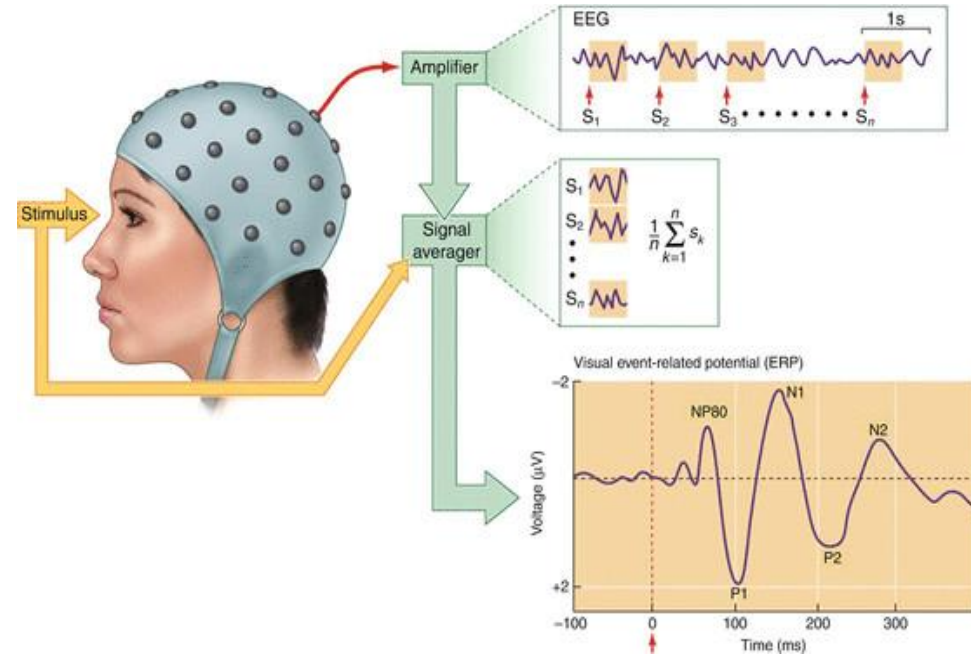
These endogenous or exogenous BCI produces well-established EEG patterns that are used for BCI, namely, Event-Related Potentials (ERPs), Sensorimotor Rhythms (SMRs), and Slow Cortical Potentials (SCPs) .

BCI – Neurophysiological patterns - ERPs

ERPs are very small voltage changes in brain activity that are time-locked to a specific event, such as a sensory stimulus or a cognitive process. They are like electrical "blips" in the brain's ongoing activity.

ERPs are extracted from the EEG by averaging the brain's response to many repetitions of the same event.

This helps to isolate the ERP from the background "noise" of the EEG.



BCI – Neurophysiological patterns - ERPs

Key patterns used in BCIs:

- **P300:** A positive deflection that occurs around 300 milliseconds after an unexpected or infrequent stimulus. Used in BCIs for spelling, selecting options, and controlling devices.
- **Steady-State Visually Evoked Potentials (SSVEPs):** brain responses that occur at the same frequency as a flickering visual stimulus. This means they are precisely synchronized with an external event (the flickering), which is a defining characteristic of ERPs.
- **Mismatch Negativity (MMN):** A negative deflection that occurs when the brain detects a change in a repetitive pattern of stimuli. Used in BCIs for attention monitoring and error detection.
- **Error-Related Negativity (ERN):** A negative deflection that occurs after a person makes a mistake. Used in BCIs for performance monitoring and adapting the system to the user's needs.

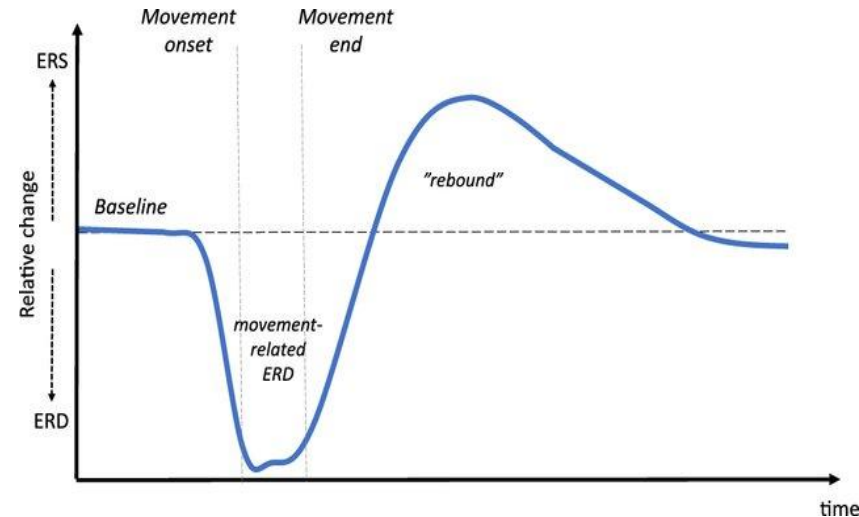
BCI – Neurophysiological patterns - SMRs

SMRs are brainwave patterns characterized by their frequency and amplitude that occur in the sensorimotor cortex, the part of the brain responsible for planning and executing movements. They are characterized by specific frequencies:

Key patterns used in BCIs:

Mu rhythm (8-13 Hz): Most prominent when the body is at rest and decreases in amplitude when a person performs or imagines a movement.

Beta rhythm (13-30 Hz): Can increase or decrease with movement or imagined movement, depending on the specific task.



BCI – Categorization

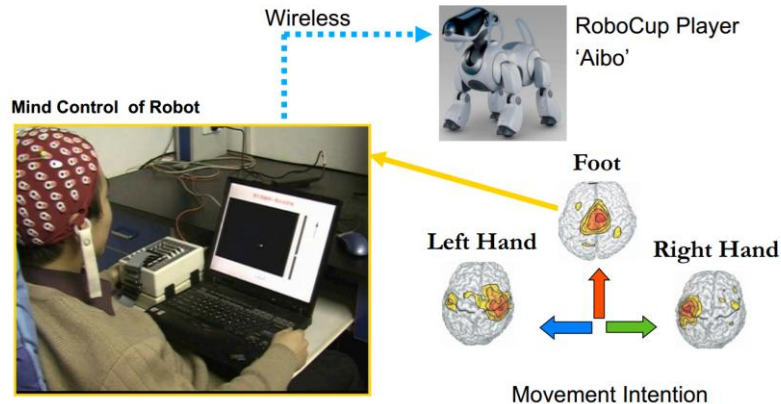
Likewise, BCI can be categorized by the level of user involvement as Passive, Active and Reactive.

BCI Type	Brain Signal Source	User Control	Purpose
Passive	Ongoing brain activity	No intentional control	Mental state monitoring, adaptive systems
Active	Intentional modulation of brain activity	Direct, conscious control	Device control, communication
Reactive	Brain response to external stimuli	Indirect control through attention	Communication, control

BCI - Paradigms

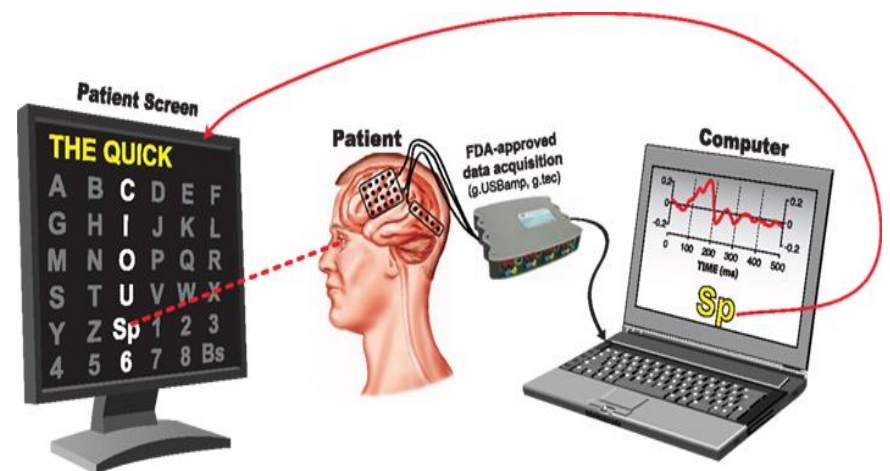
Finally, a core concept around BCI is the Paradigm, which refers to the specific task subjects must perform in order to elicit the signals. Among the most used paradigms we have:

Motor Imagery



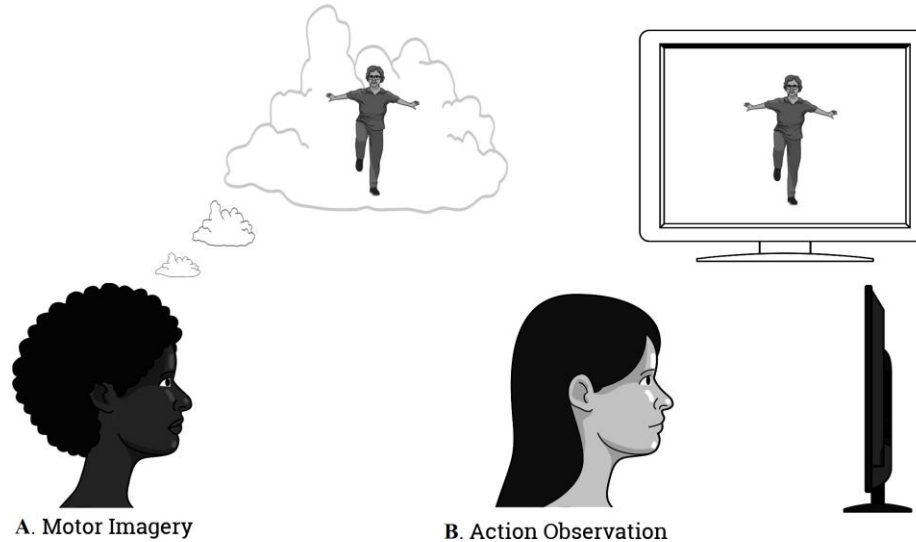
Wang et al., NE Workshop 11, Hsinchu, Taiwan

P300/SSVEP



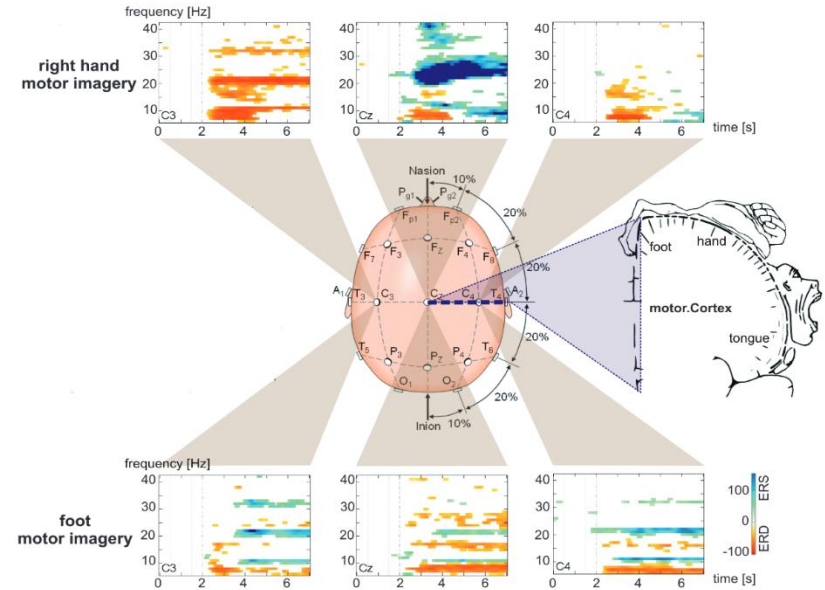
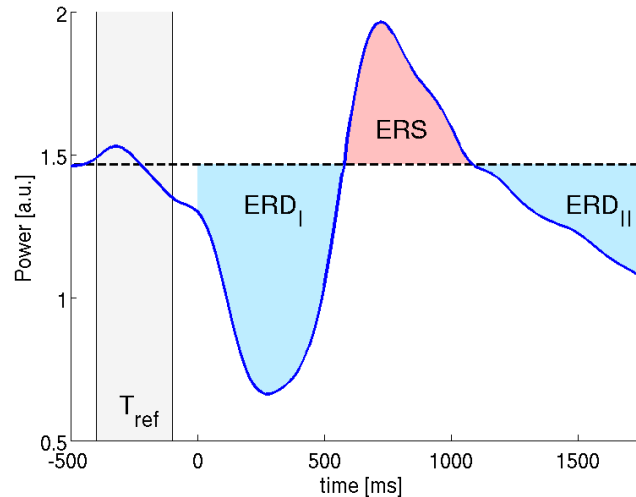
BCI – Paradigms – Motor Imagery (MI)

MI is the mental rehearsal of a motor action without actually performing any movement.



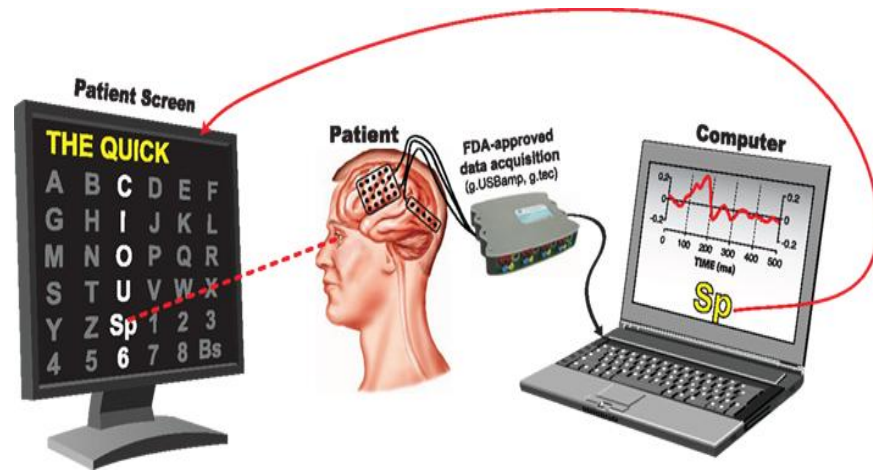
BCI – Paradigms – Motor Imagery (MI)

Such tasks elicit cortical rhythms identified by two types of activity: the attenuation (event-related desynchronization, ERD) or the enhancement (event-related synchronization, ERS) of the power in mu (7-12Hz) and beta (13-30Hz) bands.



BCI – Paradigms - P300/SSVEP

The core of the P300 paradigm is the "oddball task." The user is presented with a series of stimuli, where one of them is different or infrequent (the "oddball"). The brain generates a P300 response to this oddball stimulus because it's unexpected or noteworthy.



Stimulus Presentation: Typically, a grid of letters, numbers, or symbols is displayed. Rows and columns of this grid are flashed in a random sequence.

User Focus: The user focuses their attention on the character they want to select. When the row or column containing that character flashes, their brain generates a P300 response.

Detection: The BCI detects the P300 response and identifies the row and column that elicited it, thus determining the selected character.

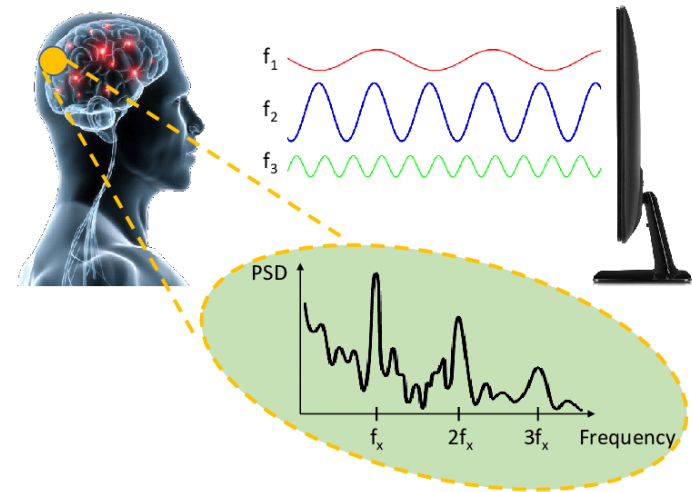
BCI – Paradigms - P300/SSVEP

In the SSVEP paradigm, different visual stimuli (like boxes or icons) are flashed at different frequencies (e.g., one at 10 Hz, another at 12 Hz, etc.). When the user focuses on a specific flickering stimulus, their brain's electrical activity starts to synchronize with that frequency. This creates a strong SSVEP response in the EEG at that same frequency.

Stimulus Presentation: Multiple stimuli, each flickering at a unique frequency, are presented on a screen.

User Focus: The user gazes at the stimulus they want to select.

Detection: The BCI analyzes the EEG data to identify the dominant frequency of the SSVEP response, which corresponds to the selected stimulus.



<http://dx.doi.org/10.1109/ACCESS.2019.2951327>

BCI Paradigm	Neurophysiological Pattern	Type of BCI	Source of Signal	Applications	Advantages	Disadvantages	Signal Features	Scalp Localization
P300	Event-Related Potential (P300)	Reactive	Exogenous	Spelling interfaces, device control, gaming	Easy to learn, relatively high accuracy	Requires external stimuli, can be slow	Positive deflection ~300ms after target stimulus	Central-parietal regions
SSVEP	Event-Related Potential (SSVEP)	Reactive	Exogenous	Device control, communication, gaming	High information transfer rate, relatively easy to use	Requires visual stimuli, can be fatiguing for some	Steady-state oscillations at specific frequencies	Occipital regions
Motor Imagery	Sensorimotor Rhythms (SMR) - Mu - Beta	Active	Endogenous	Control of prosthetics, wheelchairs, cursors	No external stimuli needed, allows for continuous control	Requires extensive training, susceptible to variability	Decrease or increase in power during movement/imagery	Sensorimotor cortex
SCP	Slow Cortical Potentials (SCP)	Active	Endogenous	Control of devices, neurofeedback training	No external stimuli needed	Requires extensive training, challenging to control	Slow shifts in cortical excitability	Widespread
Hybrid	Combination of patterns (e.g., P300 and SSVEP)	Reactive/Active	Exogenous/Endogenous	Combines strengths of different paradigms	Can improve accuracy and flexibility	May be more complex to implement	Varies	Varies
Passive BCI	Various (e.g., alpha waves, ERPs, SMRs)	Passive	Exogenous/Endogenous	Mental state monitoring, adaptive systems, fatigue detection	No user effort required, can be used in real-time	Limited control, interpretation of signals can be challenging	Varies	Varies

Today's plan

- ~~Brain-Computer Interface (BCI)~~
 - ~~Overview~~
 - ~~Stages - Pipeline~~
 - ~~Applications~~
 - ~~Neurophysiological patterns, Categorization, Paradigms~~
- Create the experiment
 - Psychopy
 - Labstreaminglayer
- Pipeline
 - Signal processing
 - Feature extraction
 - Classification
 - Evaluation
- Hands on

Introduction PsychoPy

What is it ?

It is an open-source package for creating and running psychological experiments. PsychoPy uses both OpenGL (most widely adopted 2D and 3D graphics library) and Python to give scientists and students a free and simple stimulus presentation and control package. It is widely used by many labs worldwide for psychophysics, cognitive neuroscience and experimental psychology.

www.psychopy.org



Introduction PsychoPy

Advantages?

Instead of using paid software (Matlab, up 600 \$USD; e-Prime, up 300 \$USD), PsychoPy is totally free and open-source.

As any open-source software, you are able to download and modify the package as much as you want. PsychoPy is developed in a single piece of software enabling to be precise enough for psychophysics, intuitive enough for undergraduate psychology and flexible enough for anyone who is intending to develop own experiments.



Introduction PsychoPy

Major features:

- Simple install process;
- Precise timing;
- Platform independent - run the same script on Win, macOS or Linux;
- Flexible stimulus units (degrees, cm, or pixels);
- Multi-monitor support;
- Automated monitor calibration (for supported photometers);
- Online running (through Padovia)
- Coder interface, for those that like programming
- Builder interface, for those that don't;



Introduction PsychoPy

Huge variety of stimuli generated in real-time:

- Images
- Random dots
- Text (unicode in any truetype font)
- Shapes and movies (DivX, mov, mpg...)
- sounds (tones, numpy arrays, wav, ogg...)

Several output responses

- Keyboard
- Joystick
- Mouse
- Textbox
- Mic

Vast of input devices (synchronized in near real-time)

- EEG
- Eye-tracking
- Serial port

Installation

PsychoPy standalone:

<https://www.psychopy.org/download.html>

Pycharm IDLE Community for programming:

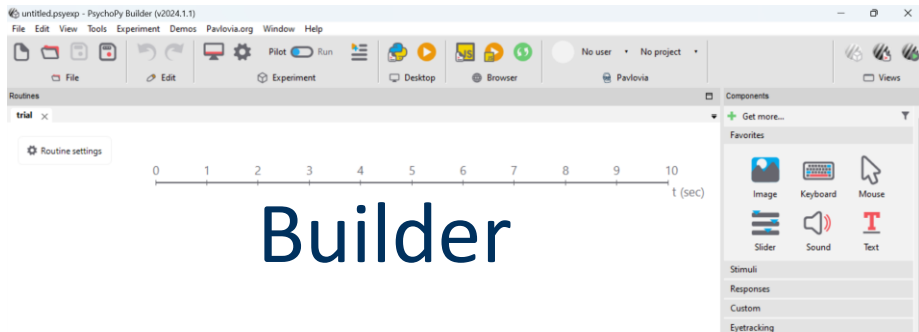
<https://www.jetbrains.com/pycharm/download/?section=windows>

Also, you can use Anaconda:

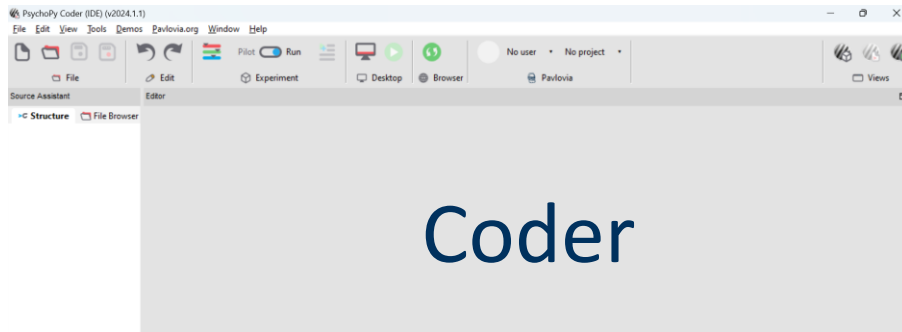
<https://www.anaconda.com/download>



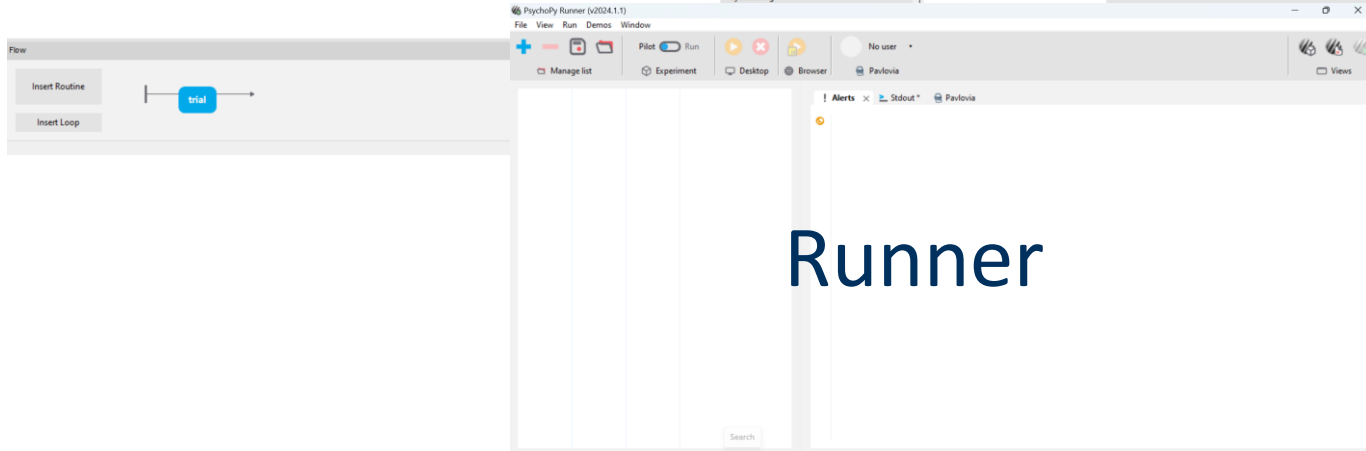
First impressions - Installation



Builder

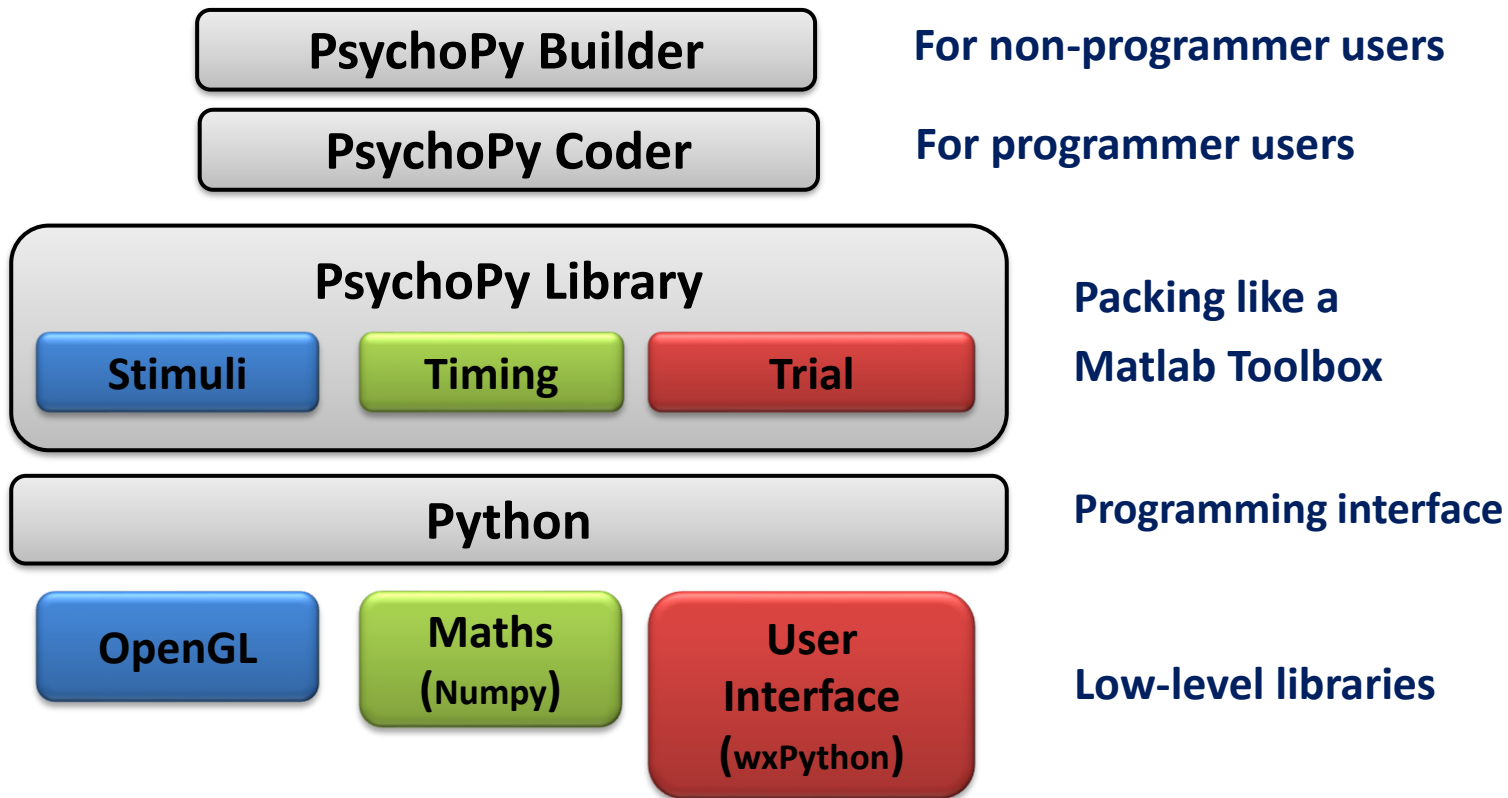


Coder



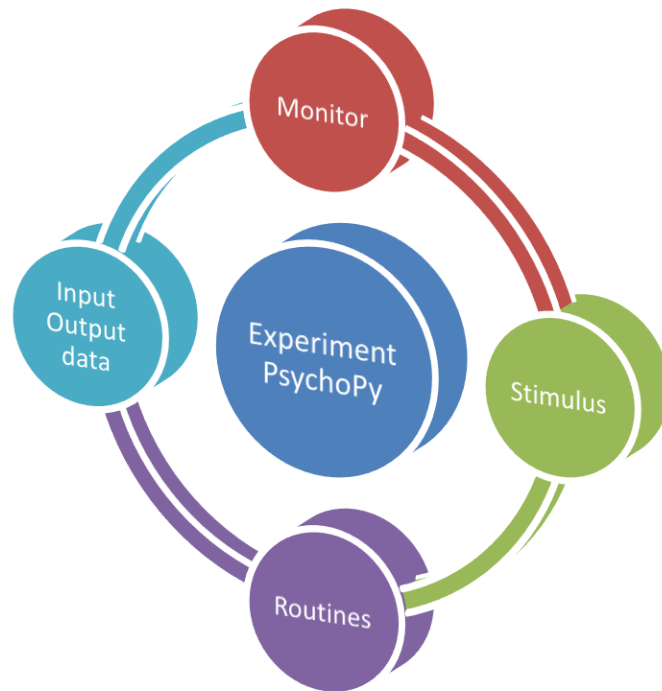
Runner

Architecture – Builder and Coder



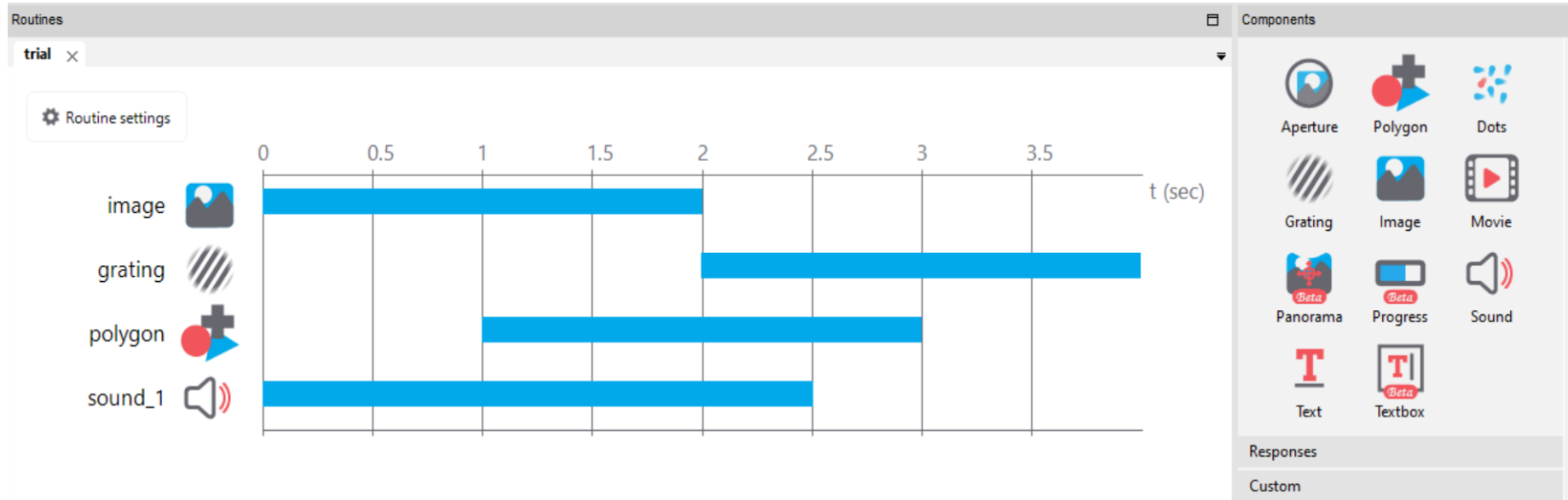
First steps – PsychoPy experiments

PsychoPy uses a monitor for presenting stimulus (text, images, sounds, videos); thus, these objects are showed on the monitor (duration and way), known as routines, therefore, the created routines represent the whole experiment itself. Within the routines, how the data (user answers) is collected should be set-up to save this information for a posterior analysis.



Main interface

Routines + Components



Routines

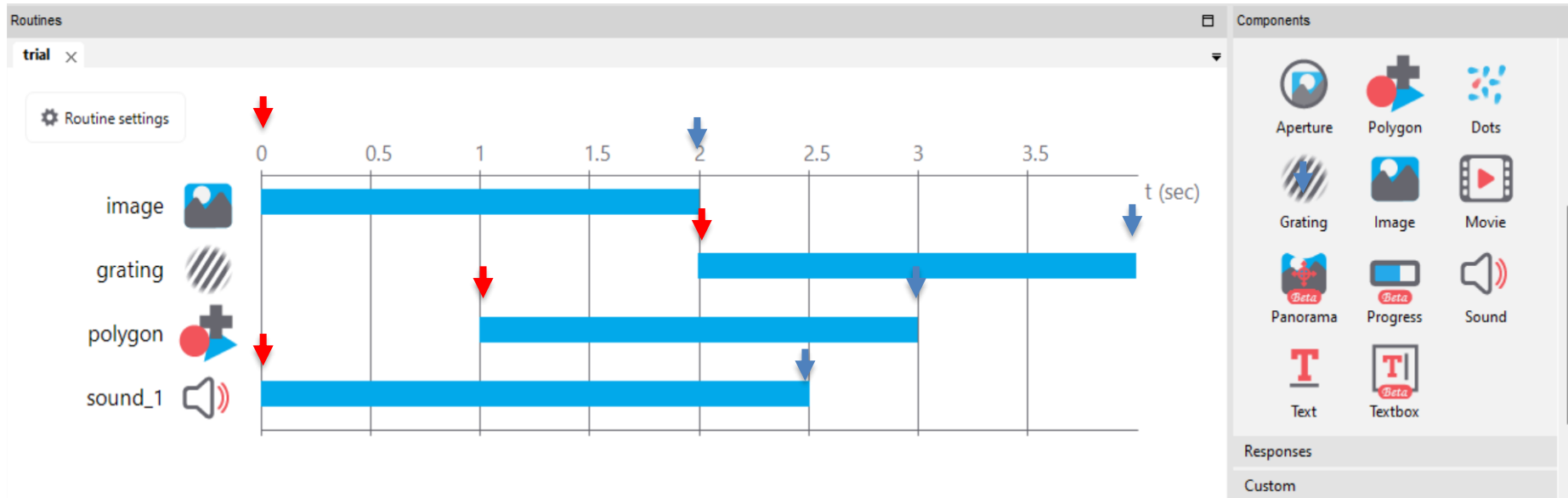
A Routine might specify the timing of events within a trial or the presentation of instructions or feedback.

Within a Routine there are a number of components. These components determine the occurrence of a stimulus, or the recording of a response.

These object can overlap and there is also objects with a infinite duration (whole experiment);

Routines

Any number of components can be added to a Routine. Each has a duration (**start** and **stop** time) which is represented by a line in the routine panel.



Main Interface - Flow

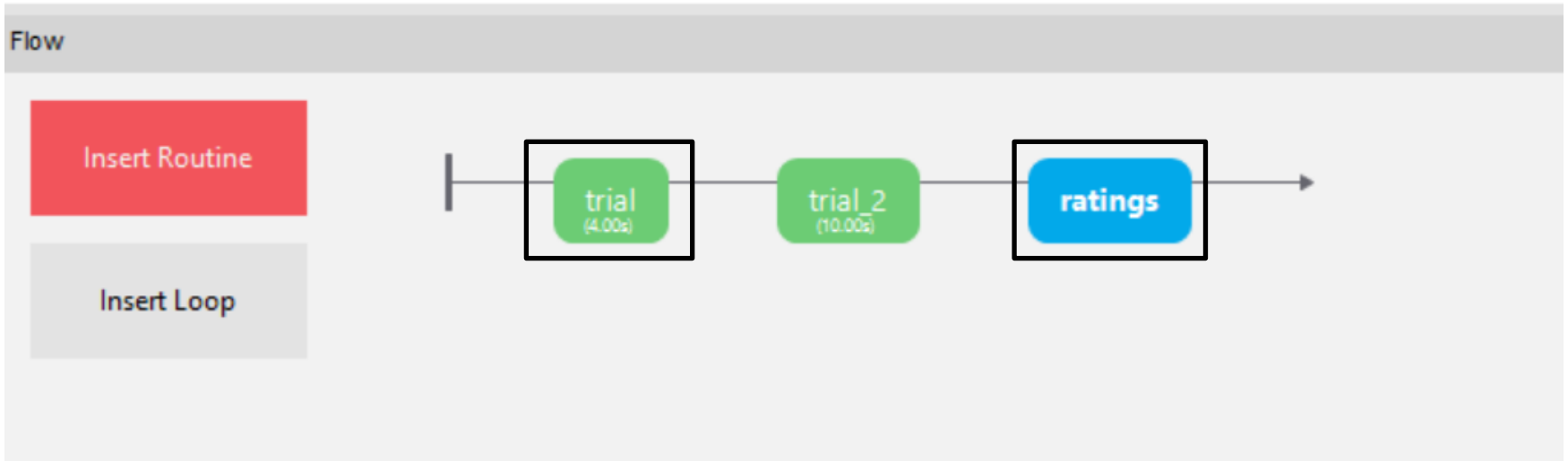
You can add as many routines as you need. Multiple Routines can then be combined in the **Flow**, which controls the order in which these occur and the way in which they repeat.

For instance, your study may have a Routine that presented initial instructions and waited for a key to be pressed, followed by a Routine that presented one trial which should be repeated 5 times with various different parameters set. All of this is achieved in the Flow panel.



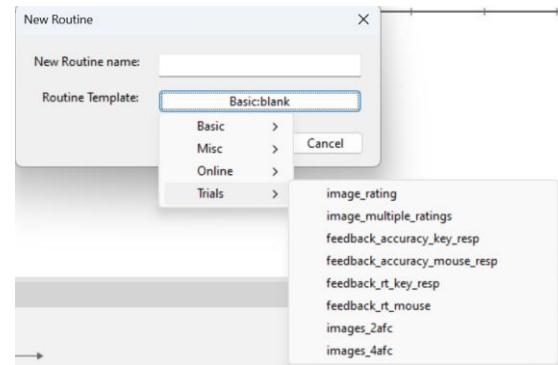
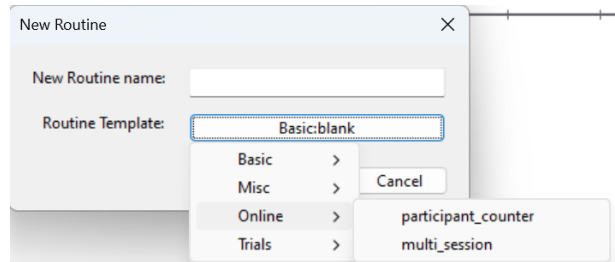
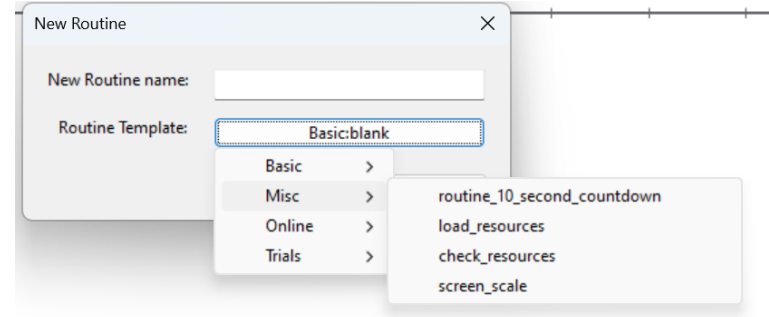
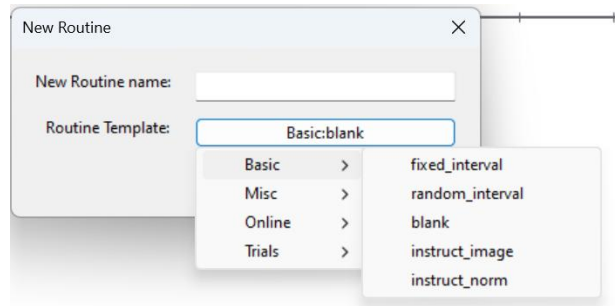
Main interface - Flow

We have two kind of routines or trials, and it depends if we predefine the timing (**green**) or it is endless (based on the input) (**blue**)



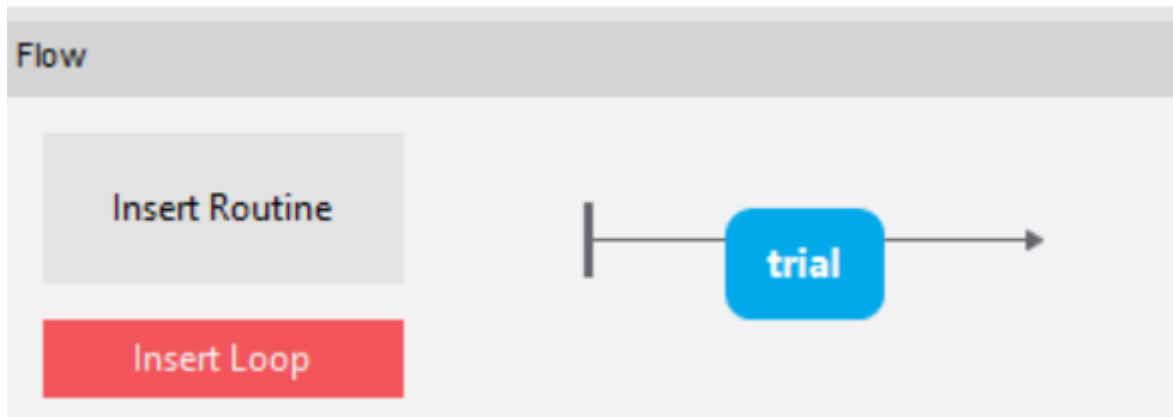
Main interface - Flow

We have also several templates to create Routines:



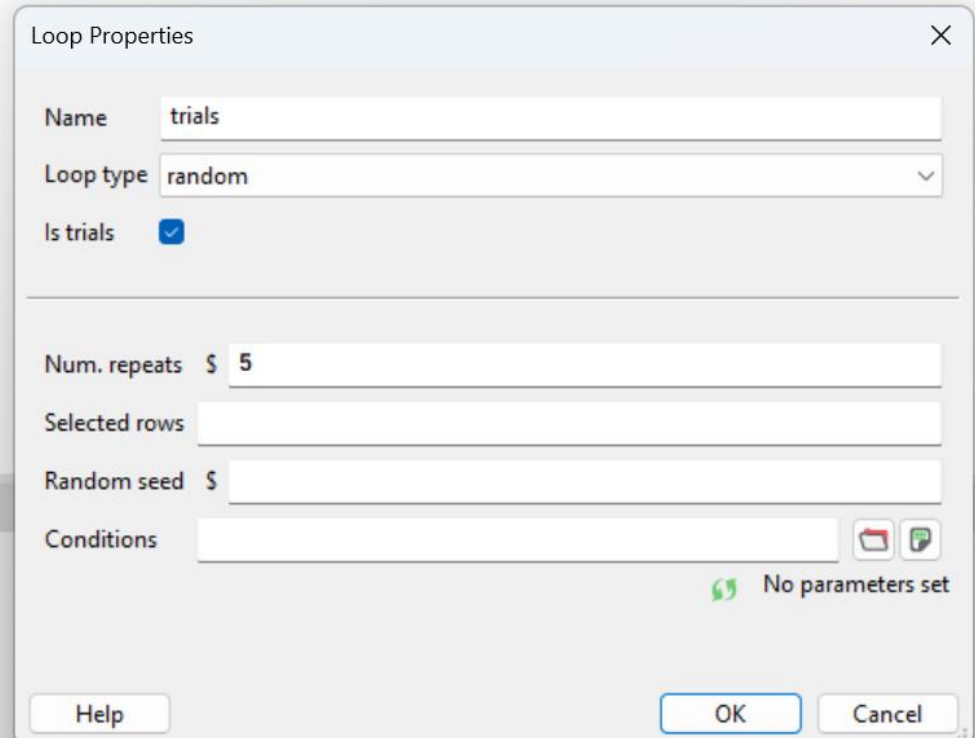
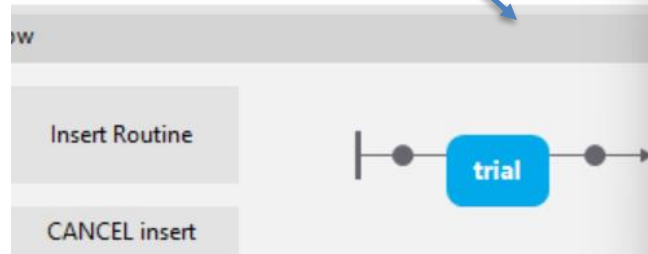
Loops

Loops control the repetition of Routines as well as which stimulus parameters for each. PsychoPy can generate the next trial based on constants, sequential and random methods or using an adaptive staircase. To insert a Loop use the button on the left of the Flow panel, or the item in the Experiment menu of the Builder. Loops can encompass one or more Routines and other loops (i.e. they can be nested).



Loops – Loop type

**You must choose
where the loop
starts and
finishes.**



Loop Properties

Name: trials

Loop type: random

Is trials: ☒

Num. repeats: 5

Selected rows:

Random seed: \$

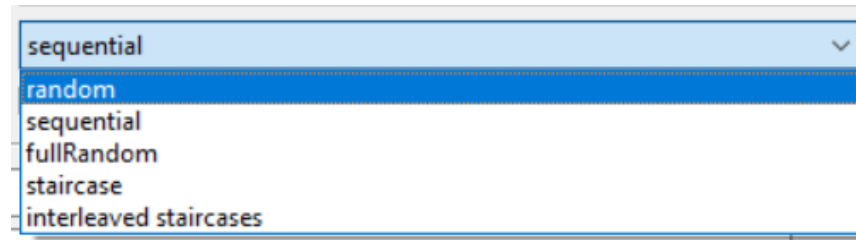
Conditions:

No parameters set

Help OK Cancel

Loops – Loop type

- If the loop type is **sequential** then, on each iteration through the Routines, the next item will be selected in the order listed in the conditions file (we will see it later).
- Under a **random** order, the next item will be selected at random (without replacement); it can only be selected again after all the other items have also been selected. **nReps** determines how many repeats will be performed (for all conditions).
- With the **fullRandom** option, the entire list of trials including repetitions is used in random order, allowing the same item to appear potentially many times, and to repeat without necessarily having done all of the other trials.



Loops – Loop type

For example, in a list [a,b,c], with three repetitions...

- ***sequential*** could only ever give one sequence with this order: [a b c a b c a b c];
- ***random*** will give one of 216 different orders ($= 3! * 3! * 3! = nReps * (nTrials!)$) for example: [b a c a b c c a b]. Here the letters are effectively in sets of (abc) (abc) (abc), and randomization is only done within each set;
- ***fullRandom*** will return one of 362,880 different orders ($= 9! = (nReps * nTrials!)!$), such as [b b c a a c c a b], which *random* never would, indeed, it is possible to obtain the unrandom-looking sequence like [a a a b b b c c c];

Loops – Conditions

Selecting a loop type either random, sequential, or fullRandom will result in a method of constants experiment, whereby the types of trials that can occur are predetermined. That is, the trials cannot vary depending on how the subject has responded on a previous trial.

In this case, a file must be provided that describes the parameters for the repeats. This should be either Excel Spreadsheet (xlsx), comma-separated-value (csv) or pick-up files in which **columns** refer to parameters that are needed to describe stimuli etc. and **rows** one for each type of trial.

The top row should be a row of headers: text labels describing the contents of the respective columns.

Loops – Conditions

For example, the next table has four different conditions (or trial types, one per line). The header line describes the parameters in the three columns: ori, text and corrAns. It's really useful to include a column called corrAns that shows what the correct key press is going to be for this trial (if there is one).

Ori	Text	corrAns
0	aaaa	left
90	aaaa	left
0	bbbb	right
90	bbb	right

Conditions

Browse...

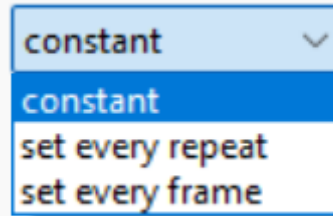
No parameters set

Loops – Variables

The parameters from your loops are accessible to any component enclosed within that loop. The simplest (and default) way to address these variables is simply to call them by the name of the parameter, and a symbol **\$** to indicate that this is the name of a variable. For example, if your Flow contains a loop with the above table as its input trial types file then you could give one of your stimuli an orientation **\$ori** which would depend on the current trial type being presented. Example scenarios:

- You want to loop randomly over some conditions in a loop called trials (table above). You can then access these values from any component using **\$ori**, **\$text**, and **\$corrAns**
- You create a random loop called blocks and give it a file with a single column called **movieName** listing filenames to be played. On each repeat you can access this with **\$movieName**
- You create a staircase loop called stairs. On each trial you can access the current value in the staircase with **\$thisStair**

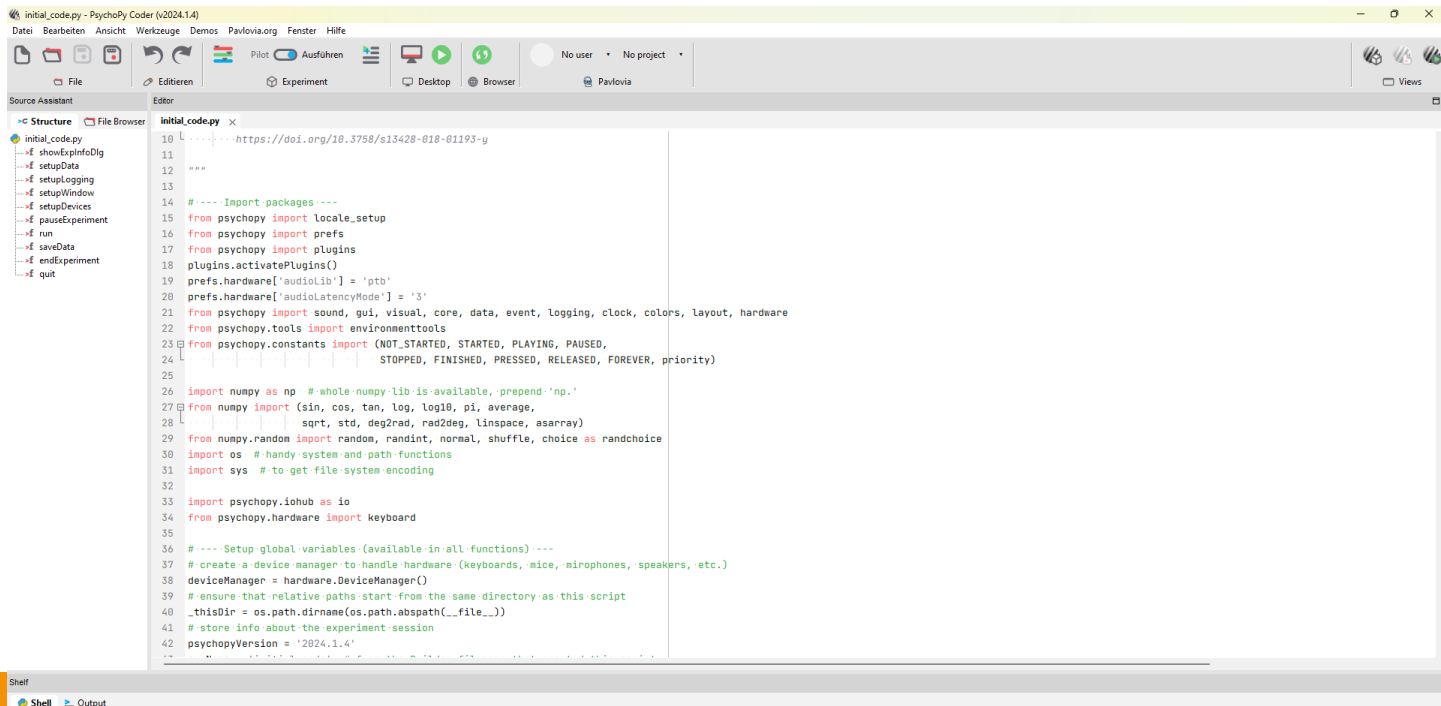
Loops – Variables



Note

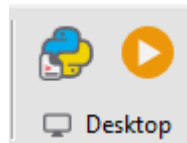
When you set a component to use a parameter that will change (e.g on each repeat through the loop) you should **remember to change the component parameter from `constant` to `set every repeat` or `set every frame`** or it won't have any effect!

Every Builder experiment has their own Python code:



The screenshot displays the PsychoPy Builder interface. The top menu bar includes 'Datei', 'Bearbeiten', 'Ansicht', 'Werkzeuge', 'Demos', 'Pavlovia.org', 'Fenster', and 'Hilfe'. Below the menu is a toolbar with icons for file operations, running the experiment, and viewing the desktop. The main window is divided into a 'Source Assistant' on the left and an 'Editor' on the right. The 'Source Assistant' shows a tree view of the experiment components, including 'initial_code.py'. The 'Editor' displays the Python code for 'initial_code.py', which includes imports for various modules like 'psychopy', 'numpy', and 'os', and sets up the experiment environment.

```
10 L ..... https://doi.org/10.3758/s13428-018-01193-y
11
12 ===
13
14 #---Import packages---
15 from psychopy import locale_setup
16 from psychopy import prefs
17 from psychopy import plugins
18 plugins.activatePlugins()
19 prefs.hardware['audioLib'] = 'ptb'
20 prefs.hardware['audioLatencyMode'] = '3'
21 from psychopy import sound, gui, visual, core, data, event, logging, clock, colors, layout, hardware
22 from psychopy.tools import environmenttools
23 from psychopy.constants import (NOT_STARTED, STARTED, PLAYING, PAUSED,
24                                STOPPED, FINISHED, PRESSED, RELEASED, FOREVER, priority)
25
26 import numpy as np # whole numpy lib is available, prepend 'np.'
27 from numpy import (sin, cos, tan, log, log10, pi, average,
28                   sqrt, std, deg2rad, rad2deg, linspace, asarray)
29 from numpy.random import random, randint, normal, shuffle, choice as randchoice
30 import os # handy system and path functions
31 import sys # to get file system encoding
32
33 import psychopy.iohub as io
34 from psychopy.hardware import keyboard
35
36 #--- Setup global variables (available in all functions)---
37 # create a device manager to handle hardware (keyboards, mice, microphones, speakers, etc.)
38 deviceManager = hardware.DeviceManager()
39 # ensure that relative paths start from the same directory as this script
40 _thisDir = os.path.dirname(os.path.abspath(__file__))
41 # store info about the experiment session
42 psychopyVersion = '2024.1.4'
```



Coding

```

381 #---- Prepare to start Routine "trial"----
382 continueRoutine = True
383 # update component parameters for each repeat
384 thisExp.addData('trial.started', globalClock.getTime(format='float'))
385 # keep track of which components have finished
386 trialComponents = []
387 for thisComponent in trialComponents:
388     thisComponent.tStart = None
389     thisComponent.tStop = None
390     thisComponent.tStartRefresh = None
391     thisComponent.tStopRefresh = None
392     if hasattr(thisComponent, 'status'):
393         thisComponent.status = NOT_STARTED
394 # reset timers
395 t = 0
396 _timeToFirstFrame = win.getFutureFlipTime(clock="now")
397 frameN = -1
398
399 #---- Run Routine "trial"----
400 routineForceEnded = not continueRoutine
401 while continueRoutine:
402     # get current time
403     t = routineTimer.getTime()
404     tThisFlip = win.getFutureFlipTime(clock=routineTimer)
405     tThisFlipGlobal = win.getFutureFlipTime(clock=None)
406     frameN = frameN + 1 # number of completed frames (so 0 is the first frame)
407     # update/draw components on each frame
408
409     # check for quit (typically the Esc key)
410     if defaultKeyboard.getKeys(keyList=["escape"]):
411         thisExp.status = FINISHED
412     if thisExp.status == FINISHED or endExpNow:
413         endExperiment(thisExp, win=win)
414         return

```

Check the
indentation

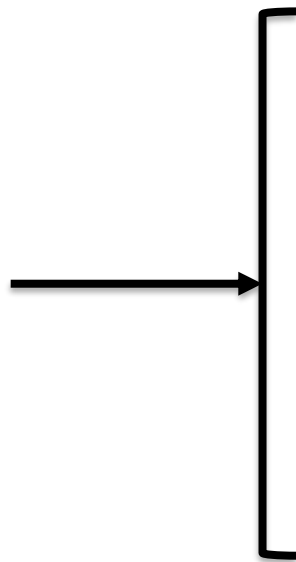
```

409
410 # check for quit (typically the Esc key)
411 if defaultKeyboard.getKeys(keyList=["escape"]):
412     thisExp.status = FINISHED
413     if thisExp.status == FINISHED or endExpNow:
414         endExperiment(thisExp, win=win)
415         return
416
417 # check if all components have finished
418 if not continueRoutine: # a component has requested a forced end of routine
419     routineForceEnded = True
420     break
421 continueRoutine = False # will revert to True if at least one component still running
422 for thisComponent in trialComponents:
423     if hasattr(thisComponent, "status") and thisComponent.status != FINISHED:
424         continueRoutine = True
425         break # at least one component has not yet finished
426
427 # refresh the screen
428 if continueRoutine: # don't flip if this routine is over or we'll get a blank screen
429     win.flip()
430
431 #---- Ending Routine "trial"----
432 for thisComponent in trialComponents:
433     if hasattr(thisComponent, "setAutoDraw"):
434         thisComponent.setAutoDraw(False)
435 thisExp.addData('trial.stopped', globalClock.getTime(format='float'))
436 thisExp.nextEntry()
437 # the Routine "trial" was not non-slip safe, so reset the non-slip timer
438 routineTimer.reset()
439
440 # mark experiment as finished
441 endExperiment(thisExp, win=win)

```

Coding

The usual experiment
order in PsychoPy

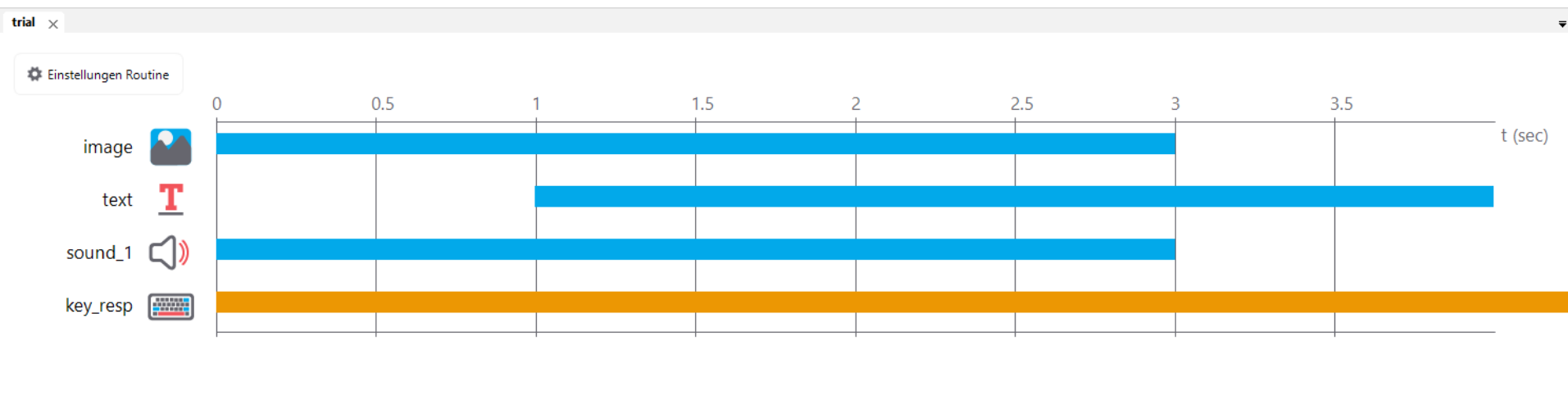


```

516 if __name__ == '__main__':
517     # call all functions in order
518     expInfo = showExpInfoDlg(expInfo=expInfo)
519     thisExp = setupData(expInfo=expInfo)
520     logFile = setupLogging(filename=thisExp.dataFileName)
521     win = setupWindow(expInfo=expInfo)
522     setupDevices(expInfo=expInfo, thisExp=thisExp, win=win)
523     run(
524         expInfo=expInfo,
525         thisExp=thisExp,
526         win=win,
527         globalClock='float'
528     )
529     saveData(thisExp=thisExp)
530     quit(thisExp=thisExp, win=win)
531 
```

Coding

Each component has their corresponding code



Coding

```
21 from psychopy import sound, gui, visual, core, data, event, logging, clock, colors, layout, hardware, parallel
22 from psychopy.tools import environmenttools
23 from psychopy.constants import (NOT_STARTED, STARTED, PLAYING, PAUSED,
24                                STOPPED, FINISHED, PRESSED, RELEASED, FOREVER, priority)
25
```

sound_1



key_resp



```
254 ... # create speaker 'sound_1'
255 deviceManager.addDevice(
256     deviceName='sound_1',
257     deviceClass='psychopy.hardware.speaker.SpeakerDevice',
258     index=-1
259 )
260 if deviceManager.getDevice('key_resp') is None:
261     # initialise key_resp
262     key_resp = deviceManager.addDevice(
263         deviceClass='keyboard',
264         deviceName='key_resp',
265     )
266 # return True if completed successfully
267 return True
```

Coding

Initialize components for Routine:

```
322 def run(expInfo, thisExp, win, globalClock=None, thisSession=None):
```

image



text



sound_1



key_resp



```
367 #-----Initialize components for Routine "trial"-----
368 image = visual.ImageStim(
369     win=win,
370     name='image',
371     image='hello.png', mask=None, anchor='center',
372     ori=0.0, pos=(0, 0), size=(0.5, 0.5),
373     color=[1,1,1], colorSpace='rgb', opacity=None,
374     flipHoriz=False, flipVert=False,
375     texRes=128.0, interpolate=True, depth=0.0)
376 text = visual.TextStim(win=win, name='text',
377     text='Some text',
378     font='Arial',
379     pos=(0, 0.25), height=0.05, wrapWidth=None, ori=0.0,
380     color='white', colorSpace='rgb', opacity=None,
381     languageStyle='LTR',
382     depth=-1.0);
```

```
383 sound_1 = sound.Sound(
384     'A',
385     secs=-1,
386     stereo=True,
387     hamming=True,
388     speaker='sound_1', name='sound_1'
389 )
390 sound_1.setVolume(1.0)
391 key_resp = keyboard.Keyboard(deviceName='key_resp')
```

Coding

Prepare to start Routine:

image



text



sound_1



key_resp



```
322 def run(expInfo, thisExp, win, globalClock=None, thisSession=None):
```

```

    # ---- Prepare to start Routine "trial" ----
    continueRoutine = True
    # update component parameters for each repeat
    thisExp.addData('trial.started', globalClock.getTime(format='float'))
    sound_1.setSound('world.wav', secs=3, hamming=True)
    sound_1.setVolume(1.0, log=False)
    sound_1.seek(0)
    key_resp.keys = []
    key_resp.rt = []
    _key_resp_allKeys = []
    # keep track of which components have finished
    trialComponents = [image, text, sound_1, key_resp]
    for thisComponent in trialComponents:
        thisComponent.tStart = None
        thisComponent.tStop = None
        thisComponent.tStartRefresh = None
        thisComponent.tStopRefresh = None
        if hasattr(thisComponent, 'status'):
            thisComponent.status = NOT_STARTED
    # reset timers

```

Coding

Inside the main loop:

image



```
447 while continueRoutine:
```

```
455 .....#*image* updates
456 .....
457 .....#if image is starting this frame...
458 □ .....if image.status == NOT_STARTED and tThisFlip >= 0.0-frameTolerance:
459 .....#keep track of start time/frame for later
460 .....image.frameNStart = frameN #exact frame index
461 .....image.tStart = t #local t and not account for scr refresh
462 .....image.tStartRefresh = tThisFlipGlobal #on global time
463 .....win.timeOnFlip(image, 'tStartRefresh') #time at next scr refresh
464 .....#add timestamp to datafile
465 .....thisExp.timestampOnFlip(win, 'image.started')
466 .....#update status
467 .....image.status = STARTED
468 .....image.setAutoDraw(True)
469 .....
```

```
470 .....#if image is active this frame...
471 □ .....if image.status == STARTED:
472 .....#update params
473 .....pass
474 .....
475 .....#if image is stopping this frame...
476 □ .....if image.status == STARTED:
477 .....#is it time to stop? (based on global clock, using actual start)
478 □ .....if tThisFlipGlobal > image.tStartRefresh + 3.0-frameTolerance:
479 .....#keep track of stop time/frame for later
480 .....image.tStop = t #not accounting for scr refresh
481 .....image.tStopRefresh = tThisFlipGlobal #on global time
482 .....image.frameNStop = frameN #exact frame index
483 .....#add timestamp to datafile
484 .....thisExp.timestampOnFlip(win, 'image.stopped')
485 .....#update status
486 .....image.status = FINISHED
```

Coding

Inside the main loop:

text **T**

```
447 while continueRoutine:
```

```
489 ..... #*text* updates
490 .....
491 ..... #if text is starting this frame...
492 if text.status == NOT_STARTED and tThisFlip >= 1-frameTolerance:
493 ..... #keep track of start time/frame for later
494 text.frameNStart = frameN #exact frame index
495 text.tStart = t #local t and not account for scr refresh
496 text.tStartRefresh = tThisFlipGlobal #on global time
497 win.timeOnFlip(text, 'tStartRefresh') #time at next scr refresh
498 #add timestamp to datafile
499 thisExp.timestampOnFlip(win, 'text.started')
500 #update status
501 text.status = STARTED
502 text.setAutoDraw(True)
```

```
500 .....
504 ..... #if text is active this frame...
505 if text.status == STARTED:
506 ..... #update params
507 ..... pass
508 .....
509 ..... #if text is stopping this frame...
510 if text.status == STARTED:
511 ..... #is it time to stop? (based on global clock, using actual start)
512 if tThisFlipGlobal > text.tStartRefresh + 3-frameTolerance:
513 ..... #keep track of stop time/frame for later
514 text.tStop = t #not accounting for scr refresh
515 text.tStopRefresh = tThisFlipGlobal #on global time
516 text.frameNStop = frameN #exact frame index
517 #add timestamp to datafile
518 thisExp.timestampOnFlip(win, 'text.stopped')
519 #update status
520 text.status = FINISHED
521 text.setAutoDraw(False)
```

Coding

Inside the main loop:

sound_1 

```
523 ..... #if sound_1 is starting this frame...
524 ▢ ..... if sound_1.status == NOT_STARTED and tThisFlip >= 0.0-frameTolerance:
525 .....     #keep track of start time/frame for later
526 .....     sound_1.frameNStart = frameN #exact frame index
527 .....     sound_1.tStart = t #local t and not account for scr refresh
528 .....     sound_1.tStartRefresh = tThisFlipGlobal #on global time
529 .....     #add timestamp to datafile
530 .....     thisExp.addData('sound_1.started', tThisFlipGlobal)
531 .....     #update status
532 .....     sound_1.status = STARTED
533 .....     sound_1.play(when=win) #sync with win flip
```

```
447 ▢ ..... while continueRoutine:
```

```
535 .....     #if sound_1 is stopping this frame...
536 ▢ .....     if sound_1.status == STARTED:
537 .....         #is it time to stop? (based on global clock, using actual start)
538 ▢ .....         if tThisFlipGlobal > sound_1.tStartRefresh + 3-frameTolerance:
539 .....             #keep track of stop time/frame for later
540 .....             sound_1.tStop = t #not accounting for scr refresh
541 .....             sound_1.tStopRefresh = tThisFlipGlobal #on global time
542 .....             sound_1.frameNStop = frameN #exact frame index
543 .....             #add timestamp to datafile
544 .....             thisExp.timestampOnFlip(win, 'sound_1.stopped')
545 .....             #update status
546 .....             sound_1.status = FINISHED
547 .....             sound_1.stop()
548 .....             #update sound_1 status according to whether it's playing
549 ▢ .....             if sound_1.isPlaying:
550 .....                 sound_1.status = STARTED
551 ▢ .....             elif sound_1.isFinished:
552 .....                 sound_1.status = FINISHED
```

Remarks

- PsychoPy coding implies understanding the flow (creation, initialization, execution, and closing) of the objects (stimuli) in the experiment.
- These dynamics should be guided by the Core functions of PsychoPy (always see the API documentation)
- You can use the Builder for creating the main code schema and later learning by adding new routines/flow.
- Mastering the code will allow you to create custom experiments, remember, the builder is build-up by the code, not the opposite.

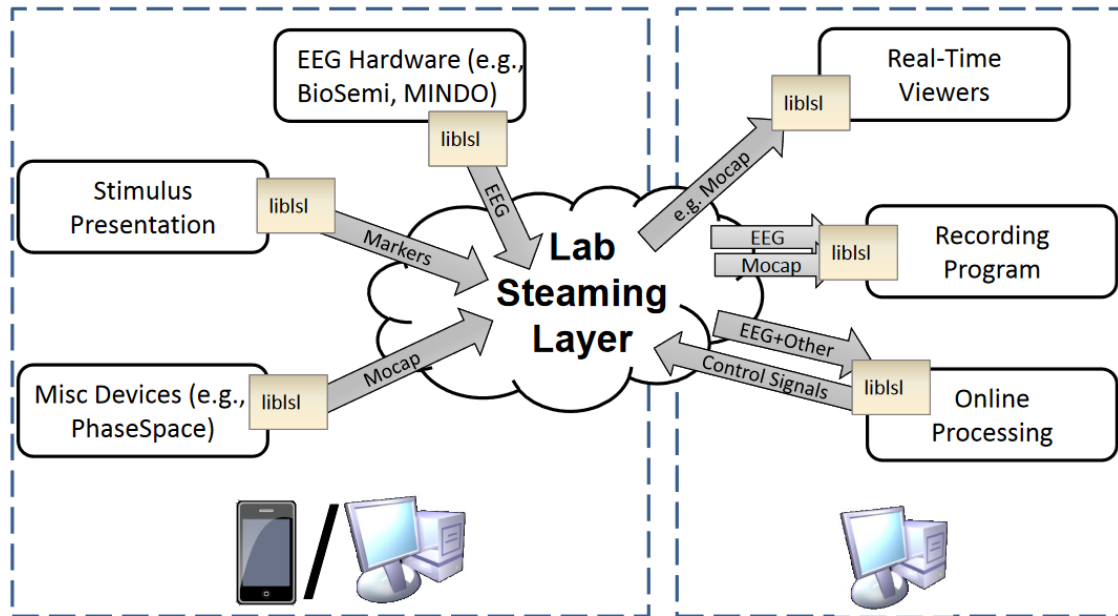
Labstreaminglayer (LSL)

Lab streaming layer (LSL) is a system for the unified collection of measurement time series in research experiments that handles both the networking, time-synchronization, (near-) real-time access as well as optionally the centralized collection, viewing and disk recording of the data.



Labstreaminglayer (LSL)

Network layer:



Labstreaminglayer (LSL)

Features:

- Open-source software library
- Highly extensible and supports wrappers for numerous languages
- Overlay network
- Real-time data streaming
- Facilitates multimodal data stream synchronization without hardware triggers

Labstreaminglayer (LSL)

Why use LSL: Synchronizing several devices.

EEG and ExG



Full-Body Motion Capture



Eye-Tracking



Human Interface Devices, System State, Etc.



Labstreaminglayer (LSL)

Why use LSL: Synchronizing several devices is a challenge:

- EEG/EMG/EOG – i.e. with BioSemi – 512 Hz
- Stimulus Presentation Markers – i.e. with Presentation – random times
- Motion Capture – i.e. with PhaseSpace – 400 Hz
- Eyetracking – i.e. with EyeLink – 60 Hz
- HCI, GSR, Heartrate, Forceplate, etc. ?? Hz

So we need:

- Attach timestamps to data
- Attach timestamps to record times
- Measure and test EVERYTHING OVER AND OVER

Labstreaminglayer (LSL)

So that LSL (liblsl) does, timestamping and streaming multimodal data across a network. Basically, provides methods for creating 3 objects:

1. stream info (metadata)
2. stream outlet (send data)
3. stream inlet (receive data)

```
from pylsl import StreamInfo, StreamOutlet
info = StreamInfo('Markers', 'Markers', 1, 0, 'string', 'mi-tutorial')
outlet = StreamOutlet(info)
outlet.push_sample(['left_cue'])    # sent the instant the cue appears
```

Labstreaminglayer (LSL)

So that LSL (liblsl) does, timestamping and streaming multimodal data across a network.
Basically, provides methods for creating 3 objects:

1. stream info (metadata)
 - name ('ActiChamp')
 - type ('EEG')
 - channel count ('64')
 - sample rate('512')
 - format ('float32')
 - source Id ('8JIAes263D' some such serial number)

Append meta-data

XML based information that adheres to a basic template, but can be extended to contain anything, anyhow

Labstreaminglayer (LSL)

So that LSL (liblsl) does, timestamping and streaming multimodal data across a network. Basically, provides methods for creating 3 objects:

2. stream outlet (send data)
 - stream info object
 - (optionally) chunk size
 - (optionally) data buffer size

Once created, a stream outlet object sits on an open socket(a data stream publisher) and can be pinged by clients to initiate connection (subscription). Later, a loop on a separate thread can stream data or poll for data (push sample) to output sporadically and thus destroy after using.

Labstreaminglayer (LSL)

So that LSL (liblsl) does, timestamping and streaming multimodal data across a network. Basically, provides methods for creating 3 objects:

3. stream inlet (receive data)

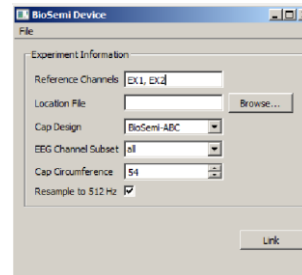
- @param prop ('name', 'type', etc.)
- @param value ('ActiChamp', 'EEG', etc.)
- @return std::vector<stream info>
- @param info (one of the stream info objects returned by resolve stream)(optional)
- @params buffer length, chunk length, recover (bool)

automatically connect to corresponding outlet, launch a 'listen' thread to repeatedly call pull sample. If recover=true, automatically respawn if lost. Destroy when all done.

Labstreaminglayer (LSL)

But how it actually works (not technically):

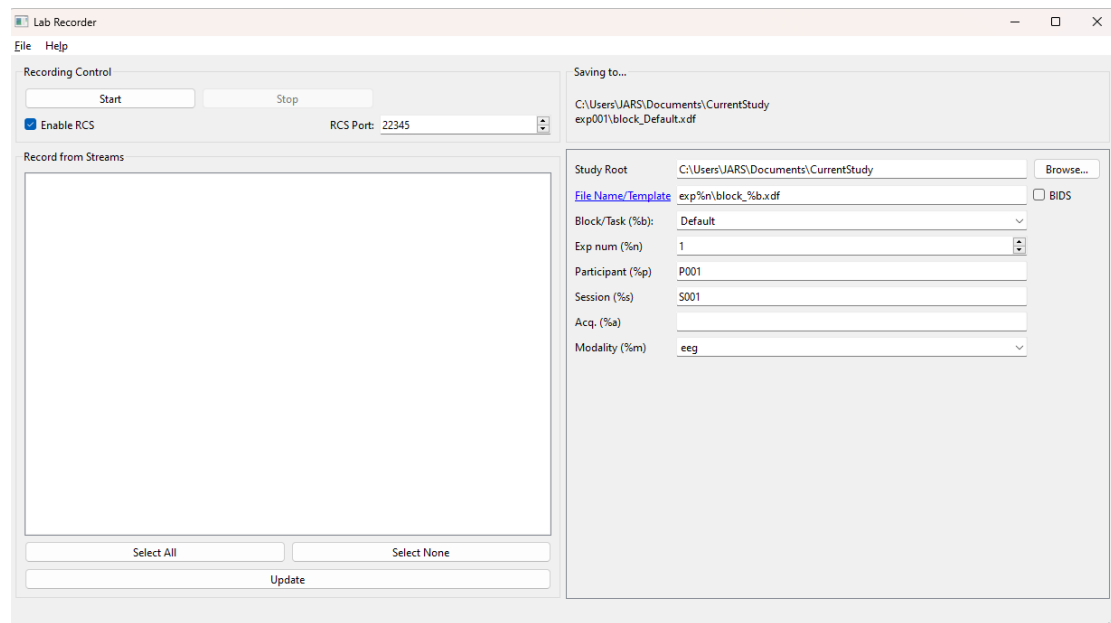
- Link to LSL and vendor libraries (see supported devices).
- Setup 'streaminfo' and metadata (xml)
- Open LSL 'outlet'
- Launch listener thread to 'push' LSL data as it arrives.



But we have also a main synchronizer: LabRecorder.

Lab Recorder

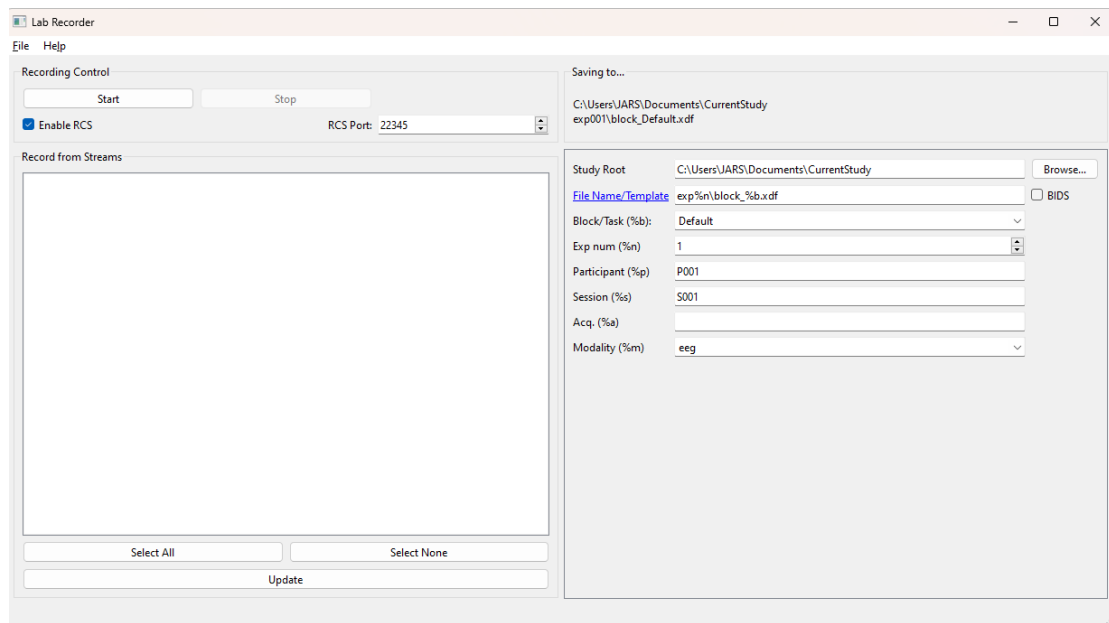
Your new best friend:



https://labstreaminglayer.readthedocs.io/info/supported_devices.html

Lab Recorder

Your new best friend:

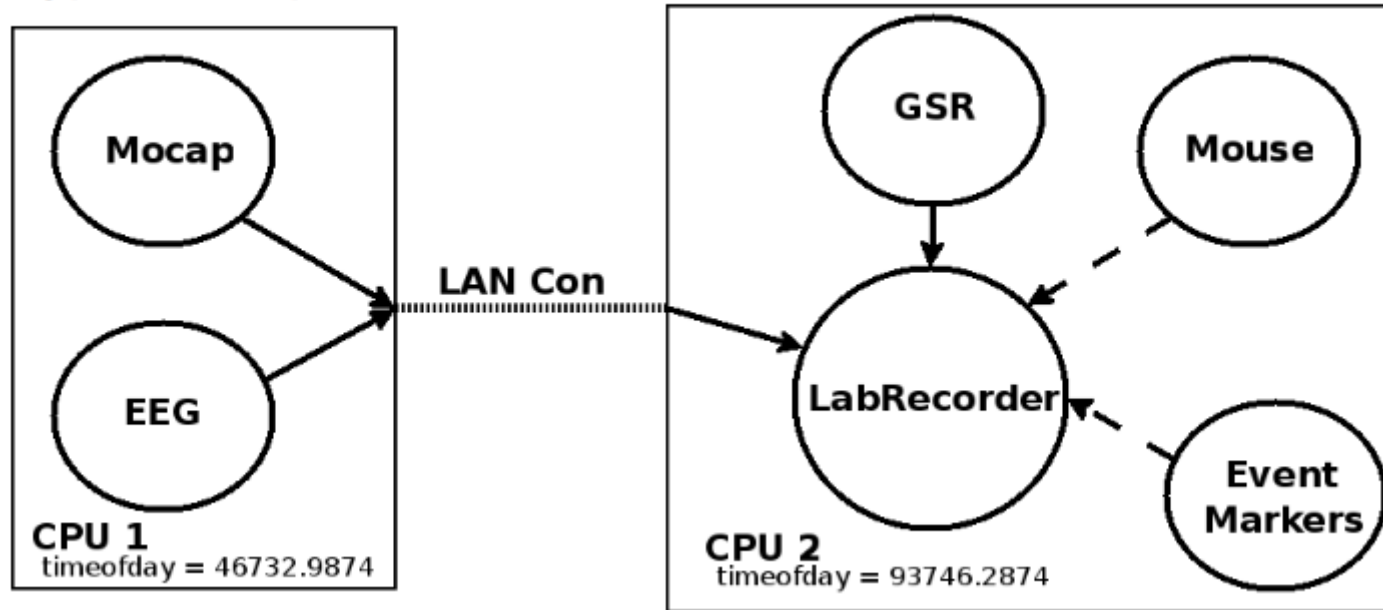


- Takes all of your streams (configured by the apps)
- Synchronize them
- Store in a .XDF file with an elegant data structure and filename.

<https://github.com/labstreaminglayer/App-LabRecorder/releases> Download and install it!

Lab Recorder

Typical Setup:



Lab Recorder

Lab Recorder

- Locate streams, write metadata to output file header
- Read streams, write data to output file
- Periodically check clock offsets (NTP), write offsets to output file footer

XDF File

- Organize meta-data into a data structure
- Linearize non-sporadic timestamps
- Use clock offsets and known latencies to synchronize data streams

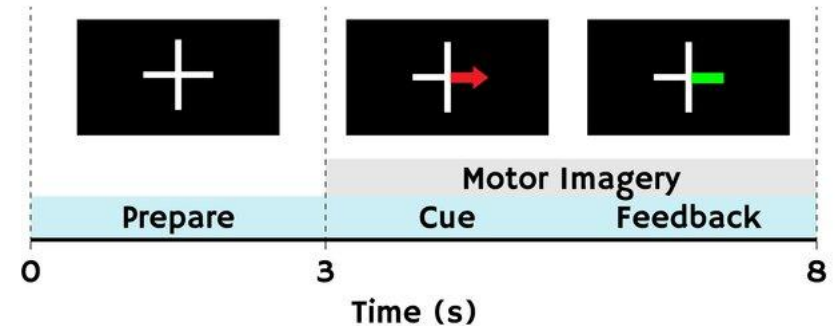
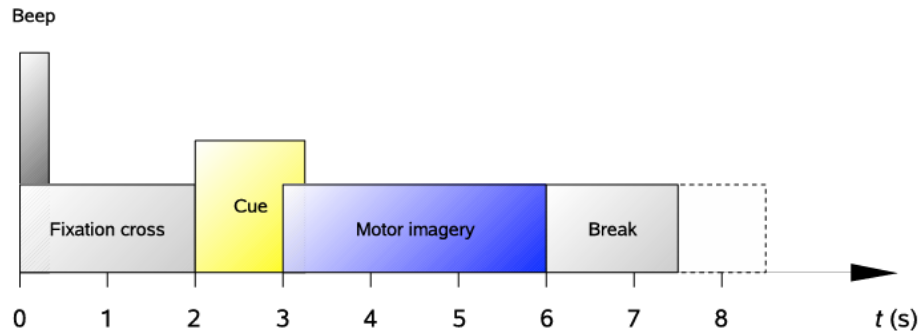
BCI Experiment

As we discussed , BCI has several paradigms (P300, SSVEP, Motor Imagery) associated to how the signal is produced (Exogenous, Endogenous), a specific neurophysiological patterns (P300, SSVEP, SMR) and type of BCI (Active, Reactive, Passive).

We will focus today in Motor Imagery paradigm, using Endogenous signals to identify Sensorimotor rhythms (SMR) under an Active BCI.

BCI Experiment

One of the most established paradigm in Motor Imagery BCI (MI-BCI) is the Graz BCI:



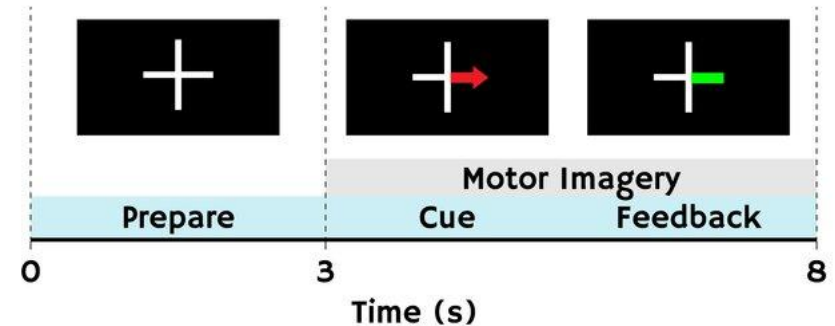
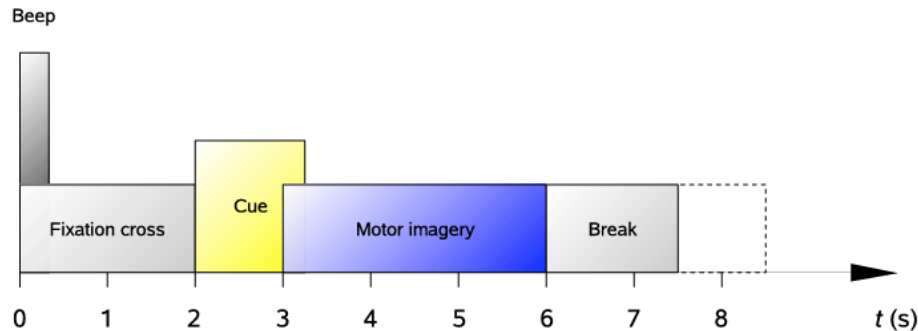
<https://ieee-dataport.org/documents/bci-competition-2008%E2%80%9393graz-data-set>

Atilla, Fred & Postma, Marie & Alimardani, Maryam. (2024). Gamification of Motor Imagery Brain-Computer Interface Training Protocols: a systematic review.

BCI Experiment

One of the most established paradigm in Motor Imagery BCI (MI-BCI) is the Graz BCI:

Lets create our BCI. Open BCI_Tutorial_ExperimentDesign.ipynb



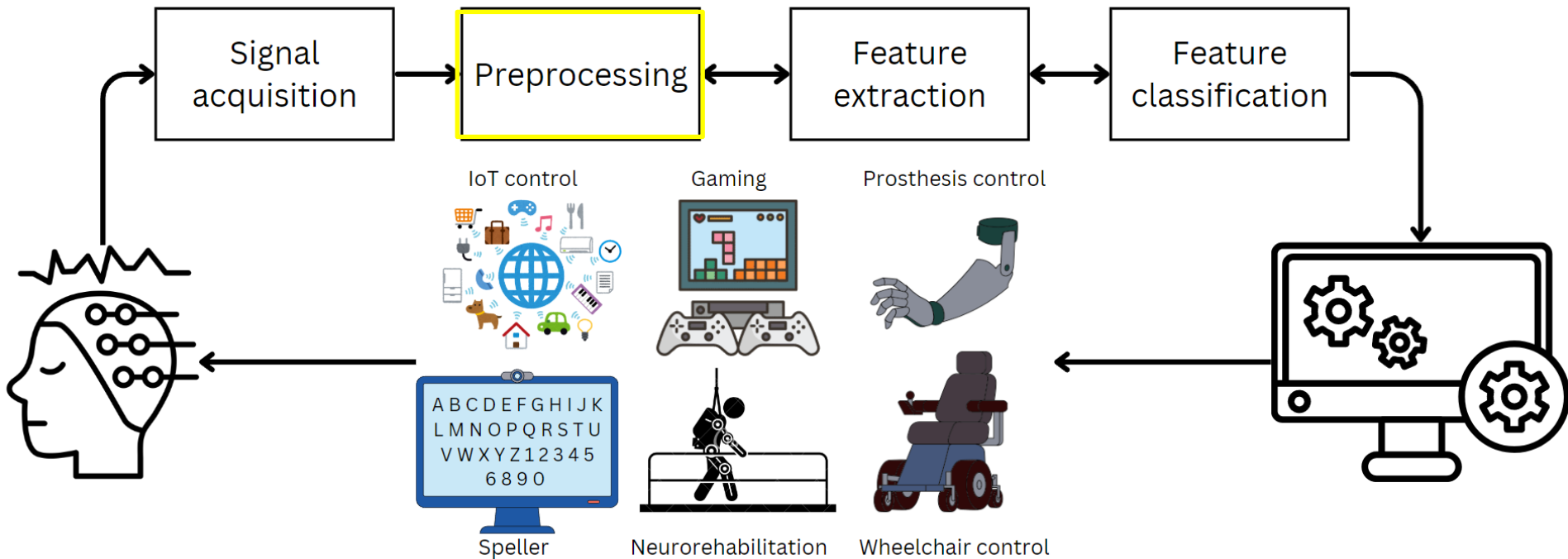
<https://ieee-dataport.org/documents/bci-competition-2008%E2%80%9393graz-data-set>

Atilla, Fred & Postma, Marie & Alimardani, Maryam. (2024). Gamification of Motor Imagery Brain-Computer Interface Training Protocols: a systematic review.

Today's plan

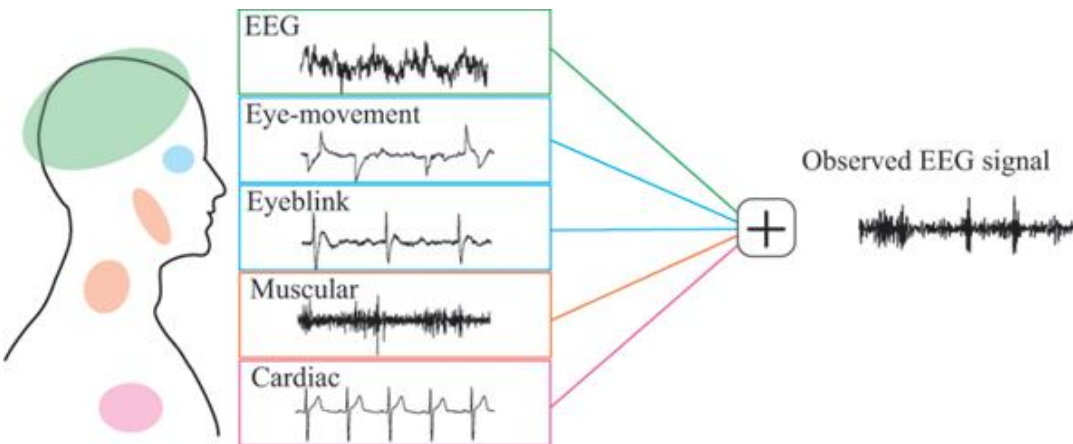
- ~~Brain-Computer Interface (BCI)~~
 - ~~Overview~~
 - ~~Stages – Pipeline~~
 - ~~Applications~~
 - ~~Neurophysiological patterns, Categorization, Paradigms~~
- ~~Create the experiment~~
 - ~~Psychopy~~
 - ~~Labstreaminglayer~~
- Pipeline
 - Signal processing
 - Feature extraction
 - Classification
 - Evaluation
- Hands on





Signal processing

EEG signals normally come contaminated with artifacts from:



Several techniques are employed to clean EEG signals, such as:

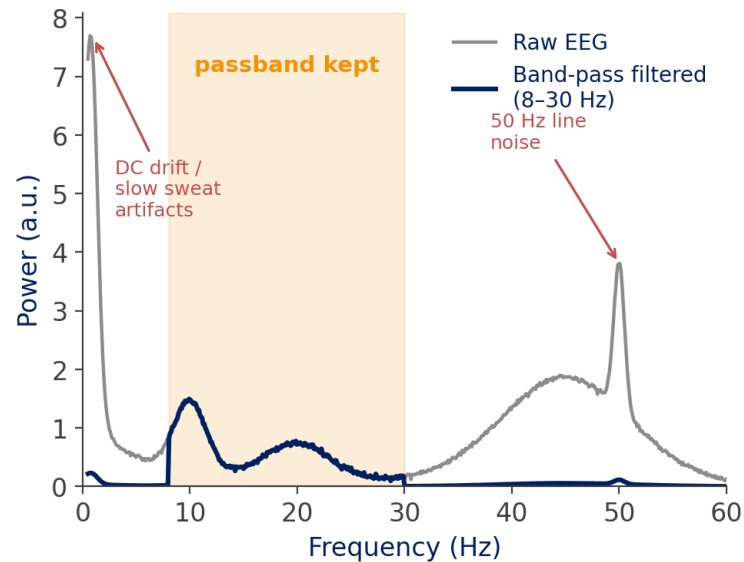
- Digital Filters (Finite Impulse Response or Infinite Impulse Response).
- Blind signal separation (Independent Component Analysis, ICA).
- Sub-sampling and signal normalization.
- Source reconstruction.

Raw EEG carries the neural signal mixed with slow drifts (electrode/skin polarization, sweat) and high-frequency noise (EMG, electronics). **Digital filters** (high-pass, low-pass, band-pass) keep only the frequency range of interest — e.g. 8–30 Hz for the mu/beta rhythms used in Motor Imagery.

Zero-phase (offline, e.g. **filtfilt**) vs. causal (online, stateful) filtering is a key design choice: zero-phase avoids delay/distortion but needs the whole signal in advance, so a real-time decoder must use a causal filter instead.

Advantages: cheap, fast, removes well-known noise bands with a handful of parameters.

Disadvantages: a too-narrow band can filter out genuine neural signal; causal filters add phase delay/edge transients; filter order & ripple need care.



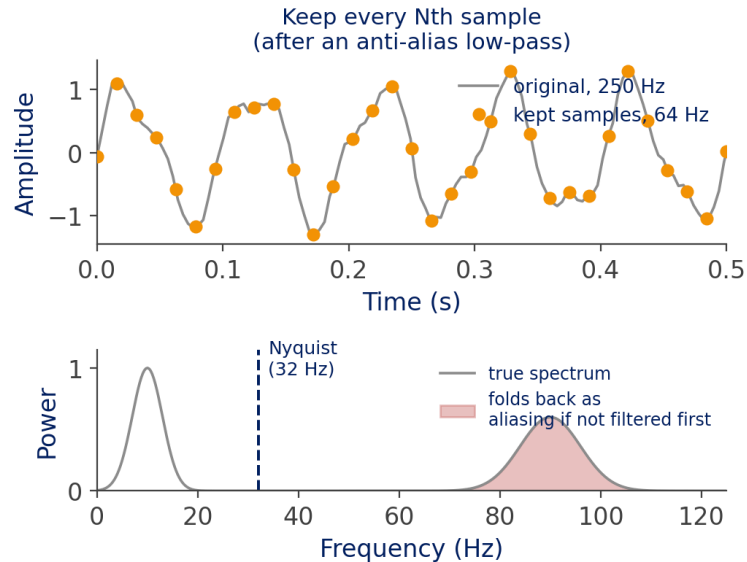
Sample Reduction (Downsampling)

EEG is often recorded at 250–1000+ Hz, far higher than most BCI features need. **Downsampling** keeps only every Nth sample, cutting file size and compute time for filtering, epoching and classification.

It must always be preceded by an **anti-alias low-pass filter** at (or below) the new Nyquist frequency — otherwise energy above that frequency folds back and contaminates the lower, kept frequencies (aliasing).

Advantages: faster pipelines, smaller files, matches the sampling rate features/models actually need.

Disadvantages: permanently discards high-frequency content; too aggressive a reduction can remove discriminative information (e.g. gamma-band features).



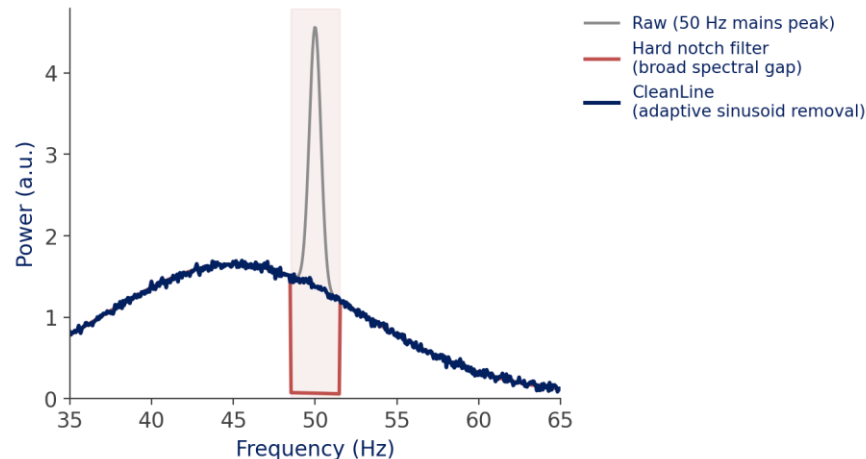
Line-Noise Removal (CleanLine)

Mains electricity injects a sharp, narrowband spike at 50 Hz (Europe) or 60 Hz (US), plus harmonics, into every channel.

A simple **notch filter** removes it but punches a broad gap in the spectrum, also damaging real neural activity nearby (e.g. low-gamma). **CleanLine** instead estimates the sinusoid's amplitude/phase per window (multi-taper regression) and subtracts only that component, leaving the rest of the spectrum intact.

Advantages: minimal spectral distortion; safe for studies that need frequencies close to the line frequency.

Disadvantages: more computationally demanding; needs enough data per window to estimate the sinusoid reliably; residual noise if the line frequency drifts.



Artifact Removal: ICA & ASR

Independent Component Analysis (**ICA**) decomposes multichannel EEG into maximally independent sources. Components matching stereotyped artifacts (eye blinks, muscle, heartbeat) are flagged and removed, then the rest are projected back to channel space.

Artifact Subspace Reconstruction (**ASR**) instead works on the continuous signal: it learns a statistical threshold from a clean calibration window, then reconstructs (via PCA) any window/channel whose variance exceeds it — well suited to running online.

Advantages: ICA removes artifact sources thoroughly and is well established; ASR needs no manual labeling and can correct large transients in real time.

Disadvantages: ICA needs enough stationary data to fit and is normally fit once, offline (then re-applied, not refit, for online use — as in this tutorial); ASR needs a genuinely clean calibration segment and can blur brief neural transients.



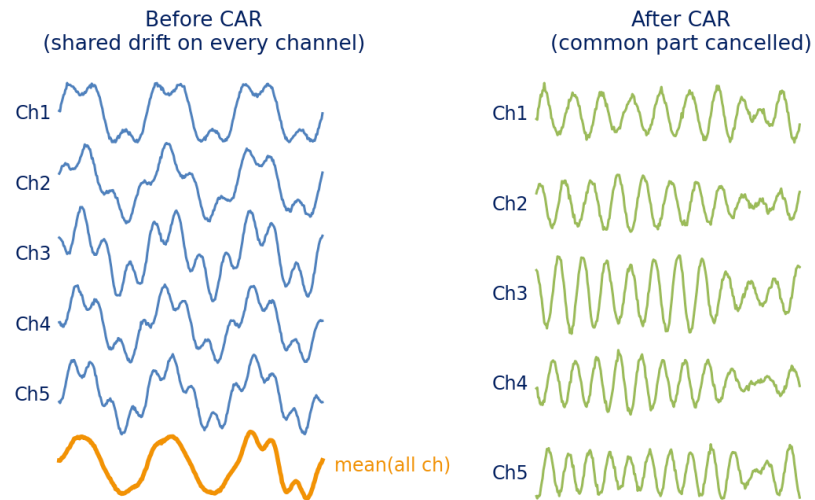
Common Average Reference (CAR)

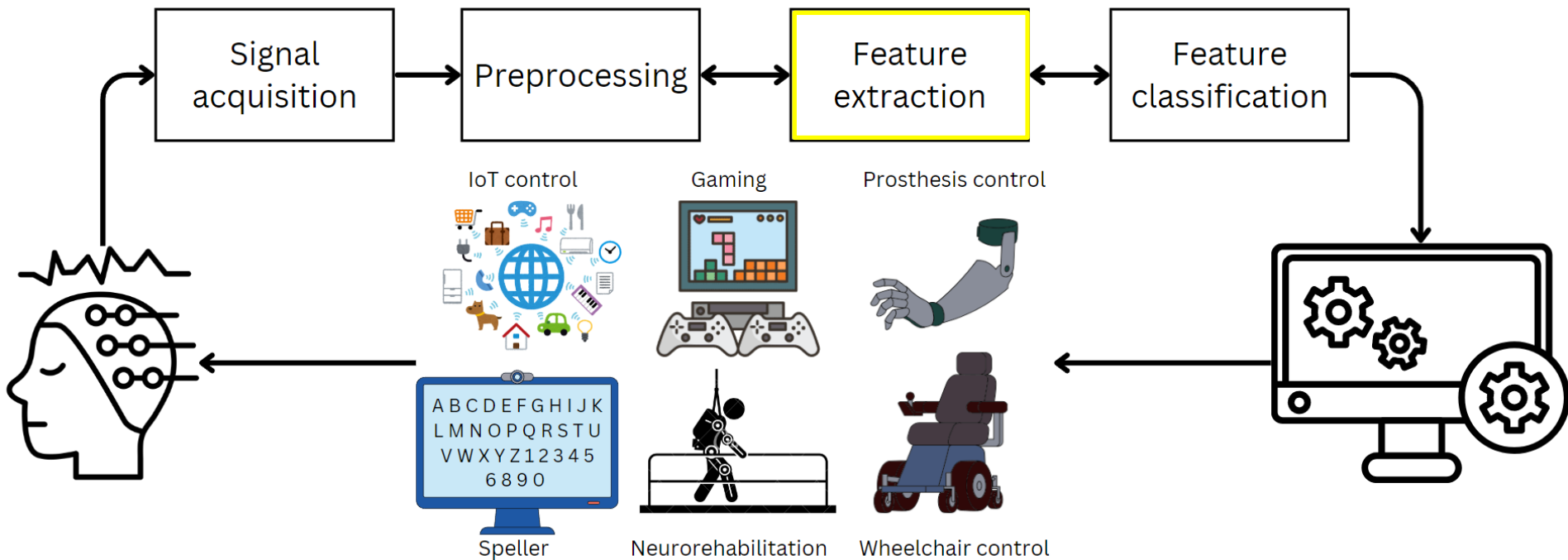
EEG is always a relative measurement: every channel's amplitude is defined with respect to a reference electrode, so whatever noise or bias sits on that reference leaks into every channel.

Common Average Reference re-references every channel by subtracting the instantaneous mean across all channels, approximating a neutral “zero” reference and emphasizing locally-generated activity.

Advantages: simple, free of extra hardware, cancels noise shared by all electrodes, no single reference channel to fail.

Disadvantages: assumes reasonably uniform, dense head coverage — with few/sparse channels it can distort rather than clean the signal; a very strong local source can leak into other channels through the average.





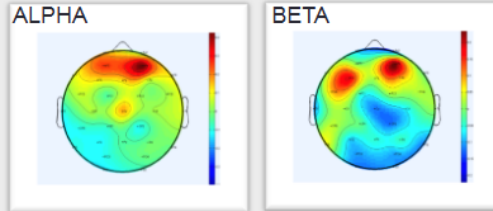
What are features?

- Features are some values computed from the signals
- Features should be
 - Representative of the signal
 - Reproducible
- Other criteria of the features will depend on the application
 - Smaller dimension than the signal
 - Inter-class variance high/intra-class variance low
 - Robust/enhanced representation of the signal (invariant to changes in noise, scale factors, etc.)

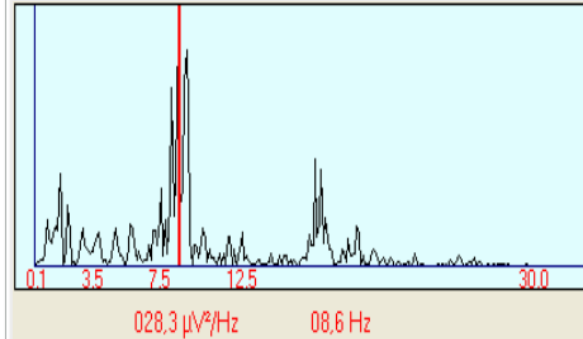
EEG representation

Topographical maps

→ plot EEG data on a map of the brain.
Data is interpolated between electrodes.
ERP maps → potential changes

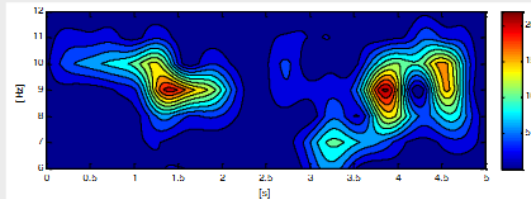


Spectral maps → frequency changes



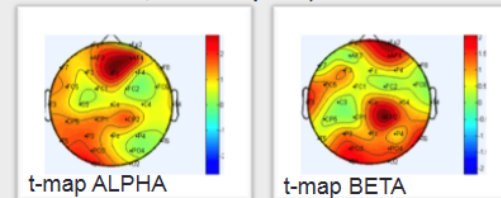
Time-frequency maps

→ time-frequency changes



Statistical maps

→ statistical comparisons (e.g. conditions, techniques)



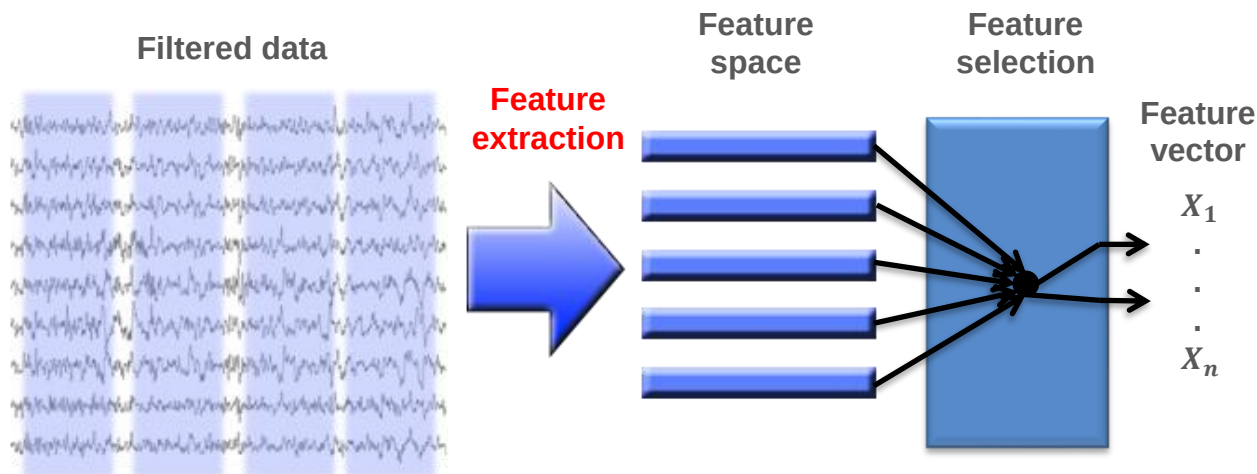
What are features?

- Features are some values computed from the signals
- Features should be
 - Representative of the signal
 - Reproducible
- Other criteria of the features will depend on the application
 - Smaller dimension than the signal
 - Inter-class variance high/intra-class variance low
 - Robust/enhanced representation of the signal (invariant to changes in noise, scale factors, etc.)

Some simple features

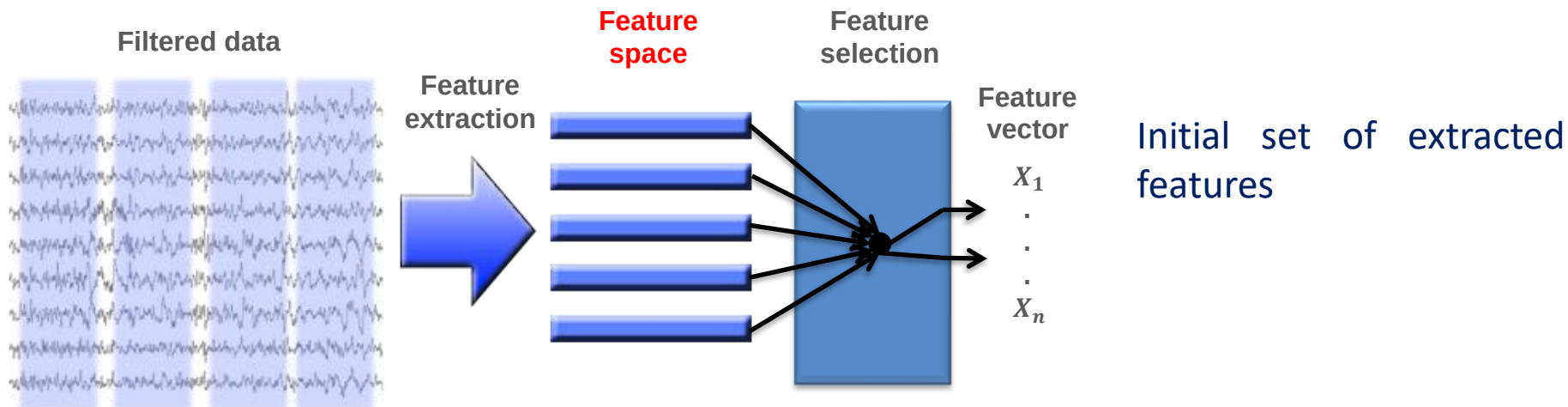
- Simple features are like
 - Mean (but not very useful as we usually set it to zero)
 - Standard deviation
 - Energy (which can be computed by using variance after setting mean to zero)
- Very often, we measure the energy in different spectral bands like delta, theta, alpha, beta and gamma and use them as features
 - To do this, we filter the EEG signal in the specific bands and then compute the energy

EEG signals being high dimensional data, the filtered signals are transformed into lower dimensional values, reducing irrelevant information while retaining the most relevant one.



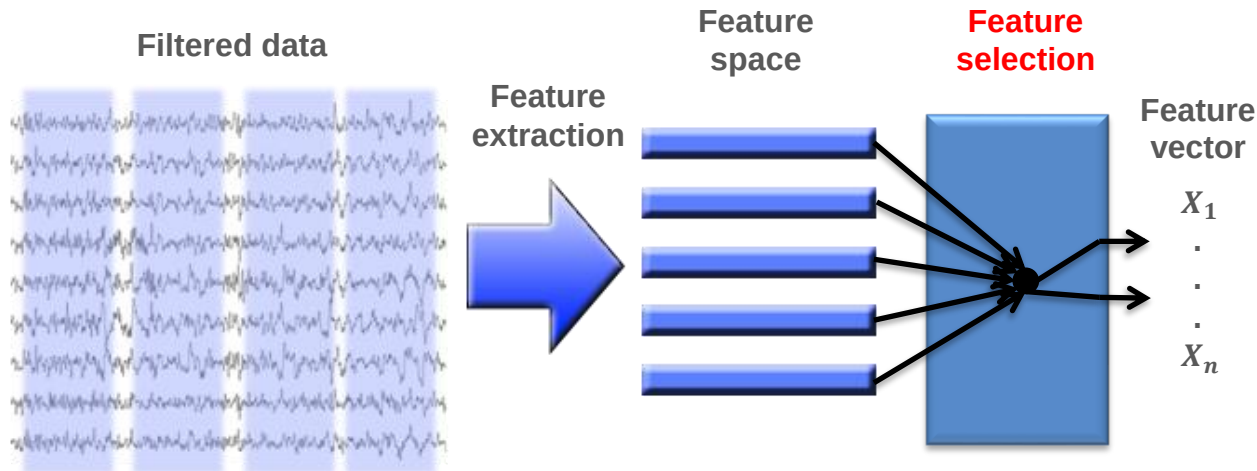
Methods or techniques to extract features. These could be temporal, spectral, hybrid, etc (we see later).

EEG signals being high dimensional data, the filtered signals are transformed into lower dimensional values, reducing irrelevant information while retaining the most relevant one.



Feature Extraction

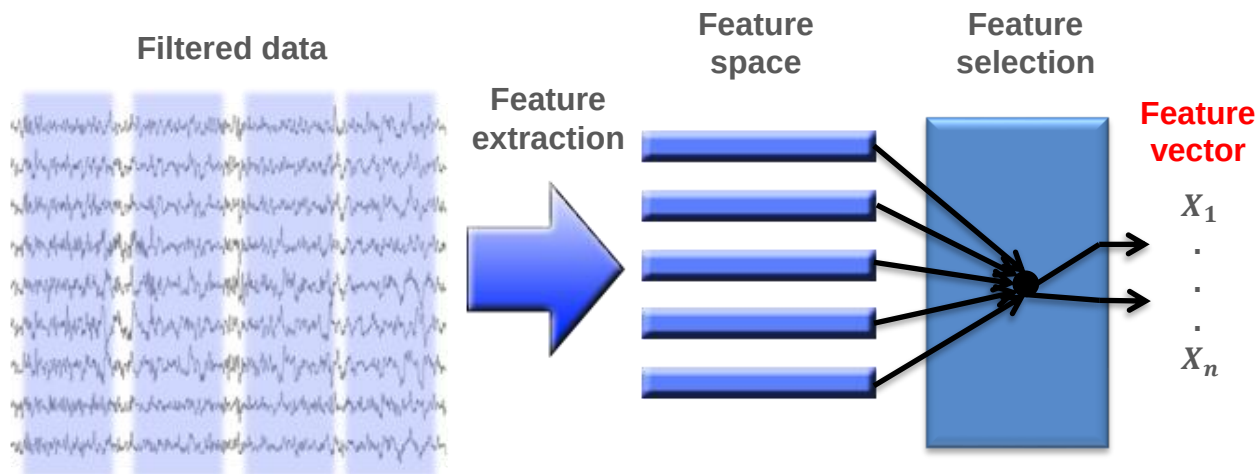
EEG signals being high dimensional data, the filtered signals are transformed into lower dimensional values, reducing irrelevant information while retaining the most relevant one.



Methods or techniques to keep the most relevant features (MRMR, Entropy, Information Gain).

Feature Extraction

EEG signals being high dimensional data, the filtered signals are transformed into lower dimensional values, reducing irrelevant information while retaining the most relevant one.



Final set of features to train the model.

Feature extraction methods

Time

Frequency

Time-
Frequency

Spatial

Advanced

Feature extraction methods

Time

Frequency

Time-
Frequency

Spatial

Advanced

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Time

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Time

- **Hjorth Parameters**
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

Introduced by Bo Hjorth in 1970, these describe the "shape" of a signal in the time domain using three statistical properties.

- **Activity:** Represents the signal power (variance).
- **Mobility:** Represents the mean frequency or the proportion of standard deviation of the power spectrum.
- **Complexity:** Indicates the "similarity" of the signal to a pure sine wave (how much the frequency changes).

Time

- Hjorth Parameters
- **Statistical Features**
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

These are standard descriptive statistics that quantify the distribution of EEG amplitudes within a time window.

- **Skewness:** Measures the asymmetry of the amplitude distribution.
- **Kurtosis:** Measures the "peakedness" or the presence of outliers (e.g., artifacts like eye blinks often increase kurtosis).
- **RMS (Root Mean Square):** Measures the average power/magnitude of the signal.

Time

- Hjorth Parameters
- Statistical Features
- **Fractal dimension**
- Kalman filter
- Particle Filter
- Matched filter

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

FD measures the **complexity** or "self-similarity" of the EEG signal. A more "jagged" or complex signal (like high-frequency noise or active processing) has a higher fractal dimension.

- **Higuchi Fractal Dimension (HFD)**: The most common algorithm for EEG due to its accuracy with short segments.
- **Katz Fractal Dimension**: Simpler and faster but often less robust than Higuchi.

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- **Kalman filter**
- **Particle Filter**
- Matched filter

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

These are **state-space models** used more for **signal enhancement** or **parameter tracking** than as static features.

- **Kalman Filter:** An iterative mathematical process that uses a series of measurements observed over time (containing noise) to produce estimates of unknown variables. In BCI, it's often used to track the "true" brain state or for motor imagery trajectory decoding.
- **Particle Filter:** A non-linear version of the Kalman filter. It uses "particles" to represent the probability distribution of the brain state, making it useful when the signal noise is non-Gaussian.

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- **Matched filter**

Time Domain Features are extracted directly from the raw or preprocessed EEG amplitude values over time. Unlike frequency-domain methods, they are computationally efficient and capture the morphology (shape) and temporal evolution of the signal.

A matched filter is the optimal linear filter for maximizing the Signal-to-Noise Ratio (SNR) in the presence of additive noise.

- **Concept:** You "template match" the incoming EEG against a known wave shape (like a specific P300 component or an Evoked Potential).
- **Usage:** If the filter output crosses a threshold, the BCI system detects that specific brain pattern has occurred.

Feature extraction methods

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Frequency

- Discrete Fourier Transform
- Fast Fourier Transform

Time-Frequency

Spatial

Advanced

Frequency

Moving to the Frequency Domain, we shift our focus from "how the signal changes over time" to "how much of each frequency is present in the signal." This is critical for BCI because brain states (like relaxation vs. concentration) are highly correlated with specific frequency bands.

- Discrete Fourier Transform
- Fast Fourier Transform

Frequency

- **Discrete Fourier Transform**
- **Fast Fourier Transform**

Moving to the Frequency Domain, we shift our focus from "how the signal changes over time" to "how much of each frequency is present in the signal." This is critical for BCI because brain states (like relaxation vs. concentration) are highly correlated with specific frequency bands.

Discrete Fourier Transform (DFT): Converts a finite sequence of equally-spaced samples of a signal into a list of coefficients of finite combinations of complex exponentials.

Fast Fourier Transform (FFT): An efficient algorithm to compute the DFT. It reduces the computational complexity from $O(N^2)$ to $O(N \log N)$, making real-time BCI possible.

Usage: It identifies the "spectral content"—the amplitude and phase of each frequency component in the EEG.

Frequency

- **Discrete Fourier Transform**
- **Fast Fourier Transform**
- **Power Spectral Density (PSD)**

Moving to the Frequency Domain, we shift our focus from "how the signal changes over time" to "how much of each frequency is present in the signal." This is critical for BCI because brain states (like relaxation vs. concentration) are highly correlated with specific frequency bands.

PSD describes how the power of your EEG signal is distributed over frequency. Unlike a raw FFT, PSD (often calculated via Welch's method) provides a smoother estimate by averaging over windowed segments, which reduces noise.

Absolute Power: The actual power in a specific band (μV^2).

Relative Power: The power in a band divided by the total power (useful for reducing inter-subject variability).

Frequency

- **Discrete Fourier Transform**
- **Fast Fourier Transform**
- **Power Spectral Density (PSD)**
- **Power bands**

Moving to the Frequency Domain, we shift our focus from "how the signal changes over time" to "how much of each frequency is present in the signal." This is critical for BCI because brain states (like relaxation vs. concentration) are highly correlated with specific frequency bands.

Delta (0.5–4 Hz): Deep sleep, pathology.

Theta (4–8 Hz): Drowsiness, meditation.

Alpha (8–13 Hz): Relaxed wakefulness (eyes closed).

Beta (13–30 Hz): Active thinking, focus, motor engagement.

Gamma (>30 Hz): High-level cognitive processing.

Feature extraction methods

Frequency

- **DFT**
- **FFT**
- Power Spectral Density (PSD)
- Power bands
- **Probability Density Function (PDF)**

Moving to the Frequency Domain, we shift our focus from "how the signal changes over time" to "how much of each frequency is present in the signal." This is critical for BCI because brain states (like relaxation vs. concentration) are highly correlated with specific frequency bands.

In the frequency domain, we can treat the normalized PSD as a probability distribution.

Spectral Entropy: Measures the complexity or "flatness" of the spectrum. A "peaky" spectrum (dominant alpha wave) has low entropy, while a white-noise-like spectrum (broadband activity) has high entropy.

Usage: Excellent for detecting changes in consciousness or cognitive load.

Feature extraction methods

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Frequency

- Discrete Fourier Transform
- Fast Fourier Transform
- PSD
- Power Bands
- PDF

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Spatial

Advanced

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Moving into Time-Frequency Domain features, we address the biggest limitation of pure frequency analysis: the loss of temporal information. While a standard FFT tells you what frequencies are present, time-frequency methods tell you when those frequencies occur. This is vital for BCI tasks like Motor Imagery, where we need to track the exact moment a Mu-rhythm desynchronizes.

Time-Frequency

- **Autoregressive model**
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Moving into Time-Frequency Domain features, we address the biggest limitation of pure frequency analysis: the loss of temporal information. While a standard FFT tells you what frequencies are present, time-frequency methods tell you when those frequencies occur. This is vital for BCI tasks like Motor Imagery, where we need to track the exact moment a Mu-rhythm desynchronizes.

AR models treat the EEG signal as a linear stochastic process. The current value of the signal is predicted as a weighted sum of its previous values plus noise.

Concept: The AR coefficients themselves become the "features."

BCI Benefit: They provide a very smooth spectral estimate even with very short data segments, which is common in real-time BC

Time-Frequency

- Autoregressive model
- **Short-time Fourier Transform**
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Moving into Time-Frequency Domain features, we address the biggest limitation of pure frequency analysis: the loss of temporal information. While a standard FFT tells you what frequencies are present, time-frequency methods tell you when those frequencies occur. This is vital for BCI tasks like Motor Imagery, where we need to track the exact moment a Mu-rhythm desynchronizes.

STFT works by dividing the EEG signal into several small, overlapping windows and applying a Fourier Transform to each.

Concept: It creates a "Spectrogram," showing frequency changes over time.

Trade-off: It suffers from the Heisenberg Uncertainty Principle—you can have good time resolution OR good frequency resolution, but not both simultaneously (determined by your window length).

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- **Continuous Wavelet Transform**
- **Discrete Wavelet Transform**

Moving into Time-Frequency Domain features, we address the biggest limitation of pure frequency analysis: the loss of temporal information. While a standard FFT tells you what frequencies are present, time-frequency methods tell you when those frequencies occur. This is vital for BCI tasks like Motor Imagery, where we need to track the exact moment a Mu-rhythm desynchronizes.

Wavelets solve the resolution problem of STFT by using a variable window size. They use long windows for low frequency (good frequency resolution) and short windows for high frequency (good time resolution).

CWT: Uses a continuous range of scales and shifts; excellent for detailed visual analysis and capturing transient ERPs.

DWT: Decomposes the signal into "Approximation" (low-pass) and "Detail" (high-pass) coefficients. It is much faster and often used for denoising or feature reduction.

Feature extraction methods

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Frequency

- Discrete Fourier Transform
- Fast Fourier Transform
- PSD
- Power Bands
- PDF

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Spatial

- Laplacian
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- XDAWN

Advanced

Spatial

In the Spatial Domain, we stop looking at when or at what frequency a signal occurs and focus on where it comes from on the scalp. Because the skull acts as a low-pass spatial filter (blurring the electrical signals), these methods aim to "sharpen" the data or find the specific electrode combinations that best separate different brain states.

- Laplacian
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- XDAWN

Spatial

- **Laplacian**
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- XDAWN

In the Spatial Domain, we stop looking at when or at what frequency a signal occurs and focus on where it comes from on the scalp. Because the skull acts as a low-pass spatial filter (blurring the electrical signals), these methods aim to "sharpen" the data or find the specific electrode combinations that best separate different brain states.

The Surface Laplacian is a high-pass spatial filter used to reduce the effects of volume conduction. It calculates the second spatial derivative of the potential on the scalp.

Concept: For a target electrode, it subtracts the average activity of its nearest neighbors. This "cleans" the signal by removing common activity shared across the scalp, highlighting local cortical activity directly under the electrode.

BCI Use: Often used as a preprocessing step for Motor Imagery to isolate activity over the motor cortex (C3, C4).

Spatial

- Laplacian
- **Common Spatial Features (CSP)**
- Independent Component Analysis (ICA)
- XDAWN

In the Spatial Domain, we stop looking at when or at what frequency a signal occurs and focus on where it comes from on the scalp. Because the skull acts as a low-pass spatial filter (blurring the electrical signals), these methods aim to "sharpen" the data or find the specific electrode combinations that best separate different brain states.

CSP is a supervised algorithm that finds spatial filters that maximize the variance of the signal for one class while minimizing it for another.

Concept: It "rotates" the multi-channel data so that the resulting components have maximum power difference between two tasks (e.g., Left Hand vs. Right Hand imagery).

BCI Use: The "Gold Standard" for oscillatory-based BCIs. It effectively reduces 32 or 64 channels down to a few highly discriminative components.

Feature extraction methods

Spatial

- Laplacian
- Common Spatial Features (CSP)
- **Independent Component Analysis (ICA)**
- XDAWN

In the Spatial Domain, we stop looking at when or at what frequency a signal occurs and focus on where it comes from on the scalp. Because the skull acts as a low-pass spatial filter (blurring the electrical signals), these methods aim to "sharpen" the data or find the specific electrode combinations that best separate different brain states.

While often used for artifact removal, ICA is a powerful spatial feature extraction tool. It assumes the EEG is a linear mix of independent "source" signals.

Concept: It blindly separates the scalp data into independent components (ICs). Unlike CSP, it doesn't need "labels" (unsupervised).

BCI Use: Identifying specific brain sources (e.g., a "Mu-rhythm" component) that stay spatially stable even if the frequency varies slightly.

Spatial

- Laplacian
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- **XDAWN**

In the Spatial Domain, we stop looking at when or at what frequency a signal occurs and focus on where it comes from on the scalp. Because the skull acts as a low-pass spatial filter (blurring the electrical signals), these methods aim to "sharpen" the data or find the specific electrode combinations that best separate different brain states.

It is similar to CSP but designed specifically for Evoked Potentials (ERPs) like the P300.

Concept: It finds spatial filters that maximize the Signal-to-Signal-plus-Noise Ratio (SSNR) of the ERP template.

BCI Use: Boosting the detection of P300 waves in spellers.

Feature extraction methods

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Frequency

- Discrete Fourier Transform
- Fast Fourier Transform
- PSD
- Power Bands
- PDF

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Spatial

- Laplacian
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- XDAWN

Advanced

- Riemannian Geometry
- Tensors
- CNN

Advanced

In the Advanced Domain, we move beyond simple signal shapes and frequencies into Manifold Learning and Deep Learning. These methods often handle the "end-to-end" pipeline, meaning they can learn to extract features and classify them simultaneously, often outperforming traditional methods in complex BCI tasks.

- Riemannian Geometry
- Tensors
- CNN

Feature extraction methods

Advanced

- **Riemannian Geometry**
- Tensors
- CNN

In the Advanced Domain, we move beyond simple signal shapes and frequencies into Manifold Learning and Deep Learning. These methods often handle the "end-to-end" pipeline, meaning they can learn to extract features and classify them simultaneously, often outperforming traditional methods in complex BCI tasks.

Traditional BCI methods (like CSP) use Euclidean geometry, which doesn't always "fit" the curved space of brain signals. Riemannian Geometry treats the covariance matrix of EEG segments as points on a curved manifold of Symmetric Positive Definite (SPD) matrices.

Concept: Instead of flattening data into a vector, we calculate the distance between brain states directly on this curved surface using the Riemannian metric.

BCI Use: Highly robust for Motor Imagery and ERPs; it handles noise and small datasets better than almost any other method.

Feature extraction methods

Advanced

- Riemannian Geometry
- **Tensors**
- CNN

In the Advanced Domain, we move beyond simple signal shapes and frequencies into Manifold Learning and Deep Learning. These methods often handle the "end-to-end" pipeline, meaning they can learn to extract features and classify them simultaneously, often outperforming traditional methods in complex BCI tasks.

EEG data is naturally multi-dimensional (Channels $\times$ Time $\times$ Frequencies $\times$ Subjects). Tensors allow us to analyze all these dimensions simultaneously without "flattening" them into a 2D matrix.

Concept: Methods like Canonical Polyadic (CP) or Tucker Decomposition break a large data "cube" into core components that represent specific brain patterns across time, space, and frequency.

BCI Use: Excellent for Multi-subject Transfer Learning and finding stable patterns that persist across different sessions.

Feature extraction methods

Advanced

- Riemannian Geometry
- Tensors
- **CNN**

In the Advanced Domain, we move beyond simple signal shapes and frequencies into Manifold Learning and Deep Learning. These methods often handle the "end-to-end" pipeline, meaning they can learn to extract features and classify them simultaneously, often outperforming traditional methods in complex BCI tasks.

CNNs are the most prominent Deep Learning tool in BCI. Unlike images, EEG CNNs use specialized kernels to capture temporal and spatial patterns separately.

Concept: Modern architectures like EEGNet or ShallowConvNet use "Depthwise" and "Separable" convolutions to mimic spatial filters (like CSP) and frequency filters (like bandpass) automatically.

BCI Use: "End-to-end" decoding where the model learns the best features for a specific user without manual band selection.

Feature extraction methods

Time

- Hjorth Parameters
- Statistical Features
- Fractal dimension
- Kalman filter
- Particle Filter
- Matched filter

Frequency

- Discrete Fourier Transform
- Fast Fourier Transform
- PSD
- Power Bands
- PDF

Time-Frequency

- Autoregressive model
- Short-time Fourier Transform
- Continuous Wavelet Transform
- Discrete Wavelet Transform

Spatial

- Laplacian
- Common Spatial Features (CSP)
- Independent Component Analysis (ICA)
- XDAWN

Advanced

- Riemannian Geometry
- Tensors
- CNN

Feature extraction methods – Time Domain

Method	Characteristics	Advantages	Disadvantages
Hjorth Parameters	Stats describing signal shape (Activity, Mobility, Complexity).	Extremely fast ; captures complexity with low CPU.	Highly sensitive to high-frequency noise/artifacts.
Statistical Features	Standard measures: Mean, Variance, Skewness, Kurtosis.	Simple to calculate; identifies signal distribution.	Poor physiological interpretation when used alone.
Fractal Dimension	Quantifies signal complexity/self-similarity.	Excellent for identifying non-linear brain states.	Computationally heavy; sensitive to segment length.
Kalman Filter	Iterative state-space estimator for hidden variables.	Optimal for real-time tracking of brain states.	Requires a predefined mathematical model of the brain.
Particle Filter	Non-linear version of the Kalman filter.	Handles highly complex, non-linear brain dynamics.	High computational cost and memory requirements.
Matched Filter	Correlation of a signal with a known "template" wave.	Optimal SNR for specific events (e.g., P300).	Fails if the brain response varies from the template.

Feature extraction methods – Frequency Domain

Method	Characteristics	Advantages	Disadvantages
DFT / FFT	Transforms time data into discrete frequency bins.	Efficient; provides a full spectral map.	Assumes the signal is "stationary" (static over time).
PSD (Welch)	Estimates power distribution using window averaging.	Reduces noise compared to a raw FFT.	Lower frequency resolution due to segmenting.
Power Bands	Calculates total power in bands (Alpha, Beta, etc.).	Highly interpretable; matches clinical rhythms.	Loses all temporal timing information.
PDF	Probability distribution of spectral amplitudes.	Captures statistical variability of frequency content.	Requires longer data segments for statistical validity.

Feature extraction methods – Time-Frequency Domain

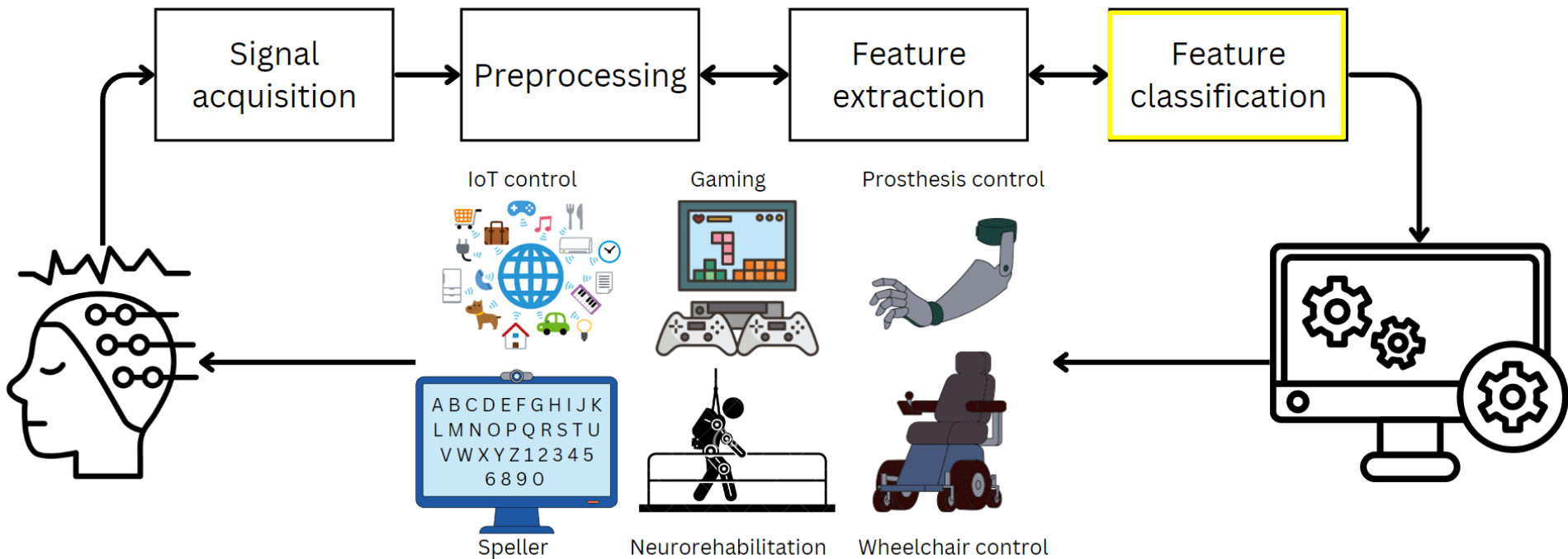
Method	Characteristics	Advantages	Disadvantages
AR Model	Predicts current samples based on past samples.	High resolution in very short time windows.	Highly sensitive to the "Model Order" selection.
STFT	Computes FFTs over small, sliding time windows.	Shows frequency evolution over time (Spectrogram).	Fixed resolution: Cannot have both time and freq precision.
CWT / DWT	Decomposes signal using variable-scaled wavelets.	Multi-resolution: Best for short, transient events.	Subjective selection of the "Mother Wavelet".

Feature extraction methods – Spatial

Method	Characteristics	Advantages	Disadvantages
Laplacian	Subtracts neighbor average to highlight local activity.	Sharpens local signals; reduces volume conduction.	Heavily dependent on electrode density/layout.
CSP	Maximizes variance for one class vs. another.	Gold Standard for Motor Imagery separation.	Sensitive to noise; requires labeled training data.
ICA	Separates signal into independent source components.	Best for artifact removal (blinks, muscle).	Computationally slow; components are unordered.
xDAWN	Spatial filtering optimized for Evoked Potentials.	Greatly enhances P300 detection in noisy data.	Only useful for time-locked (evoked) paradigms.

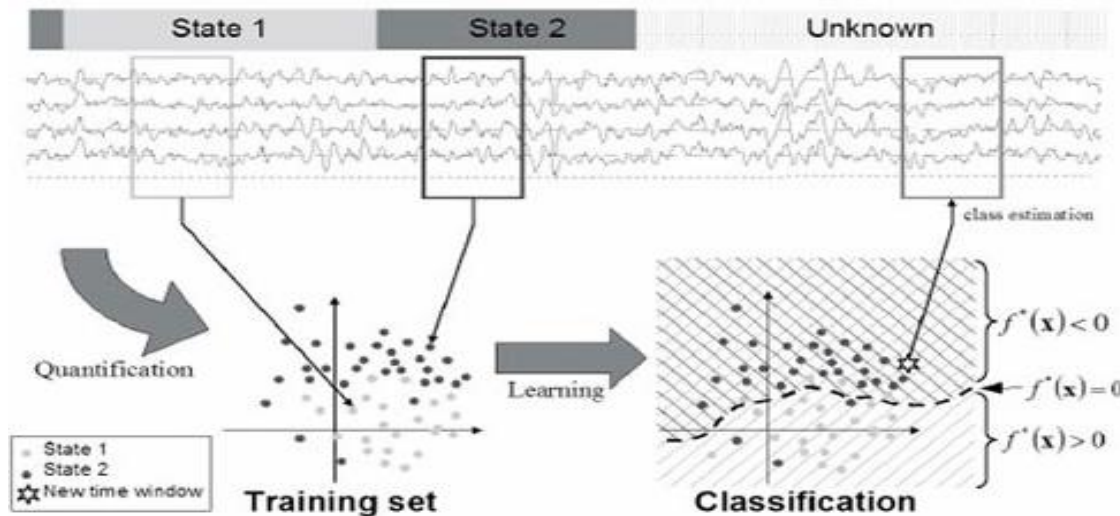
Feature extraction methods – Advanced methods

Method	Characteristics	Advantages	Disadvantages
Riemannian	Maps covariance matrices on a curved manifold.	Extremely robust; works with small datasets.	Abstract math; difficult for students to visualize.
Tensors	Treats EEG as a multi-way "cube" (Time x Space x Freq).	Captures cross-subject and cross-session patterns.	High computational load; prone to overfitting.
CNN	Deep learning that learns filters automatically.	End-to-End: No manual feature extraction.	"Black Box" nature; needs large datasets.



Feature classification

The feature vector obtained is used as input to a classifier, to assign a class to a set of features that allows identifying the mental state of the user. Initially, a set of labelled observation is used to create a classification model. Subsequently, the model is used to estimate the class of the new data.



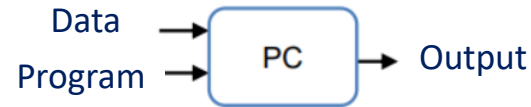
- K-Nearest Neighbors (KNN)
- Artificial Neural Networks (ANN)
- Naïve Bayes (NB).
- Support Vector Machine (SVM)
- Linear Discriminant Analysis (LDA).

What is Machine Learning (ML)?

ML is the study of algorithms that

- improve your performance P
- on a T task
- through the E experience

Traditional programming



Machine Learning



A well-defined ML task is given by (P, T, E)

Examples:

T: Play Chess

P: Percentage of games won

E: Games against oneself/others

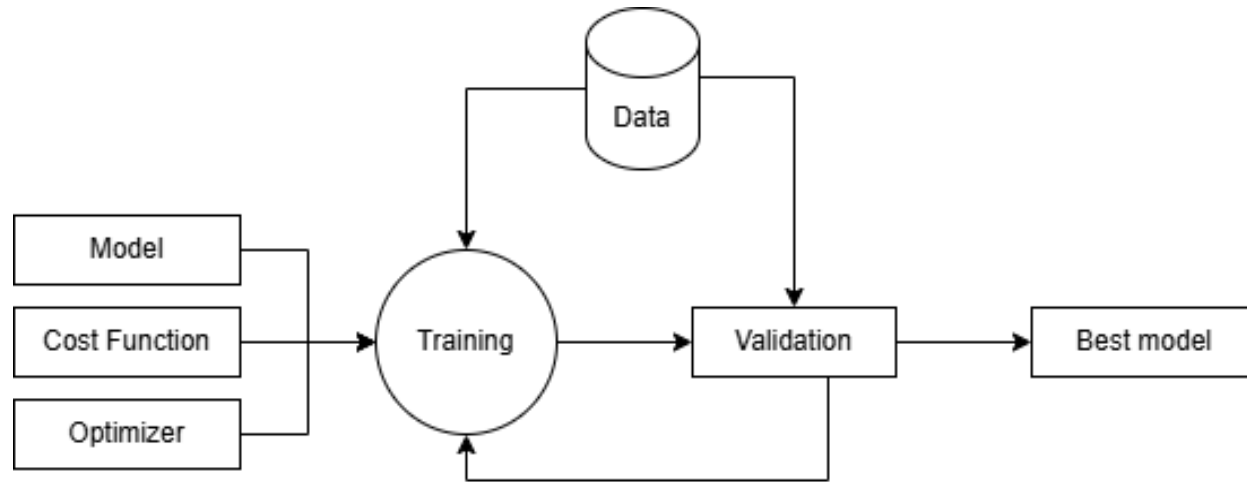
T: Driving on a track using cameras

P: Distance traveled before a human error

E: sequence of images and recorded turn commands of a human

How to create a ML model?

Training



Testing

-The best model is used with the testing data

How to create a ML model?

A model $h(x)$ represents the relationship between the input and output data:

- Predicting the output Y given a input X

Example:

Classification problem: what is the category of each image.

$$h(\text{🍏}) = \text{manzana}$$

$$h(\text{🍅}) = \text{tomate}$$

$$h(\text{🐮}) = \text{vaca}$$

A model $h(x)$ is also called “hypothesis function” (h)

Any hypothesis function (h) has parameters (which are “learned”)

This parameter learning is done by the training set only

How to create a ML model?

Hypothesis space

- It is a "type of model": it groups similar hypothesis functions (h)
- Each hypothesis function (h) belongs to a hypothesis space
 - Each h in a hypothesis space has different parameters

“No free lunch” rule:

When using a hypothesis function (a model) you always make assumptions: there is no function that is useful for all cases.



How to create a ML model?

A cost function (J) measures how good a hypothesis function is

Example: Sum of squared differences between predicted and actual

$$J(x) = \sum (h(x) - y)^2$$

What is the best hypothesis function?

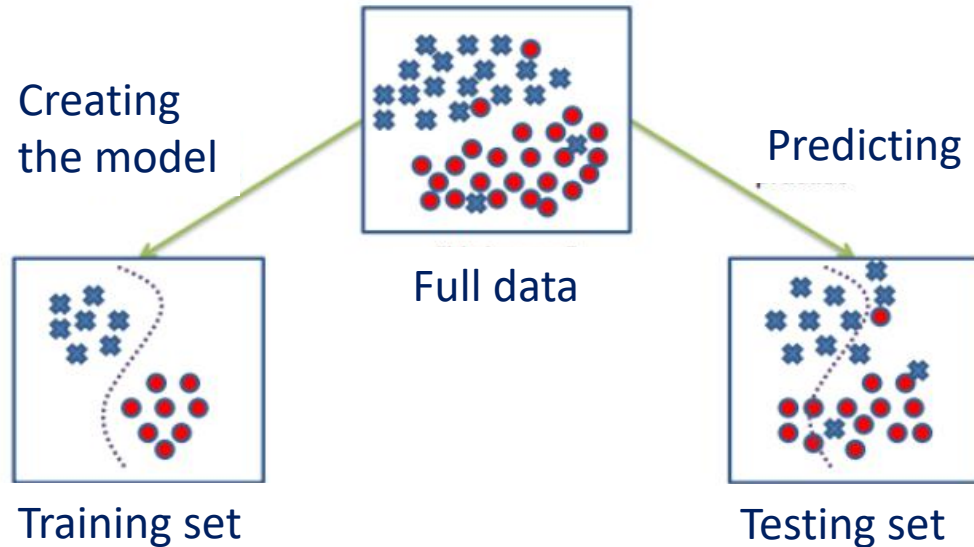
- The one whose parameters minimize the cost function
- The lower the J , the better the hypothesis function

Minimizing the cost function allows obtaining the "best" hypothesis function

- It is more common to minimize using numerical methods (example: gradient descent)
- We want J to be continuous and convex (so that it has a global minimum)

How to create a ML model?

- Data could come from several sources:
 - Texts, Number, Images, Tables, Videos, etc.
- Splitting the data:



How to create a ML model?

Data division (holdout)

- Training set
 - Used to learn model parameters (and determine the best model)
- Test set
 - Used to perform the final performance test
 - This set should NOT be used to “fine-tune” the trained model.
 - It is used only at the end of the process

Cross-validation set

- Allows you to “fine-tune” the model (know what works best)
- It's like a "fake" test set



How to create a ML model?

- Idea: How well a model (h) behaves on test data that he has not seen during training

Training set (answers are known)



Test suite



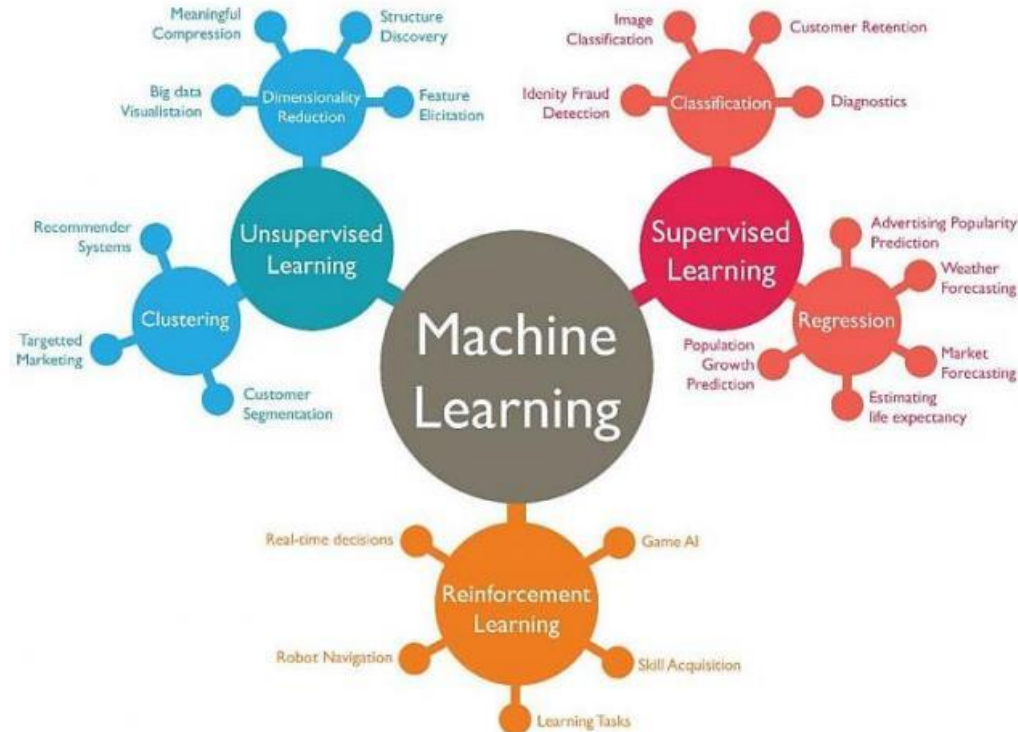
Types of Learning for ML

- Supervised (inductive) learning
 - Training data includes the correct outputs (“answers”)
 - Objective: find the prediction given the input
 - Types: regression, classification
- Unsupervised learning
 - Training data does not include correct outputs (“answers”)
 - Objective: discover similar patterns, structures, sub-spaces
 - Types: clustering, dimensionality reduction
- Reinforcement learning
 - Reward given a series of actions
 - Objective: try to learn using delayed feedback (reward)

Types of Learning for ML

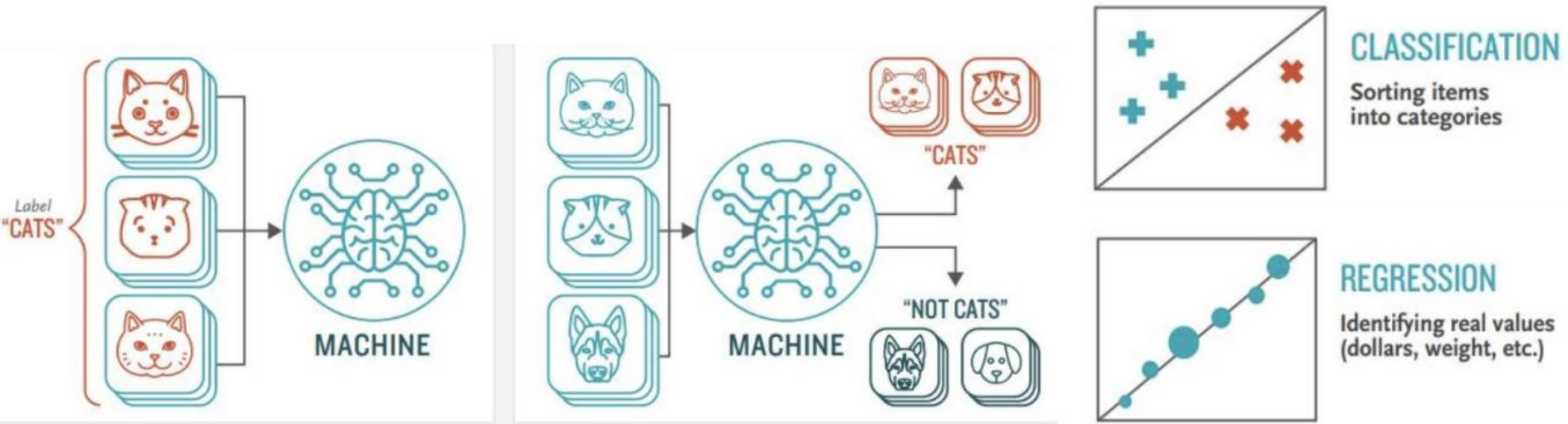
- **Supervised (inductive) learning**
 - Training data includes the correct outputs (“answers”)
 - Objective: find the prediction given the input
 - Types: regression, classification
- **Unsupervised learning**
 - Training data does not include correct outputs (“answers”)
 - Objective: discover similar patterns, structures, sub-spaces
 - Types: clustering, dimensionality reduction
- **Reinforcement learning**
 - Reward given a series of actions
 - Objective: try to learn using delayed feedback (reward)

Types of Learning for ML



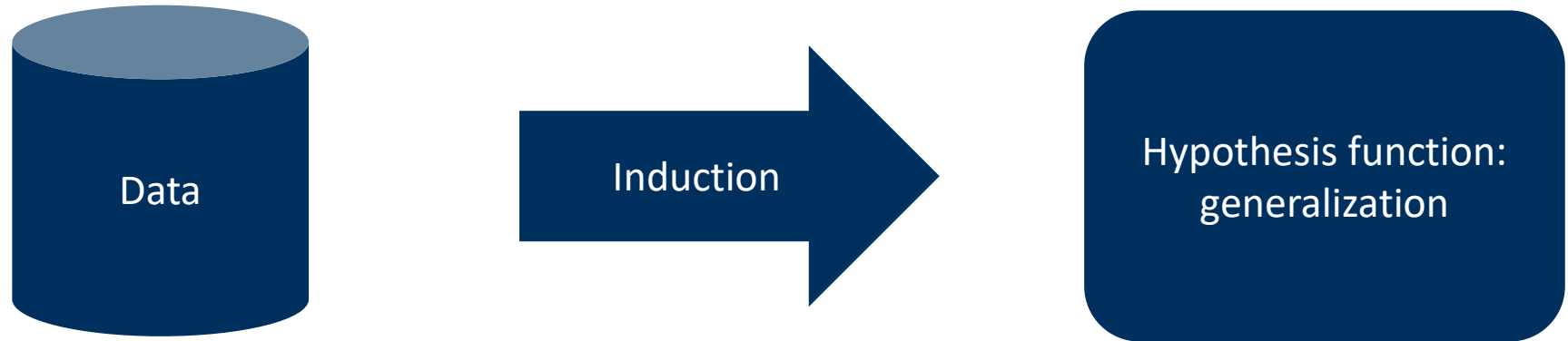
Supervised Learning

It is about training a model with categorized (labeled) data so that it can then be able to assign the corresponding classes/values to data never seen before.



Supervised Learning

The aim is to "induce" a hypothesis function based on past experience.



Supervised Learning

Input (x)

Output $f(x)$

1	2	3	4	5	6	7	8
1	4	9	16	25	36	49	64

What is the function that fits better the data?

Supervised Learning

Input (x)

Output f(x)

1	2	3	4	5	6	7	8
1	4	9	16	25	36	49	64

What is the function that fits better the data?

Hypothesis function: $f(x) = x^2$

What is the value of $f(12)$?

Supervised Learning

Input (x)	1	2	3	4	5	6	7	8
Output f(x)	1	4	9	16	25	36	49	64

Induction

Deduction



Hypothesis function: $f(x) = x^2$

Find out the relationship
between the input and
output

What is the value of $f(12) = 144$

Supervised Learning

Muffin or Chihuahua?

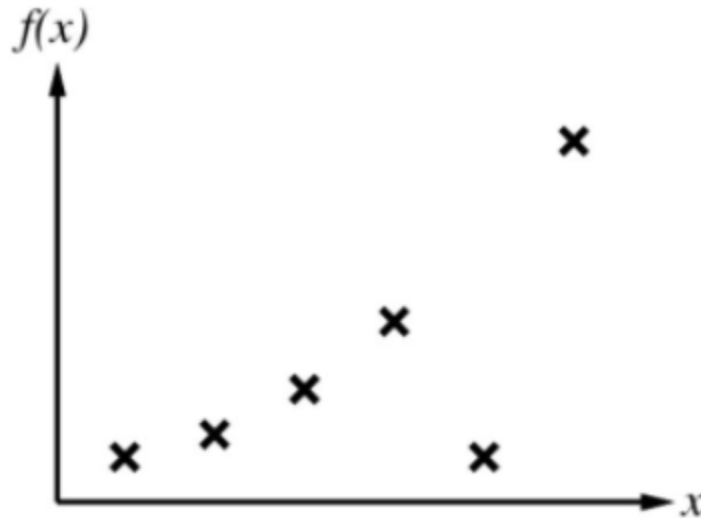
Machine learning algorithms must be able to “generalize.”

Perform the discovery of the hypothesis function (model) from of a particular set of data about the problem to be solved, and that can be used to make decisions or make predictions on new data never seen before.



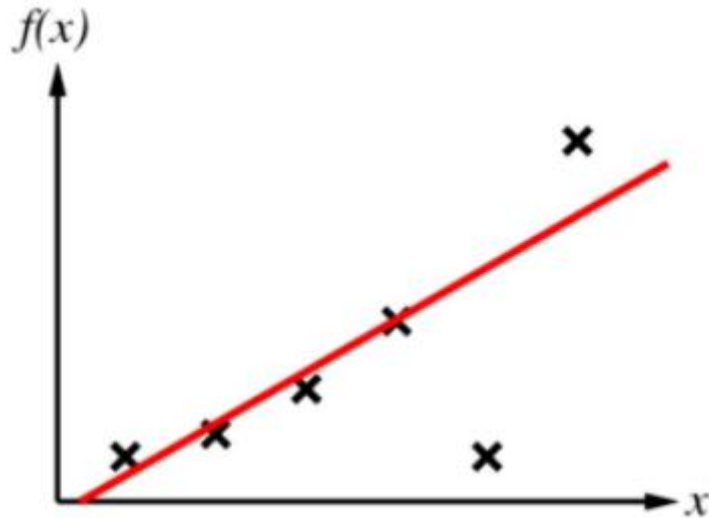
Supervised Learning

Search the hypothesis space for the most accurate hypothesis capable of describing the relationships between objects and that presents a better fit to the data.



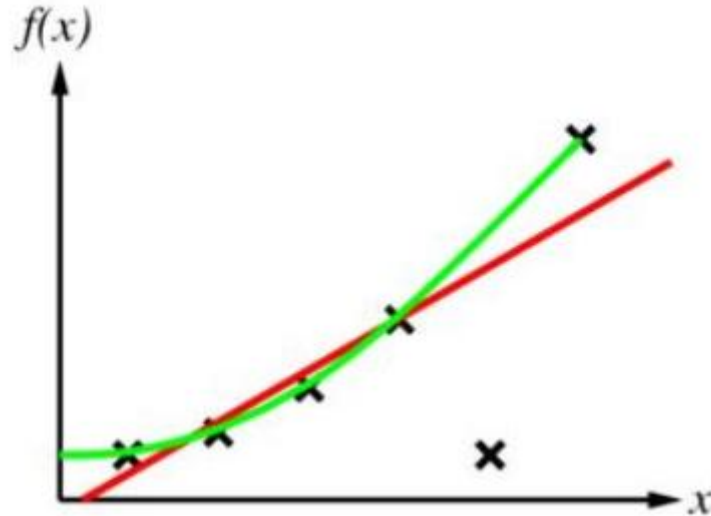
Supervised Learning

Search the hypothesis space for the most accurate hypothesis capable of describing the relationships between objects and that presents a better fit to the data.



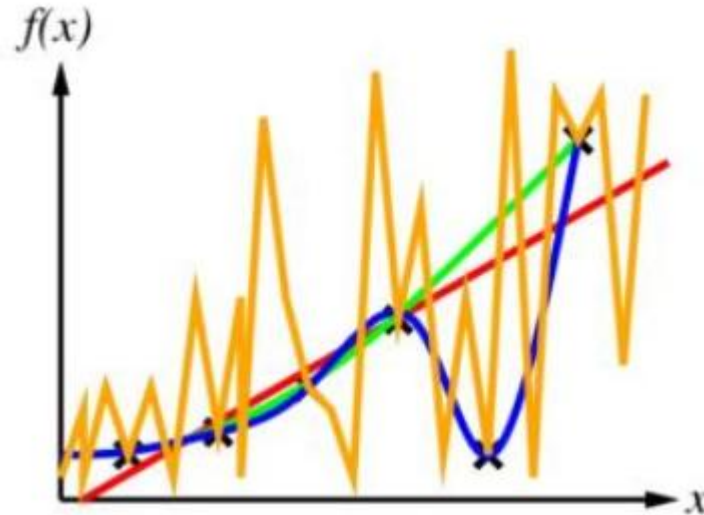
Supervised Learning

Search the hypothesis space for the most accurate hypothesis capable of describing the relationships between objects and that presents a better fit to the data.



Supervised Learning

Search the hypothesis space for the most accurate hypothesis capable of describing the relationships between objects and that presents a better fit to the data.



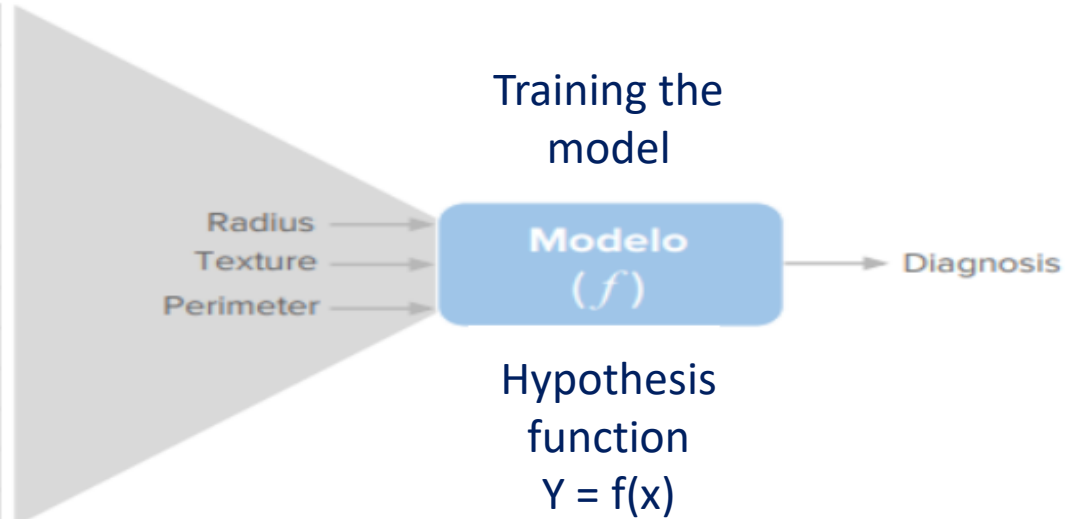
Supervised Learning

Data with labels

Radius	Texture	Perimeter	Diagnosis
14	23	94	M
15	28	97	M
15	20	95	M
16	21	108	M
20	22	130	M
15	14	102	M
9	21	55	B
9	14	59	B
9	21	60	B
11	19	72	B
9	17	59	B
13	16	81	B

Data
(input)

Labels
(output)



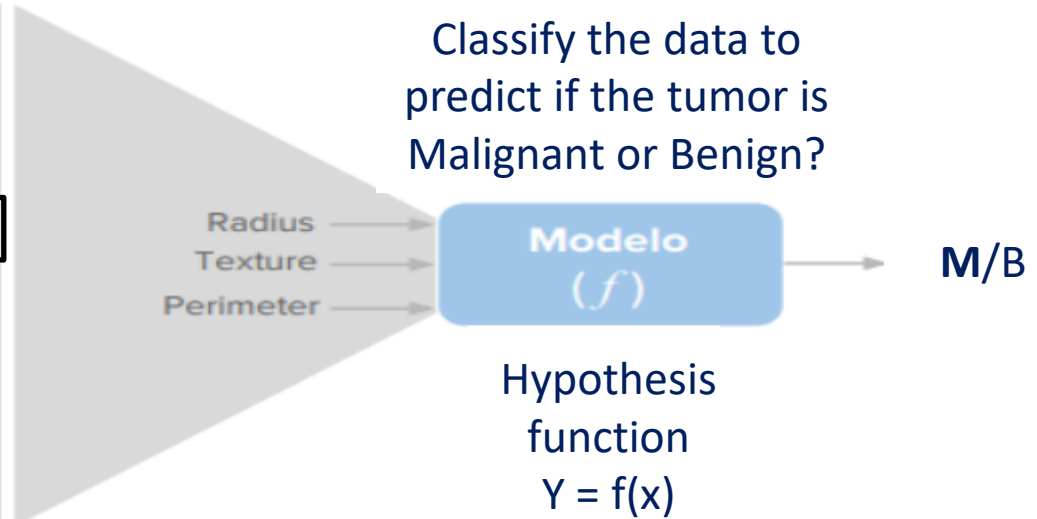
Supervised Learning

Data with labels

Radius	Texture	Perimeter	Diagnosis
14	23	94	M
15	28	97	M
15	20	95	M
16	21	108	M
20	22	130	M
15	14	102	M
9	21	55	B
9	14	59	B
9	21	60	B
11	19	72	B
9	17	59	B
13	16	81	B

Data
(input)

Labels
(output)



Supervised Learning

Data with labels

Radius	Texture	Perimeter	Diagnosis
14	23	94	M
15	28	97	M
15	20	95	M
16	21	108	M
20	22	130	M
15	14	102	M
9	21	55	B
9	14	59	B
9	21	60	B
11	19	72	B
9	17	59	B
13	16	81	B

Data
(input)

Labels
(output)

To the new data, this should be
Malignant or Benign?



Hypothesis
function
 $Y = f(x)$

New data

Radius	Texture	Perimeter	Diagnosis
17	23	98	?

Supervised Learning

It is then that from a set of training data (known examples), we train a model so that it is able to predict those new data based on the values of the input data.

Data with labels (we know the desired output)

Radius	Texture	Perimeter	Diagnosis
14	23	94	M
15	28	97	M
15	20	95	M
16	21	108	M
20	22	130	M
15	14	102	M
9	21	55	B
-	-	-	-

A given patient has a Benignant
or Malignant tumor?



Classification

A medium-size, rounded and green fruit?



Classification

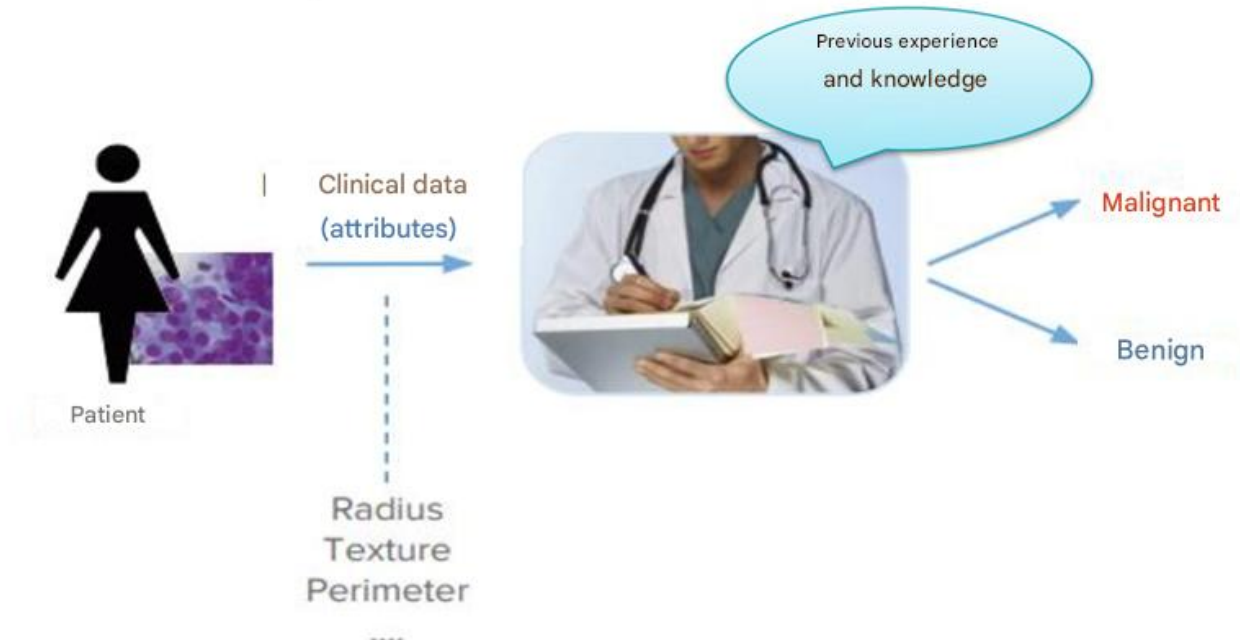
A medium-size, rounded and green fruit?

Assign a class (from a set of predefined classes) to an object or instance based on its defined attributes/characteristics.



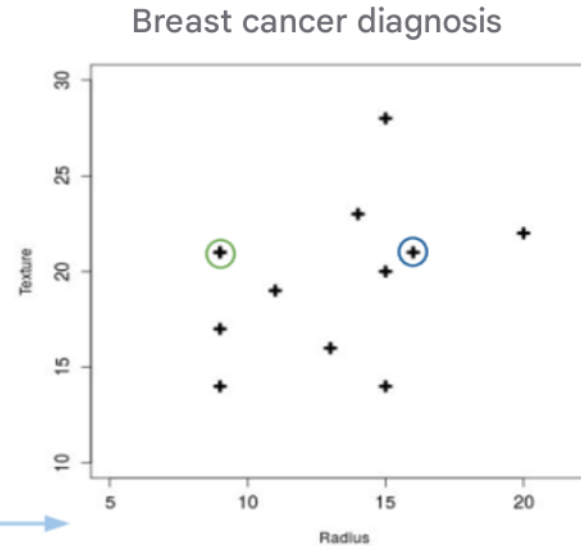
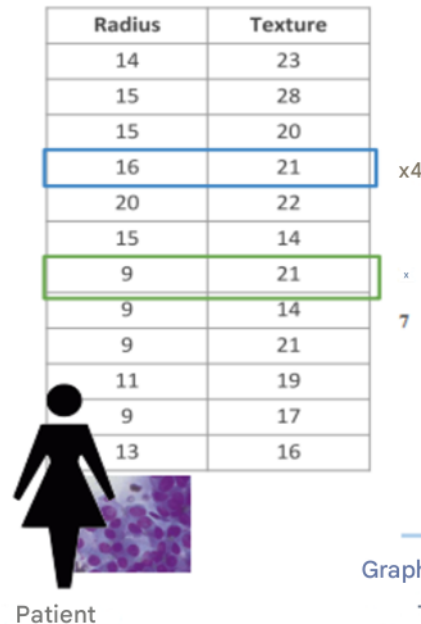
Classification

Attributes (Features) and classes (Labels)



Classification

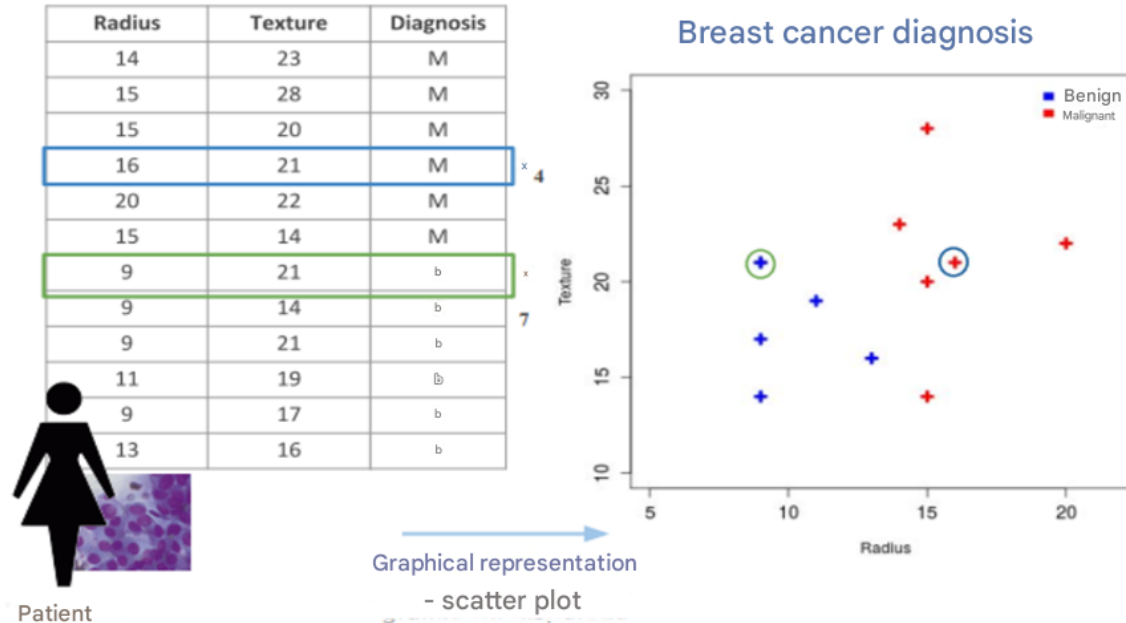
Assign a class (from a set of predefined classes) to an object or instance based on its defined attributes/characteristics.



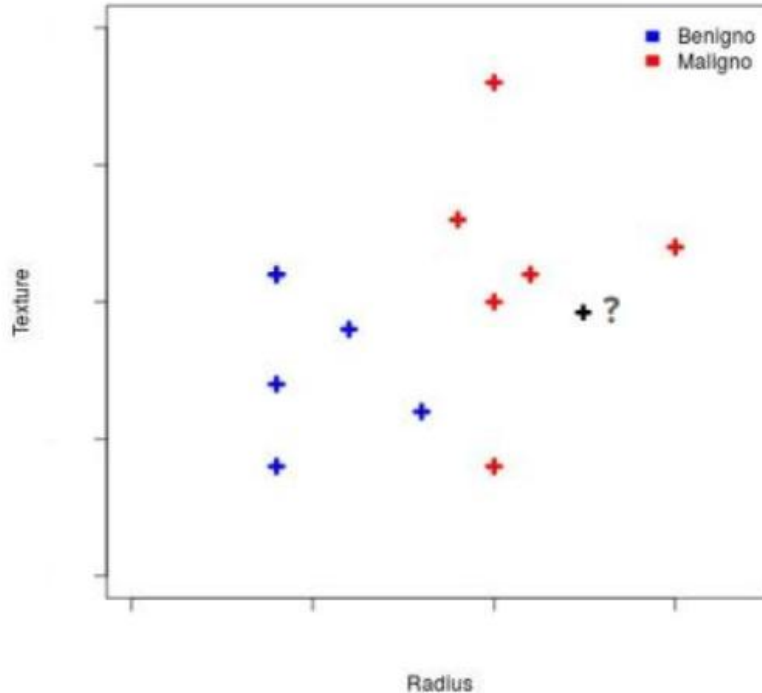
Graphical representation
- scatter plot

Classification

Assign a class (from a set of predefined classes) to an object or instance based on its defined attributes/characteristics.



Classification



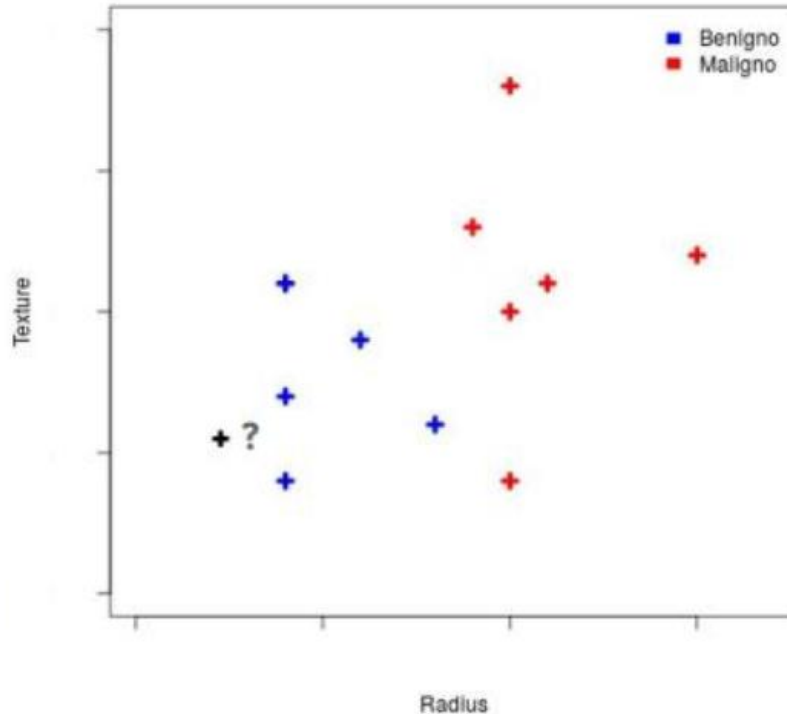
Main question: What we can say about a new instance, never before seen, based on the previous data?

Benign or **Malignant**?

Classification



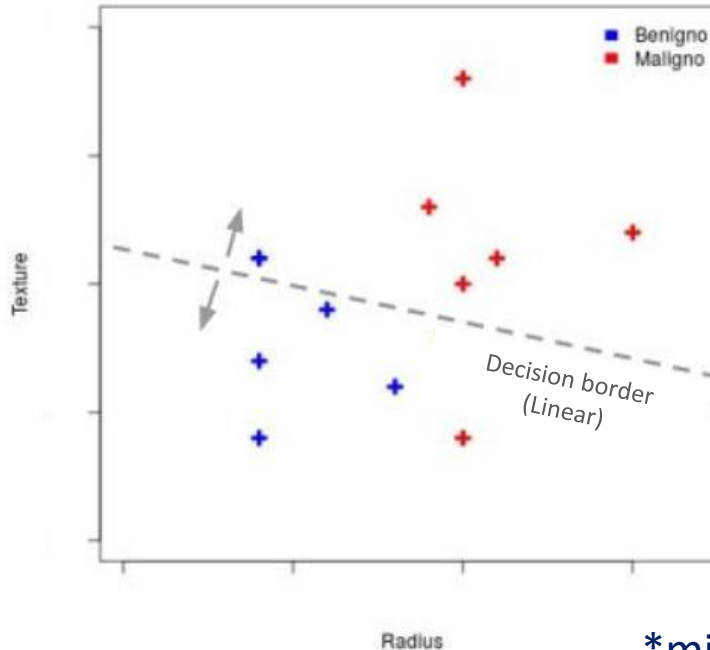
Paciente



Main question: What we can say about a new instance, never before seen, based on the previous data?

Benignant or **Malignant**?

Classification

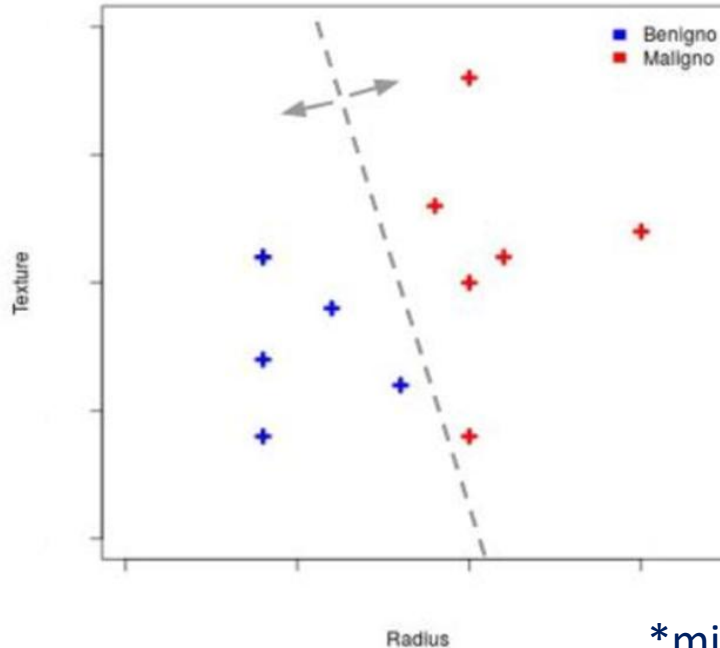


AM algorithms define a decision boundary seeking to correctly separate* the instances of each class: generalization

2 Prediction Errors

*minimizing the classification errors

Classification



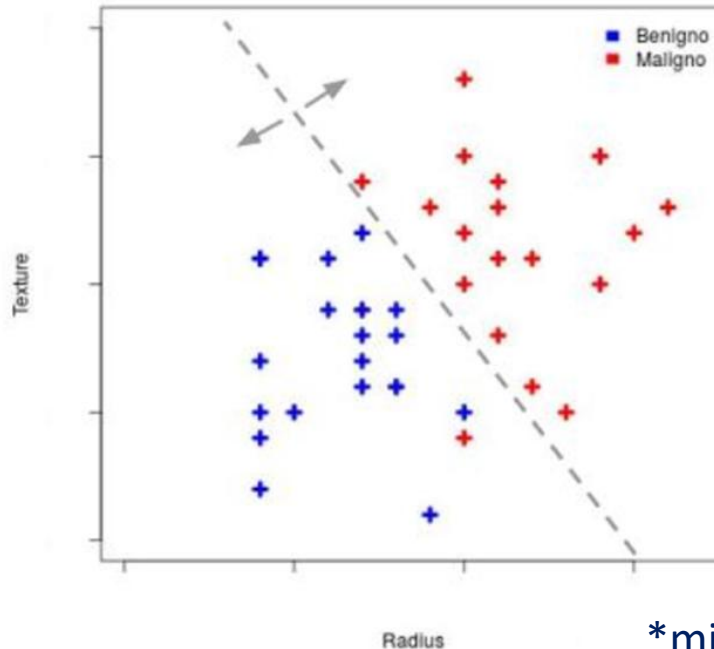
AM algorithms define a decision boundary seeking to correctly separate* the instances of each class: generalization

0 Prediction Errors

*minimizing the classification errors



Paciente



AM algorithms define a decision boundary seeking to correctly separate* the instances of each class:

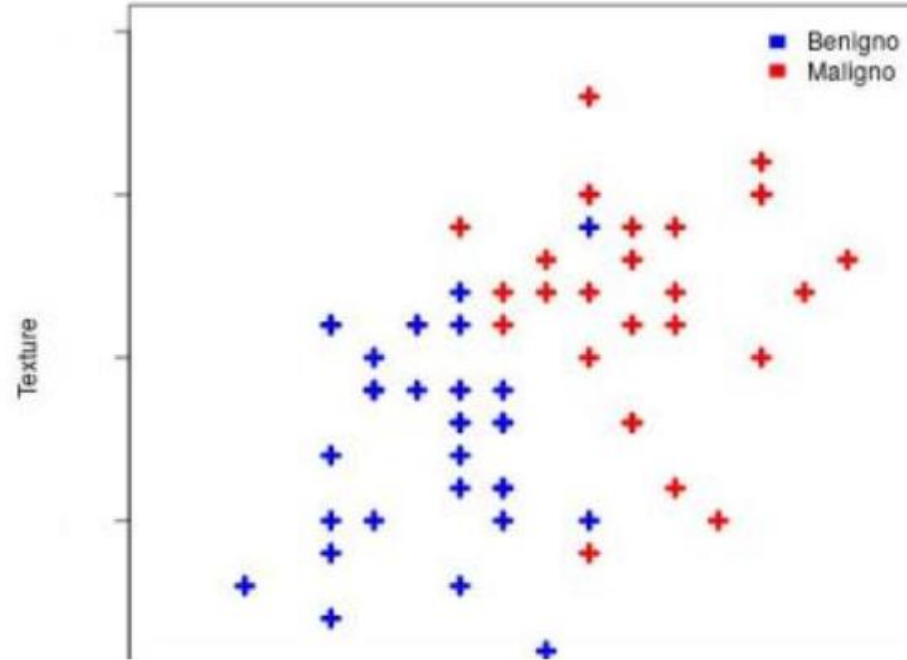
generalization

More data = update our decision border

1 Prediction Errors

*minimizing the classification errors

Classification



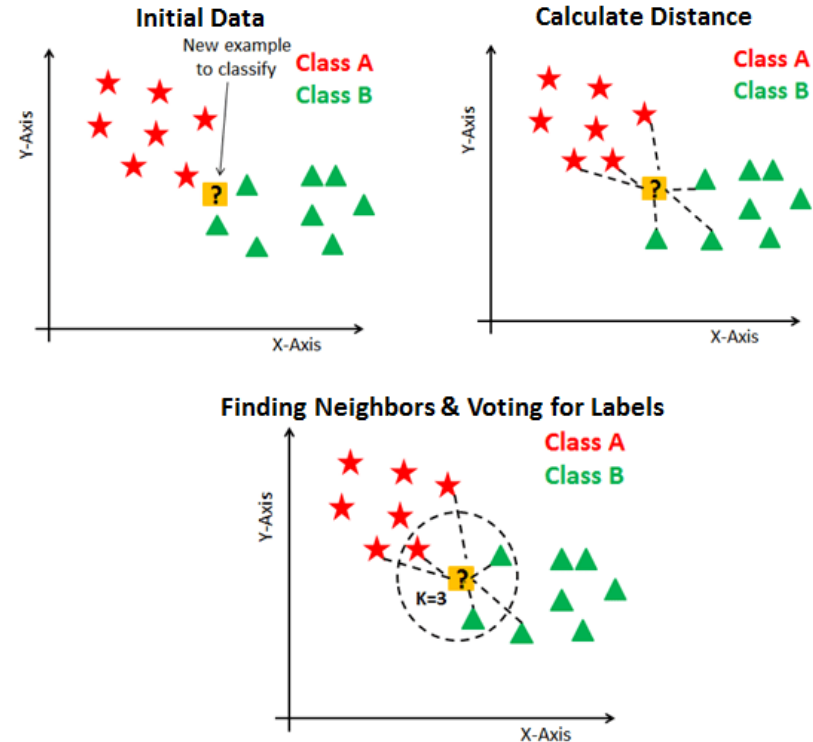
Sometimes the data is
not linearly separable

Machine Learning Classifiers for BCI

- K-Nearest Neighbors (KNN)
- Artificial Neural Networks (ANN)
- Naïve Bayes (NB).
- Support Vector Machine (SVM)
- Linear Discriminant Analysis (LDA).
- Decision Trees (DT)
- Ensembles

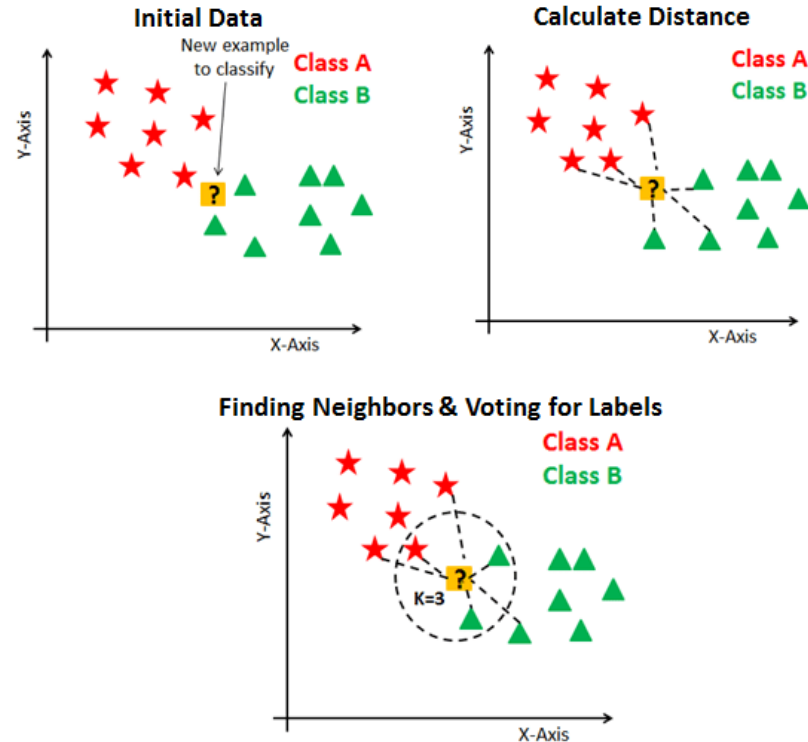
K-Nearest Neighbors (KNN)

KNN is a simple, intuitive, "instance-based" classifier. It does not create a mathematical formula to separate data; instead, it stores all training samples. When a new EEG segment needs classification, the algorithm looks at the **k** most similar samples (neighbors) in the feature space based on a distance metric. The new sample is assigned to the class that is most common among its neighbors.



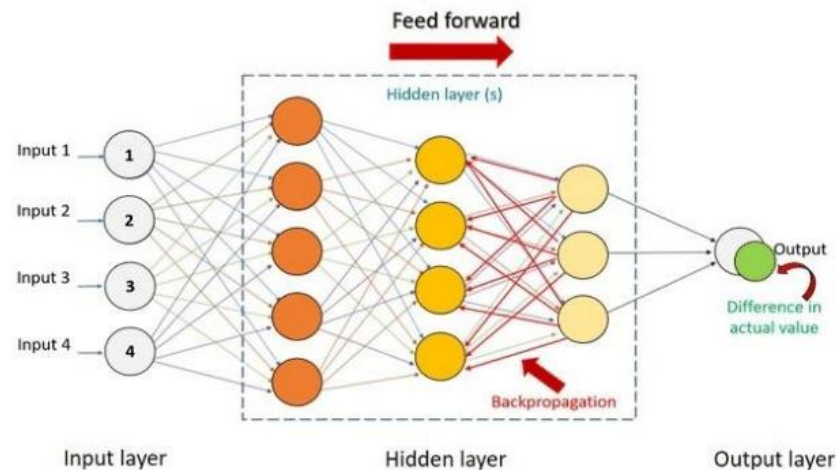
K-Nearest Neighbors (KNN)

Characteristics	Advantages	Disadvantages
Non-parametric: Makes no assumptions about data distribution.	Simple and intuitive; handles non-linear data naturally.	Slow classification: Must compare new data to all training points.
Uses distance metrics (e.g., Euclidean) to find similarity.	Easy to update the model with new training data.	High memory requirement; very sensitive to irrelevant features.



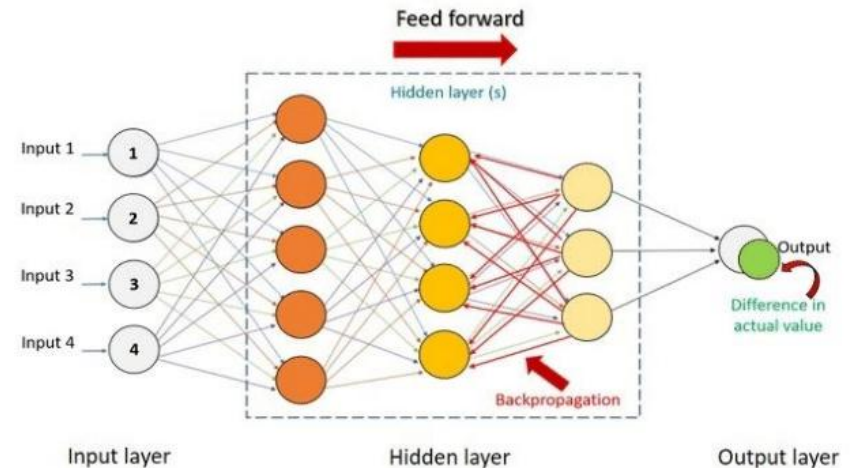
Artificial Neural Networks (ANN)

ANNs are inspired by the biological structure of the human brain. They consist of input, hidden, and output layers made of interconnected "neurons". Each connection has a weight that is adjusted during training. ANNs are excellent for BCI because they can learn to recognize extremely complex, non-linear relationships in EEG data that simpler linear models might miss.



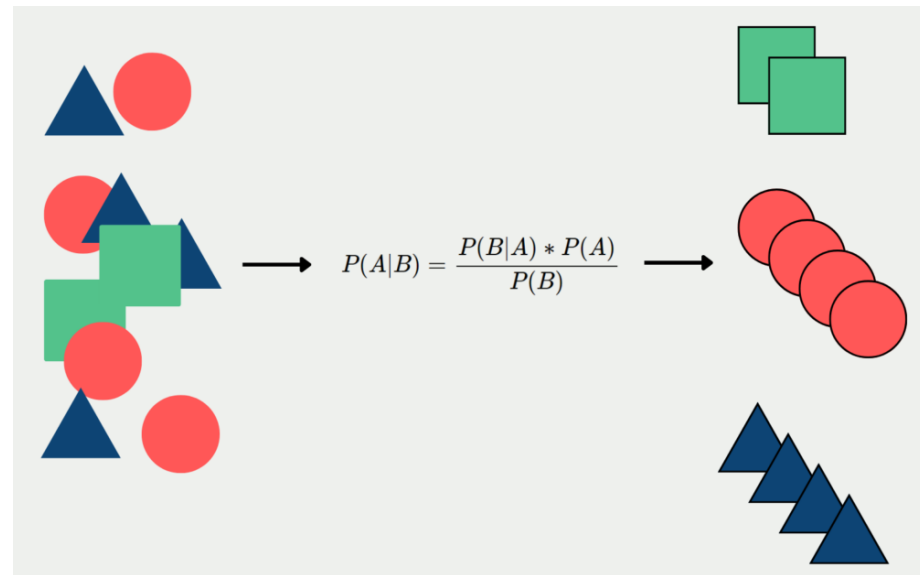
Artificial Neural Networks (ANN)

Characteristics	Advantages	Disadvantages
Processes data in parallel; learns from past events to make decisions.	Capable of modeling extremely complex, non-linear relationships.	"Black box" nature; poor interpretability of learned weights.
Can store information across the network, remaining functional if parts fail.	Gradually breaks down (fault tolerance) rather than failing suddenly.	Requires long training times and high-end hardware.



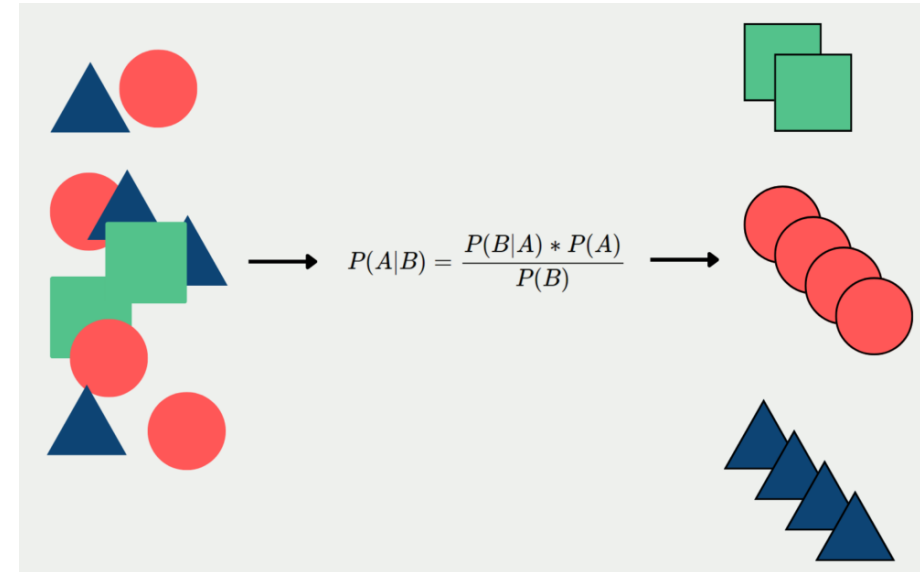
Naïve Bayes (NB)

Naïve Bayes is a probabilistic classifier based on Bayes' Theorem. It calculates the probability that a set of EEG features belongs to a specific brain state. It is called "naïve" because it assumes that every feature is completely independent of the others. Despite this oversimplification, it is incredibly fast and performs surprisingly well in BCI tasks with limited training data.



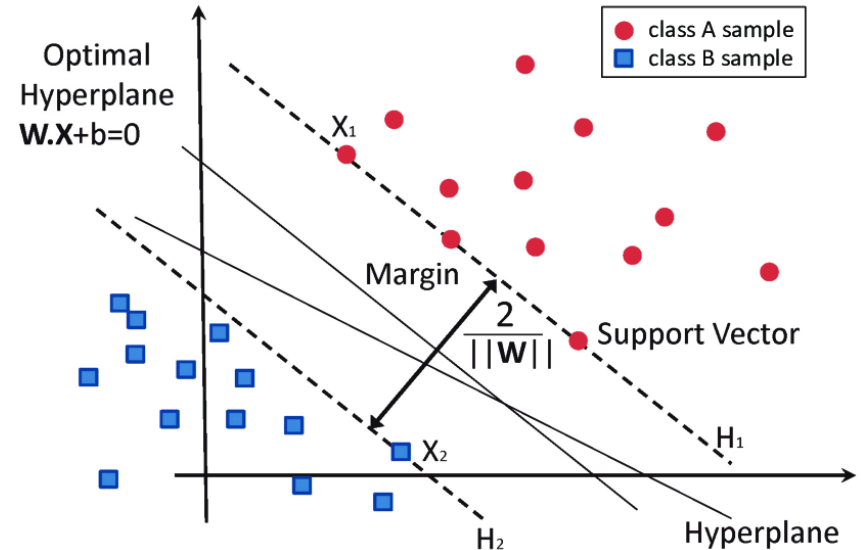
Naïve Bayes (NB)

Characteristics	Advantages	Disadvantages
Models the probability of a class given the input features.	Incredible speed; can be used for real-time BCI.	The independence assumption is rarely true for EEG features.
Requires very little training data compared to other models.	Robust to irrelevant features and missing data.	Struggles with categorical variables not seen during training.



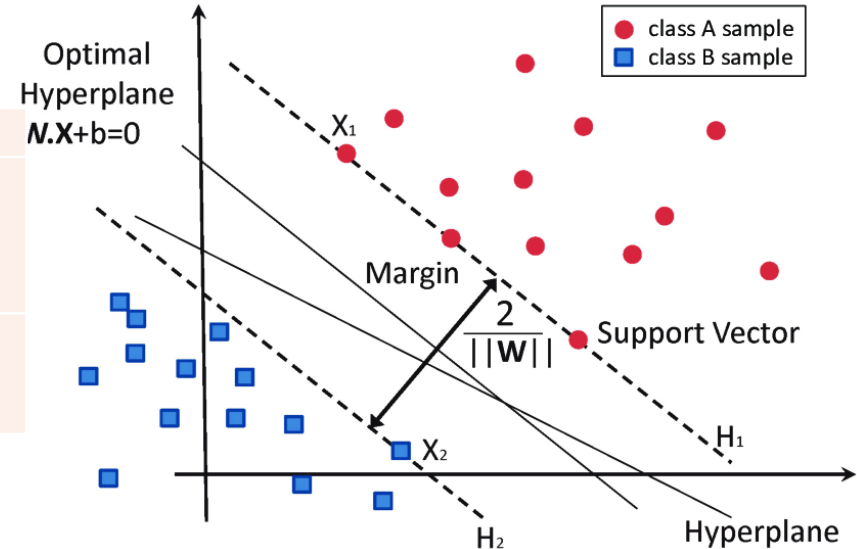
Support Vector Machine (SVM)

SVM is a powerful classifier that aims to find the "optimal hyperplane" to separate classes. Unlike other methods, it specifically focuses on the "Support Vectors"—the data points from each class that are closest to the decision boundary. The goal is to maximize the "margin" (the gap) between these vectors. For complex EEG data, it uses Kernels to project non-linear data into a higher-dimensional space where it can be easily separated by a straight plane.



Support Vector Machine (SVM)

Characteristics	Advantages	Disadvantages
Uses "Support Vectors" (points closest to the boundary) to define the model.	Excellent accuracy in high-dimensional spaces .	Slow training on very large datasets.
Can use kernels to find non-linear boundaries in higher dimensions.	Effective when dimensions exceed the number of samples.	Lacks transparency; difficult to interpret the "score".

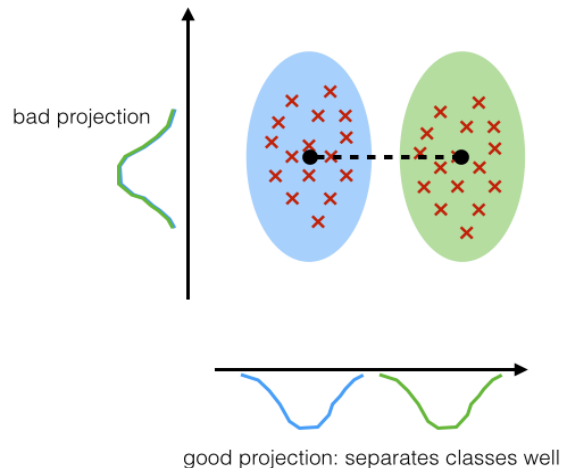


Linear Discriminant Analysis (LDA).

LDA is the most traditional and widely used classifier in BCI, particularly for motor imagery and P300 spellers. It works by projecting high-dimensional EEG features onto a lower-dimensional space. It seeks to find a linear boundary that maximizes the distance between the means of the classes while minimizing the variance within each class. This makes it highly robust against noise and very fast for real-time applications.

LDA:

maximizing the component axes for class-separation

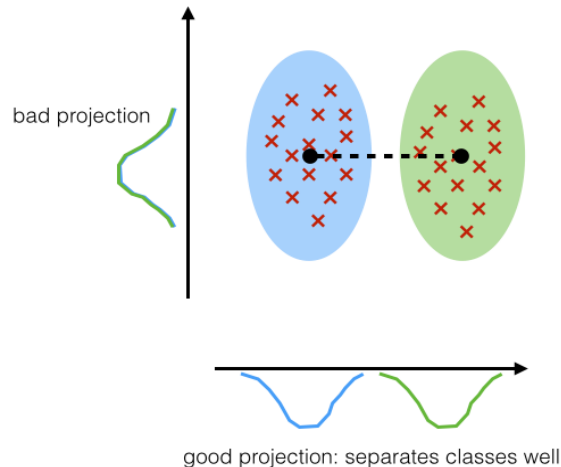


Linear Discriminant Analysis (LDA).

Characteristics	Advantages	Disadvantages
Generative model that projects high-dimensional data onto a 1D axis.	Extremely fast and simple to implement.	Assumes a Gaussian distribution of features.
Robust to correlated features.	Highly effective where features are larger than samples.	Only works well with linearly separable data.

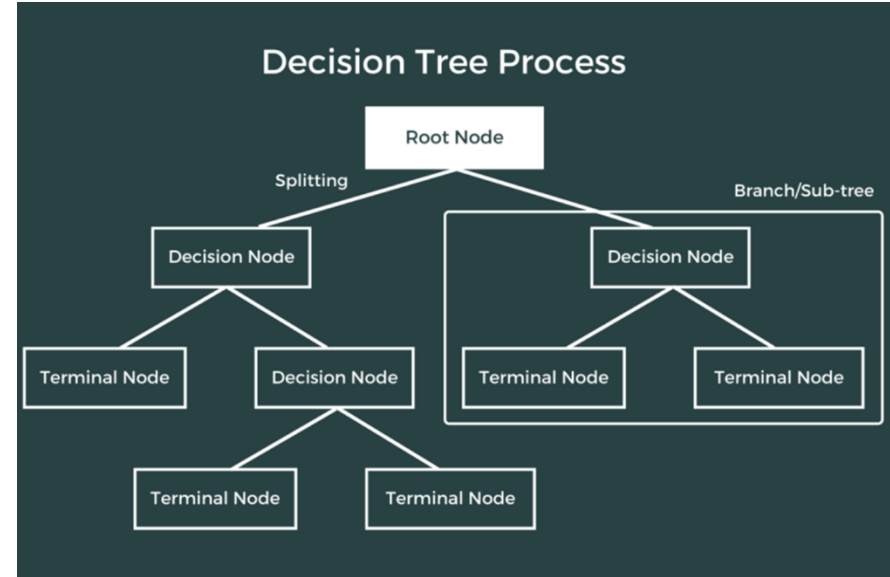
LDA:

maximizing the component axes for class-separation



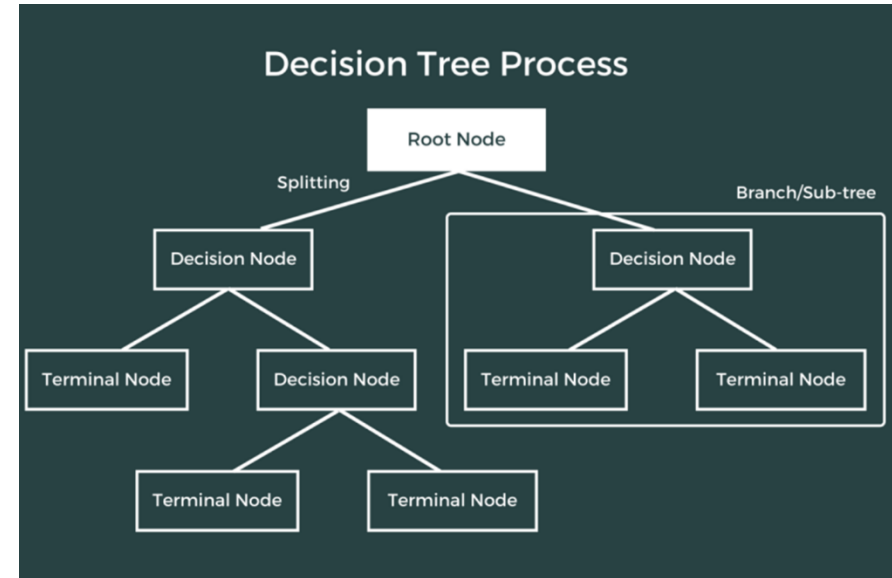
Decision Trees (DT)

A Decision Tree acts like a flowchart for classification. It breaks down a dataset into smaller subsets while at the same time an associated decision tree is incrementally developed. The final result is a tree with decision nodes (where a feature is tested) and leaf nodes (which represent the final classification).



Decision Trees (DT)

Characteristics	Advantages	Disadvantages
Non-parametric; produces a piece-wise function approximation.	High interpretability; easily visualized and explained.	Highly prone to overfitting if the tree is too deep.
Automatically performs feature selection during the splitting process.	Robust to outliers and requires minimal data preparation.	Unstable; small changes in data can create completely different trees.

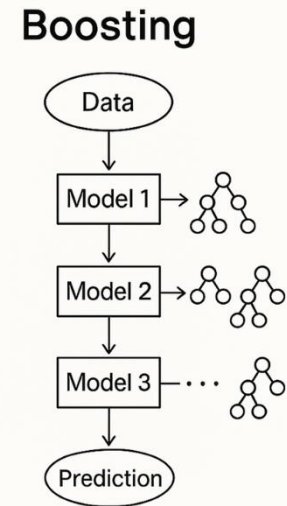
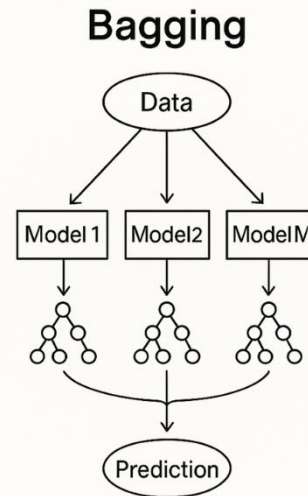


Ensemble methods are based on the "wisdom of the crowd". Instead of relying on one classifier, an ensemble combines several models to produce a single, more accurate prediction.

Bagging (e.g., Random Forest): Trains multiple trees independently and averages them to reduce error.

Boosting: Trains models sequentially, where each new model tries to correct the errors of the previous one.

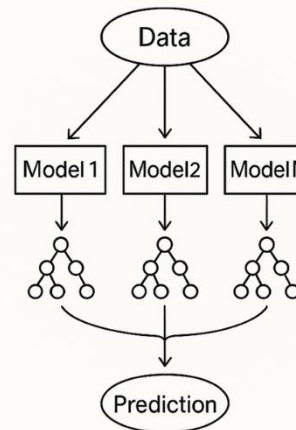
These are highly effective for EEG because they reduce the impact of noise and outliers.



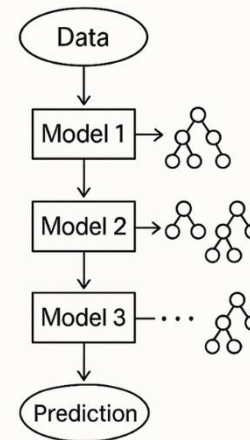
Ensembles

Characteristics	Advantages	Disadvantages
Uses techniques like bagging or boosting to reduce bias and variance.	Significantly improves performance and reduces overfitting.	Higher computational cost and memory usage.
Adaptive ensembles can track changes in non-stationary EEG signals over time.	More robust to the noise and subject variability common in BCI.	More complex to implement and validate across diverse users.

Bagging



Boosting



Machine Learning Classifiers for BCI

Method	Best For...	Interpretation	Speed
LDA	Simple/Real-time tasks	High	Very Fast
SVM	High-dimensional features	Low	Medium
KNN	Non-linear/Simple patterns	High	Slow (Testing)
NB	Small datasets	High	Instant
ANN	Complex/Large datasets	Low (Black box)	Slow (Training)
DT	Clear feature thresholds	Very High	Fast
Ensembles	Maximizing accuracy	Medium	Medium

Metrics for evaluating BCI performance

Evaluating BCI systems relies not merely in classification performance (accuracy, F1-score, confusion matrix) but also in how the system as a whole work during the interaction. Remember, we are creating an interface, and it implies a communication channel that should be evaluated accordingly to the user experience, accuracy and response time.

1. Accuracy and Confusion Matrix
2. Cohen's Kappa (κ)
3. Information Transfer Rate (ITR)

Accuracy and Confusion Matrix

While Accuracy (the percentage of correct predictions) is the easiest metric to understand, it can be misleading in BCI. If your dataset has 90% "Rest" data and 10% "Movement" data, a model that simply guesses "Rest" every time will have 90% accuracy but is completely useless as a BCI. To get the real picture, we look at the Confusion Matrix, which breaks down:

True Positives / True Negatives: The BCI correctly identified the brain state.

False Positives (Type I Error): The BCI triggered an action when the user didn't want one (e.g., typing a wrong letter).

False Negatives (Type II Error): The user tried to send a command, but the BCI missed it.

F1-Score: The harmonic mean of Precision and Recall. It is much more reliable than accuracy when your BCI classes are imbalanced.

Accuracy and Confusion Matrix

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

		Predicted Value	
Actual Value	Actual Value	True Positive (TP) The actual value is positive, and the model correctly predicts it as positive	False Positive (FP) The actual value is negative, but the model predicts it as positive
	True Negative	False Negative (FN) The actual value is positive, but the model predicts it as negative (Type II error)	True Negative (TN) The actual value is negative, and the model correctly predicts it as negative

In BCI research, Cohen's Kappa is often the preferred evaluation metric over standard accuracy. It measures the agreement between the BCI's predictions and the user's true intentions, but it subtracts the probability of the BCI guessing correctly by pure chance.

$k < 0$: Worse than random guessing.

$k = 0$: Exactly random guessing (chance level).

$k = 1$: Perfect classification.

Why BCI needs it: It provides a mathematically fair way to compare BCI models across different subjects and tasks, regardless of how imbalanced the classes are.

Cohen Kappa (k)

Kappa Statistic

$$k = \frac{2 * (TP * TN - FN * FP)}{(TP + FP) * (FP + TN) + (TP + FN) * (FN + TN)}$$

Assess the level of agreement between an actual and predicted

Predicted (rater 2) \ Actual (rater 1)	YES	NO	
	YES	NO	
YES	45 (TP)	15 (FN)	60
NO	25 (FP)	15 (TN)	40
	70	30	

@Danny Butvinik
The AI Vanguard
newsletter

Kappa Score Interpretation

Kappa	Agreement
<0	Less than chance agreement
0.01-0.20	Slight agreement
0.21-0.40	Fair agreement
0.41-0.60	Moderate agreement
0.61-0.80	Substantial agreement
0.81-0.99	Almost perfect agreement



$$k = \frac{2 * (45 * 15 - 15 * 25)}{(45 + 25) * (25 + 15) + (45 + 15) * (15 + 15)} = 0.13 \text{ (13\%)}$$

Information Transfer Rate (ITR)

ITR is the "gold standard" for measuring a BCI's real-world usability as a communication device. It calculates the amount of information the user can successfully transmit to the computer per unit of time, typically measured in bits per minute (bpm). ITR brings three critical variables into a single equation:

- N:** The number of possible commands (e.g., 2 for Motor Imagery, 36 for a keyboard speller).
- P:** The accuracy of the classifier.
- T:** The time it takes to make a single selection (speed).

Why BCI needs it: A highly accurate BCI that takes 30 seconds to type one letter might have a lower ITR than a slightly less accurate BCI that can type a letter every 3 seconds. ITR balances accuracy and speed.

Information Transfer Rate (ITR)

N: The number of possible commands (e.g., 2 for Motor Imagery, 36 for a keyboard speller).

P: The accuracy of the classifier.

T: The time it takes to make a single selection (speed).

$$B\left(\frac{Bit}{Trial}\right) = \log_2 N + P \times \log_2 P + (1 - P) \times \log_2 \left(\frac{1 - P}{N - 1}\right)$$

$$Q\left(\frac{Trials}{Min}\right) = \frac{S}{T}$$

$$ITR\left(\frac{Bit}{Min}\right) = B \times Q$$

Feature Extraction and Machine Learning for BCI

- Hands-on